博士学位論文
Doctoral Thesis

論文題目

Thesis Title

Pose Estimation of Non-Cooperative Spacecraft using Neural Networks with Restricted Computational Capacity

東北大学大学院工学研究科

Graduate School of Engineering,

TOHOKU UNIVERSITY

専攻/Department： Aerospace Engineering

学籍番号/ ID No： B9TD9117

氏名/Name： PRICE, Andrew Lawrence

指導教 員	吉田 和哉 教授
研究指導教員	吉田 和哉
（○印は主査）	○　吉田 和哉 教授 1　平田 泰久 教授 2　桒原 聡文　准教授 3　鏡 慎吾　　准教授　（情報科学研究科） 4　SALZMANN, Mathieu (主任研究員 EPFL)

Advising Professor at Tohoku Univ.	YOSHIDA Kazuya
Research Advisor at Tohoku Univ.	YOSHIDA Kazuya
Dissertation Committee Members Name marked with “○” is the Chief Examiner	○ YOSHIDA, Kazuya 1　HIRATA, Yasuhisa 2　KUWAHARA, Toshinori 3　KAGAMI, Shingo (Graduate School of Information Science) 4　SALZMANN, Mathieu (Senior Researcher, EPFL)

.

TOHOKU UNIVERSITY

Graduate School of Engineering

Pose Estimation of Non-Cooperative Spacecraft using Neural Networks with Restricted Computational Capacity

(計算能力制限下におけるニューラルネットワークに基づく非協力宇宙機の姿勢推定)

A dissertation submitted for the degree of Doctor of Philosophy (Engineering)

Department of Aerospace Engineering

by

Andrew Lawrence PRICE

3 November 2023

Pose Estimation of Non-Cooperative Spacecraft using Neural Networks with Restricted Computational Capacity

Andrew Price

Abstract

In-space rendezvous has been and will continue to be a critical capability in the advancement and maintenance of space infrastructure. The capability to remotely obtain the relative pose of a target is a core technological competency. More specifically, non-cooperative pose estimation is required for a future of asteroid exploitation, in-orbit construction, satellite repair and orbital debris removal. This work explores single camera, visible-spectrum spacecraft pose estimation.

Historically, spacecraft pose estimation systems have relied on conventional feature extraction methods such as edge and elementary shape detectors. Recent developments in neural network architectures show promise in feature extraction, pose estimation and even control capacities. However, a number of challenging barriers inhibit the adoption of neural network architectures into spacecraft systems. This work identifies the barriers, reviews relevant state-of-the-art research and then implements solutions to enable the straightforward incorporation of neural network architectures into spacecraft systems.

This dissertation contains five chapters. In **Chapter 1** the challenges of incorporating neural networks into spacecraft systems are delineated. Both terrestrial and in-space research related to (1) pose estimation training datasets, (2) pose estimation methodology, (3) neural network compression techniques, and (4) neural network stability is reviewed. In **Chapter 2** render theory is introduced and a complete end-to-end spacecraft render pipeline is developed and open-sourced. Several spacecraft datasets are rendered and a validation study comparing real images to rendered images is performed. The novel inclusion of physics-based sensor noise into the render pipeline is shown to improve the fidelity of the spacecraft renders. In **Chapter 3** multiple pose estimation methodologies are defined and explored. The first neural network pose estimation analysis of real deep-space images is performed on the deployment images of the JAXA Hayabusa Minerva-II2 rover. Results are compared against an independent JAXA study. In **Chapter 4**, neural network quantization theory is reviewed and, in collaboration with EPFL CVLab, a novel quantization strategy is introduced, validated and demonstrated. Accurate pose estimation is performed with a highly compressed neural network. Finally, in **Chapter 5**, this dissertation is summarized and the contributions of this work are defined.

Contents

1 Introduction **1**

1.1 Motivation . . . 1

1.2 Related Work . . . 3

1.2.1 Pose Estimation Datasets . . . 3

1.2.2 Pose Estimation . . . 5

1.2.3 Neural Network Compression . . . 7

1.2.4 Neural Network Stability . . . 9

1.3 Research Objectives . . . 9

1.4 Outline . . . 10

2 Spacecraft Pose Estimation Datasets **11**

2.1 Obtaining Spacecraft Imagery . . . 11

2.1.1 Real Imagery . . . 12

2.2 Render Pipeline . . . 13

2.2.1 Model and Material Properties . . . 13

2.2.2 Orbit and Pose . . . 17

2.2.3 Rendering . . . 21

2.2.4 Post Processing . . . 23

2.3 Render Pipeline Results . . . 28

2.3.1 Cubesat Dataset . . . 28

2.3.2 Rocket Body Dataset . . . 32

2.3.3 Modified Minerva-II2 Dataset . . . 33

2.3.4 Minerva-II2 Dataset . . . 34

2.3.5 Multi-Object Dataset . . . 36

2.4 Summary . . . 36

3 Spacecraft Pose Estimation **39**

3.1 Monocular Pose Estimation Strategies . . . 39

3.1.1 Correspondence Identification . . . 40

3.1.2 Correspondence Fitting . . . 44

3.1.3 Straight-Shot Approach 44
3.2 Minerva-II2 Pose Estimation Experiments 45
3.2.1 Minerva-II2 Problem Definition 46
3.2.2 Conventional Correspondence Matching Approach 47
3.2.3 CNN Feature Extractor Approach 50
3.2.4 Straight-Shot CNN Approach 54
3.2.5 Results and Comparison 55
3.3 Rocket Body Pose Estimation Experiments 58
3.3.1 CNN Feature Extractor Approach 59
3.3.2 Straight-Shot CNN Approach 59
3.3.3 Architecture Comparison Note 61
3.4 Summary 61

4 Neural Network Quantization **63**
4.1 Quantization Theory 63
4.1.1 Quantization Fundamentals 63
4.1.2 Incremental Network Quantization (INQ) 65
4.1.3 Module-wise Quantization (MQ) 67
4.2 Quantization Experiments 70
4.2.1 EPFL CVLab MQ Experiments 70
4.2.2 MQ Comparison to Spacecraft Quantization Literature 72
4.2.3 Quantization Applied to the Rocket Body Dataset 73
4.3 Summary 76

5 Conclusions **81**
5.1 Summary 81
5.2 Statement of Contributions 83
5.3 Ongoing Work 84
5.4 Future Scope 84

Bibliography **87**

Acknowledgements **99**

List of Figures

1.1 Spacecraft Pose Estimation Loop 2
1.2 CNN Architecture Comparison [1] 6

2.1 In-Orbit, Before-Launch (©Wikimedia) and Rendered Images of the Hubble Space Telescope 11
2.2 Real Spacecraft Images from Different Regions 12
2.3 Spacecraft dataset Render Process Flow 14
2.4 Minerva-II2 Symmetry, the *Original* Render Model, and the *Modified* Render Model 15
2.5 BSDF, a Generalization of the BRDF and BTDF (©Wikimedia) 15
2.6 Simulated Polar Orbit During the Summer 18
2.7 Sunlight Vector Ephemeris Plot 19
2.8 Tumble Integrator Example Visualization 20
2.9 Sampling SO(3) Rotation Space for a Uniform Rotation Distribution 21
2.10 Render Camera FoV Frustum Visualization 22
2.11 Example Earth Background Images 24
2.12 Cropped Raw Deployment Images of the Minerva-II2 Rover ©JAXA 25
2.13 Noise Sources in Spacecraft Imagery 25
2.14 Simplified CCD Sensor Schematic 26
2.15 The SRL Dark Room Air-Bearing Testbed and the Cubesat Physical Model 29
2.16 $\widehat{BRDF}$ Manual Tuning 30
2.17 Real and Rendered Cubesat Images without Post Processing Noise Included 30
2.18 Real and Rendered Cubesat Images with Post Processing Noise Included . . 31
2.19 Cubesat Real vs Render Channel Pixel Intensity Histograms Top: no post processing Bottom: with post processing sensor saturation 32
2.20 Rocket Body at Different Locations in Orbit 33
2.21 Rocket Body with Different Sunlight Intensities 33
2.22 Rocket Body Conic Motion 34
2.23 Minerva-II2 Deployment Sun Vector Estimation Left: Sun Geocentric Frame Right: Ryugu Asteroid Local Frame 35

2.24 Modified (asymmetric) Minerva-II2 Render Images of an Integrated Tumble 35
2.25 Unmodified (symmetric) Minerva-II2 Renders and Real Image Comparison 35
2.26 Multi-Object Dataset Renders 37

3.1 Idealized Hand Crafted Feature Selection on a Real Minerva-II2 Image . . . 40
3.2 Infinite Re-projection Solutions for a Line Segment 41
3.3 Generic Keypoint Extraction CNN Architecture 42
3.4 Minerva II2 Keypoints 43
3.5 Keypoints Projected onto Heatmaps for Loss Functions 43
3.6 Example Straight-Shot CNN Architecture [2] 45
3.7 JAXA Minerva-II2 Rover Deployment Summary [3] 46
3.8 Minerva-II2 Rover Deployment Telescopic Camera Images [3] 46
3.9 Minerva-II2 Rover Deployment Wide Angle Camera Image Collage © JAXA 47
3.10 Simplified Camera Trigonometry for a Sphere 48
3.11 Minerva-II2 Localization by Hand 48
3.12 Location of Target in Camera Cylindrical Coordinates 49
3.13 Image 2 Minerva-II2 Edge Detector + Hough Transform Comparison . . . 51
3.14 Image 2 Pose Fit Example (Manual Supported PnP) 51
3.15 Network Model Training Duration vs the Number of Identified Keypoints Summed Across All Test Images 52
3.16 Network Model Training Duration Keypoint Output Comparison 53
3.17 CNN FE Axis of Symmetry Estimations (Blue) for W2 Deployment Images 57
3.18 CNN Feature Extractor Pose Estimation Results for an Asymmetric Target . 58
3.19 Tumble Progression. Ground Truth (GT) vs the Network Model Estimate (Est.) 59
3.20 Schematic Representation of the Rocket Body Straight-Shot Output Vector . 60

4.1 Illustration of the Quantization Mapping Function for $Q^2(x)$ and $Q^3(x)$ . . . 64
4.2 Straight Shot Estimator 65
4.3 INQ Quantization Scheduling 67
4.4 EPFL CVLab Order Optimality Grid Search [1] 71
4.5 EPFL CVLab MQ Comparison to State-of-the-Art [1] 71
4.6 Pre-Rendezvous Pose Estimation Scenario 73
4.7 Example Rocket Body Random Pose Ground Truth and Estimation 74
4.8 Post-Rendezvous Pose Estimation Scenario 75
4.9 First 200 Straight-shot CNN Network Predictions 77
4.10 First 200 Straight-shot CNN Network Prediction Errors 78

List of Tables

2.1 Render Dataset Summary 28
2.2 Battacharya Distance Between the Real and Rendered Images (lower is better) 32

3.1 Minerva-II2 Deployment Velocity Estimation Results 55
3.2 Image 2 Axis of Symmetry Estimation Comparison 56
3.3 Ground Truth Vector Encoding. IP - image plane WF - world frame 60
3.4 Comparison of Pose Estimation Methods 62

4.1 MQ Applied to CA-SpaceNet vs CA-SpaceNet Published Quantization . . 72
4.2 Quantized Neural Networks Evaluated on the Pre-Rendezvous Scenario . . 74
4.3 Quantized Neural Networks Evaluated on the Post-Rendezvous Scenario . . 75
4.4 Quantized Network Prediction: θ_x Motion Fit 78
4.5 Quantized Network Prediction: θ_z Motion Fit 78

List of Acronyms

ACT Advanced Concept Team

ADD Average Distance Distinguishable

ADI Average Distance Indistinguishable

BRDF Bidirectional Reflectance Distribution Function

BSDF Bidirectional Scattering Distribution Function

BTDF Bidirectional Transmittance Distribution Function

CAD Computer-Aided Design

CCD Charge-Coupled Device

CMOS Complementary Metal Oxide Semiconductor

CNN Convolutional Neural Network

CoM Centre of Mass

CRD2 Commercial Removal of Debris Demonstration

CVLab Computer Vision Laboratory

DoF Degree of Freedom

ECI Earth Centered Inertial

EKF Extended Kalman Filter

EPFL École Polytechnique Fédérale de Lausanne

EPnP Efficient Perspective-n-Point

ESA European Space Agency

FE Feature Extractor

FoV Field of View

FPGA Field Programmable Gate Array

FPN Feature Pyramid Network

GEO Geostationary Equatorial Orbit

GNC Guidance Navigation and Control

GPU Graphics Processing Unit

HAWQ Hessian AWare Quantization

INQ Incremental Network Quantization

ISS International Space Station

JAXA Japanese Aerospace Exploration Agency

LEO Low Earth Orbit

LiDAR Light Detection And Ranging

LSQ Learned Step-size Quantization

MLI Multi-Layer Insulation

MSE Mean Squared Error

MQ Module-wise Quantization

NASA National Aeronautics and Space Administration

NN Neural Network

ONC Onboard Navigation Camera

PCB Printed Circuit Board

PnP Perspective-n-Point

QAT Quantization Aware Training

ResNet Residual Network

RGB Red Green Blue

RGBD Red Green Blue Depth

RMSE Root Mean Squared Error

SLAB Stanford Space Rendezvous Laboratory

SO(3) Special Orthogonal group

SPEED Spacecraft Pose Estimation Dataset

SRL Space Robotics Laboratory

STE Straight Through Estimator

SVD Singular Value Decomposition

TLE Two Line Element set

TRON Testbed for Rendezvous and Optical Navigation

W2 Wide Angle 2

WDR Wide Depth Range

Chapter 1
Introduction

The first spacecraft proximity operation was performed by Gemini 6 and 7 in 1965. One of the primary goals of the Gemini program was "to understand how spacecraft could rendezvous and dock in orbit around the Earth and the moon" [4]. Spacecraft rendezvous was critical during the Apollo era; was critical during the repair of the Hubble Telescope [5]; is critical today for satellite repair [6]; and will continue to be critical for a future with in-orbit construction, asteroid exploitation and the removal of orbital debris.

A number of in-orbit rendezvous and capture demonstrations for cooperative targets have been flown or scheduled (e.g. ETS-VII, removeDEBRIS, Pheonix etc. [7, 8, 9, 10]), but the current impetus is a number of missions focused on non-cooperative targets (e.g. Clearspace, CRD2, Starfish etc. [11, 12, 13]). A non-cooperative target refers to a noncommunicable resident space object with an unknown motion and without cooperative features such as light emitting diodes or matrix barcodes. Thus, the capability to remotely obtain the relative position and orientation (i.e. *pose estimation*), is a core technological competency for all missions with a non-cooperative target.

Pose estimation can be accomplished with a variety of sensors including: monocular cameras, stereo cameras, Light Detection And Ranging (LiDAR) systems, hyper-spectral imaging or event based cameras. This work examines the monocular visible-spectral camera system. Unlike the other systems, monocular camera systems are ubiquitous throughout the space sector.

1.1 Motivation

Historically, spacecraft relative pose estimation is a closed-loop process [14, 15, 16] as shown in figure 1.1, consisting of:

1. An **Image Feature Extractor**, which relies on conventional methods such as edge detectors, to extract features.
2. A **Pose Solver**, which receives the previously identified features. Using a-priori information for an initial pose guess, the features are injectively correlated to known 3D features and a pose is estimated.

3. A **Navigation Filter**, which maintains a running estimation of the motion using the previously estimated pose and temporal data. This estimation is, in turn, used to provide an initial pose guess for previous steps.

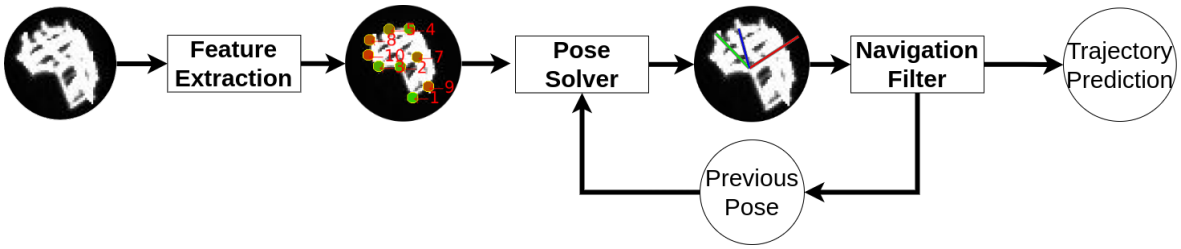


Figure 1.1: Spacecraft Pose Estimation Loop

Without appropriate a-priori information, the system may converge to an incorrect pose. In addition, these systems also require a-priori knowledge of features and consistent quality images to extract the features [15, 17]. In contrast, there has been an explosion of novel Convolutional Neural Network (CNN) pose estimation techniques for ground-based applications [18, 19, 20, 21, 22, 23, 24, 25]. While there have been some dedicated efforts applying CNNs to spacecraft pose estimation [2, 26, 27, 28, 29, 30, 31], CNN pose estimation solutions are still largely avoided by spacecraft systems for the following reasons:

1. **Limited Access to Spacecraft Pose Datasets**. Ground-based applications have access to enormous quantities of annotated data [32, 33, 34, 35, 36, 37, 38]. Although some spacecraft pose estimation datasets exist, they are often limited to a single model with an unshared render pipeline [28, 39, 40, 41, 42, 43]. Furthermore, real images are often expensive to obtain, security restricted and/or not annotated.
2. **Pose Estimation Environmental Context**. In contrast to ground-based imagery, spacecraft images can contain large spacial dynamic ranges, non diffuse and dynamic lighting, significant sources of noise, single channel data and a lack of background or planar contexts (i.e there is no ground-plane). Furthermore, the *unseen* instances of spacecraft are frequently more unique than ground-based counterparts (e.g. it can be reliably assumed that a car contains four wheels). Cars and humans have category level geometric similarities that can be exploited, whereas spacecraft geometry can vary widely.
3. **CNN Pose Estimation Network Size**. In general, ground-based applications do not experience the same hardware limitations prevalent in spacecraft systems. For this reason, the trend of state-of-the-art CNN architectures is towards larger and larger networks [1].
4. **Neural Network Stability**. Repairing or modifying a spacecraft is prohibitively expensive; consequently, spacecraft manufacturers rightly focus on mature, tested technologies with analytically defined stability bounds. Neural Network (NN) architec-

tures are viewed with cautious optimism by the Aerospace community, but largely still not adopted into mission critical systems.

The ground-based computer science community is vast and enabled by easy access to computers capable of running large, novel and complex NNs. It is thus the motivation of this work to address the aforementioned challenges and break down the barriers limiting the incorporation of state-of-the-art CNN pose estimation solutions into spacecraft applications.

1.2 Related Work

Relevant works corresponding to each of the four previously identified challenges are introduced respectively in sections 1.2.1–1.2.4.

1.2.1 Pose Estimation Datasets

Pose estimation datasets refer to compilations of images with annotated target location and attitude information. The majority of these dataset targets also have accompanying 3D geometric models available separately (i.e. the target object is already known or *seen*). Datasets can vary in target occlusion, lighting, number of targets and target symmetry.

General Pose Estimation Datasets

Ground-based datasets are numerous, benefiting jointly from relative ease of development and a vast diversified market.

The Linemod [33] and Occluded Linemod [34] datasets are a set of annotated real images of 15 low-texture household objects and their provided Computer-Aided Design (CAD) models. While each object's orientation and relative position to the environment is constant, the camera position (i.e. viewpoint) differs for each image. These datasets also mark the beginning of the BOP formatting for 6D pose estimation, a standardized reference baseline. The YCBV dataset [35] further expands to 77 household objects with changing environments. Graspnet [44] employs the usage of Red Green Blue Depth (RGBD) cameras to image 88 different objects in a variety of cluttered scenes.

In additional to static imagery, there are a number of sequential 6D pose video datasets. These datasets further support algorithms which can take advantage of prior pose information. The Google Objectron dataset [36] was one of the first large 6D pose annotated video clips; the dataset contains 15,000 video clips with hand annotated 3D bounding boxes. META AI [37] released their own CO3D dataset containing 19,000 video clips with annotated multi-view camera poses and point clouds. The Wild6D dataset [38] similarly contains 5,000 RGBD videos of different objects restricted to 5 categories.

To support in-house dataset development and in-pipeline rendering pose estimation methods, a number of CAD model repositories have been developed. Shapenet [45] was one of the first to deliver a large repository of 3D CAD models for synthetic image generation; at the time of writing, Shapenet now contains 51,300 unique 3D models grouped into 55 object categories. Objaverse [46] further recently released an 800,000+ 3D CAD model repository. Google additionally released a 1030 3D CAD model repository of real objects that were 3D scanned [47].

Finally, a few datasets were specifically curated to address the challenging unseen 6D pose estimation case. The Real275 dataset has 8000 annotated images of household items, formatted to train on specific instances[1] in a category, but to then test on different instances of the same category. The underlying assumption being that different instances of a single category share geometric similarities. More recently, Unseen6D released a dataset containing real and synthetic images with a total of 48 unseen objects in the test set [48].

Spacecraft Pose Estimation Datasets

In contrast to ground-based datasets, spacecraft imagery belongs to a more niche application with real annotated images being inordinately expensive to obtain.

SPEED was the first high fidelity spacecraft pose estimation dataset [39]. SPEED and later SPEED+ [40] utilized an openGL render pipeline informed by emperically obtained material properties to render 59,960 images. Furthermore, the SPEED team developed and built the Testbed for Rendezvous and Optical Navigation (TRON) to obtain real imagery of a spacecraft model, resulting in 9531 real laboratory images and 5 real in-space images of a single object.

Additional spacecraft datasets include:

- **URSO** [2], with Unreal[2] rendered synthetic images of the Soyuz and Dragon capsules for 6D pose estimation;
- **SwissCube** [28], with Mitsuba rendered synthetic images of a cubesat for 6D pose estimation.
- **SPARK** [41], with Unity3D rendered synthetic images of 11 spacecraft for object detection and one spacecraft for 6D pose estimation;
- **Synthetic Envisat** [49], with Cinema 4D rendered images of the Envisat spacecraft for 6D pose estimation;
- a **Segmentation** dataset [42], with Cycles rendered images for image segmentation;

[1] In the context of computer science literature, *category*-level refers to a type of object (e.g. a car) whereas *instance*-level refers to a specific instance of the object (e.g. a 2010 black Toyota Yaris).

[2] Unreal, Mitsuba, Unity3D, Cinema4D and Cycles all refer to different render engines supported by different entities.

- **Synthetic Cygnus** [50], with Cycles rendered images of the Cygnus spacecraft for detection and segmentation; and
- **SciFleet** [51], a 3D CAD repository of spacecraft models managed by the European Space Agency (ESA).

Considering the above limited set of available spacecraft imagery datasets, four observations are immediately apparent.

1. *Real* 6D pose annotated imagery is largely unavailable;
2. All of the 6D pose estimation datasets feature a singular object, with the exception of URSO (2 objects);
3. None of the 6D pose estimation datasets share a common render pipeline (indicating a limited sharing of knowledge); and
4. The 6D pose estimation datasets do not explore symmetric objects, unseen objects or high-noise imagery.

In contrast, ground-based datasets are more numerous, easy to access and well diversified to address a variety of problem formulations. It is clear that if spacecraft pose estimation is to benefit from the rapid development of ground-based pose estimation research, these spacecraft dataset limitations must be addressed.

1.2.2 Pose Estimation

Similarly to the dataset development, pose estimation architectures are largely driven by ground-based applications before being adapted to spacecraft systems.

General Pose Estimation Methods

Advancements in terrestrial pose estimation has revolved around CNN architectures. The initial CNN pose estimation networks can be classified as *straight-shot*, in that no post processing is performed after the CNN. PoseCNN [21] was an early straight-shot CNN architecture, and is composed of a backbone feature extractor which passes features to three heads: labelling, segmenting and pose regressing. SSD6D [24] reformats the pose regression into a classification problem, selecting from a discretized rotation space. A recent architecture, GDRnet [20], extracts a number of patches and 2D-3D correspondences. The correspondences are then fed to another CNN head which performs pose fitting in a Perspective-n-Point (PnP)[3] adjacent manner.

However, it has been generally demonstrated that CNN Feature Extractor (FE) architectures with post processing attitude fitting tend to outperform straight-shot CNNs. The CNN predicts 2D keypoints with known 3D correspondences, and then a pose fitting PnP solver is

[3]The fundamentals of the PnP problem formulation are introduced and addressed in section 3.

used [25]. Additional networks architectures further progressed this idea by selecting keypoints strategically to handle partial occlusions [23] and feature pyramid networks to handle large spatial dynamic ranges [22, 28].

Motivated by competitions and benchmarks, recent CNN FE architectures are moving towards larger and more complex architectures. Hybridpose [52] extracts multiple features including keypoints and edges before passing the results to a second stage for merging and pose estimation. SO-Pose [19] propose an even larger multi-stage framework: (1) an object detection framework passes a region of interest crop to the (2) encoder-decoder with parallel branches extracting keypoint features, a mask and self-occlusion maps to increase the network's robustness to real occlusion. The total network size of 292.9 million parameters provide 2D-3D correspondences that are then fed to a (3) PnP solver. Zebrapose [18] similarly adopt a multi-stage (1) detector (2) keypoint regressor (3) PnP solver approach with a final pipeline parameter count of 438.9 million parameters. While highly accurate, Zebrapose also require extensive preprocessing to build a parallel custom-rendered dataset with dense 2D-3D correspondences.

The trend of the above discussed CNN architectures is that of increasing size and slower throughput. For context, a comparison of different network architectures' parameter count and throughput are compared when tested on the Occluded Linemod 6D pose estimation dataset 1.2. In the figure, the circle size is proportional to the number of network parameters with single-stage and multi-stage architectures shown in green and orange respectively. It is clear there is a trade-off in network size and throughput for increased accuracy. In the context of an onboard spacecraft processor, it may not be feasible to deploy the most accurate network architecture.

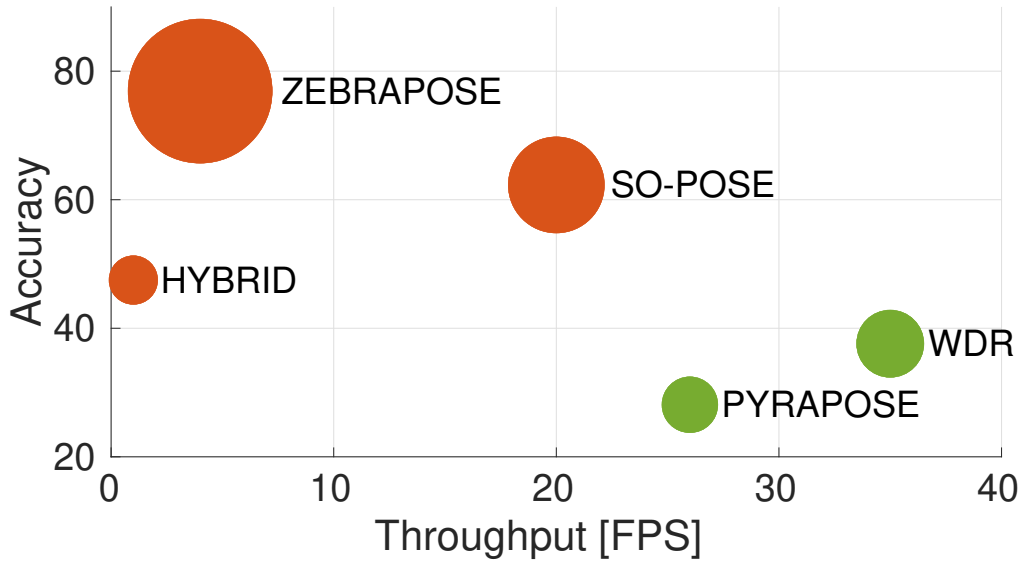


Figure 1.2: CNN Architecture Comparison [1]

Spacecraft Pose Estimation Methods

In contrast, current spacecraft pose estimation systems typically rely on: mature conventional feature extractors (e.g. a Canny line detector); an a-priori informed initial pose guess; and navigational filtering such as an Extended Kalman Filter (EKF). The section 1.1 challenges must be addressed before CNN architectures can be considered in future space missions. Notwithstanding, any pose estimation architecture *can* be applied to spacecraft imagery; in fact, there are a number of CNN architectures developed specifically for spacecraft imagery.

The straight-shot CNN, Ursonet [2] does not employ a post processing PnP step, but instead return to the [24] orientation classification idea. Being a good choice for limited spacecraft hardware, Mobile-Ursonet [30] build on Ursonet by experimenting with smaller network backbones for feature extraction.

The CNN FE network, WDR [28] employs the use of a feature pyramid network to account for the huge scale variations common to space imagery. [49] incorporate a CNN FE network with an EKF to perform continuous sequential pose estimation, leveraging uncertainty metrics from the post processing pose fitting step to inform the EKF.

1.2.3 Neural Network Compression

As showcased in figure 1.2, an increase in pose estimation accuracy has historically incurred a larger memory footprint and slower throughput. With high processing requirements being antithetical to spacecraft hardware and energy limitations, it becomes necessary to explore options to reduce CNN architecture size.

Overview of Neural Network Compression Methods

A number of network compression techniques exist and can be coarsely categorized into five groups [53]; a brief summary is provided here.

Weight Sharing refers to multiple layers within a NN using the same weights (e.g. when updating weights in layer a, the layer b weights are similarly updated). While weight sharing reduces network sparsity and the necessary memory to store weights, it is difficult for the implementation engineer to ascertain how to select which weights should be shared and to what degree the weight sharing should occur.

Pruning refers to the deletion of weights from layers in a NN. Pruning can be done randomly or by deliberate thresholding (e.g. removing weights with a low magnitude). Pruning is limited in its application to CNN type architectures because the CNN filter kernels cannot convolve across gaps in an array. Thus, entire filter kernels or input channels must be deleted if pruning is desired.

Tensor Decomposition refers to the reduction of NN tensors to a lower ranked tensor representation. An $m \times n$ sized tensor, $A \in \mathbb{R}^{m \times n}$, can be represented with two reduced rank

tensors, as $A = BC$ with $B \in \mathbb{R}^{m \times r}$ and $C \in \mathbb{R}^{r \times n}$. Naturally this incurs an approximation error[4]. Singular Value Decomposition (SVD) is a common tensor decomposition approach, but must be performed for each layer in the network. Again, the structure of a CNN does not lend itself well to this approach due to the convolving of 2D filter kernels across reduced tensor representations (i.e. spatial information is not maintained during SVD).

Knowledge Distillation refers to the training of a smaller application-specific NN with a larger high-performance NN acting in a supervisory role. The supervising NN may be used to label previously unlabeled application-specific data for the smaller NN as well. This compression technique is well explored for image classification applications, but less so for 6D pose estimation.

Quantization refers to the process of reducing the bit precision representation of parameters in a network. Typically, each network parameter is assigned 32 bits (or 64 bits) of memory to store the parameter. Quantization explores using smaller memory blocks (e.g. 8 bits, 4 bits etc.) to store the same paremeter. Quantization theoretically has no limitations on its application to a particular NN architecture or task[5].

Quantization Methods

From the above introduction of existing NN compression methods, it is clear that quantization is the most readily applicable to CNN architectures and 6D pose estimation applications. Quantization strategies are thus considered in this work. While techniques to apply quantization after network training exist [54], Quantization Aware Training (QAT) methods generally yield superior performance [1].

Quantization theory is not new, but quantization theory is receiving renewed interest due to the surge in NN related research. In Incremental Network Quantization (INQ), a bit precision is selected and a fraction of the network is quantized. INQ then alernates between retraining and increasing the fraction of the NN that is quantized. Fractional quantization can occur in a structured manner (i.e. entire kernels) or unstructured manner (i.e. partial kernels). Learned Step-size Quantization (LSQ) instead quantizes the entire network at once, but gradually. The network is not quantized to the desired bit precision directly, instead the network is quantized to an intermediate precision (i.e. intermediate *step*). INQ treats this intermediate precision *step* as a learnable hyperparameter when alternating between quantization and retraining.

In contrast to the previous *uniform* quantization methods, Hessian AWare Quantiza-

[4]The approximation error is typically expressed as the Frobenius norm, $e = \sqrt{\sum_i^m \sum_j^n |(A - BC)_{ij}|^2}$.

[5]It should be noted that a Graphics Processing Unit (GPU) typically stores parameters in 32 bit precision at runtime; the benefits of quantization may not be readily substantiated without access to hardware capable of exploiting different bit precisions. A spacecraft processor, such as a Field Programmable Gate Array (FPGA), is an example of hardware capable of exploiting the benefits of quantization.

tion (HAWQ) applies different bit precisions to different layers of the network. HAWQ uses the NN weights' Hessian[6] to infer a layer sensitivity (i.e. layers that experience higher magnitude updates during training are more likely to require a higher bit precision to maintain network performance). [55] further applies different bit precisions to individual filter kernels within a single NN layer. While these *mixed precision* quantization methods show promise, they are primarily only explored on image classification applications. Additionally, *mixed precision* methods are more difficult to implement during training; similarly, at deployment, it may not be simple or efficient to store sequential parameters with mixed-precisions.

1.2.4 Neural Network Stability

Spacecraft missions are prohibitively expensive, consequently the Guidance Navigation and Control (GNC) system stability must be rigorously proven, both empirically and analytically [56]. One such stability metric is that of Lyapunov stability. In the context of a spacecraft pose estimation system (see figure 1.1), a navigation filter, $s(t)$ is said to be Lyapunov stable if given an initial equilibrium state, s_{eq} and a perturbation, ε, then for all $t > 0$ we have $|s(t) - s_{eq}| < \varepsilon$ [57]. Notably, Lyapunov stable NN *controllers* (not feature extractors) are being demonstrated in earth-bound settings [58, 59].

Demonstrating the stability of spacecraft GNC is a mature and studied field for systems incorporating conventional feature extraction methods (i.e. not a NN). However, if a GNC system is to incorporate a NN FE, demontrating the GNC stability would require empirical and theoretical proof that the NN module outputs are bounded for all possible inputs, deterministic, and preferably converging to correct features. To the best of the author's knowledge, the stability of a spacecraft GNC incorporating NN modules has yet to be thoroughly demonstrated. In [49], the inclusion of a CNN FE enabled GNC system demonstrates good results, but the work recognizes a need to "assess the impact of perturbations on the accuracy and robustness of the navigation filter."

In addition to demonstrating the inherent stability of a NN FE, the topic of cybersecurity must also be addressed. The field of adversarial machine learning investigates how a NN will respond to a modification of its inputs [60]. This further emphasizes the need to demonstrate that a particular NN's outputs are bounded for spacecraft applications.

1.3 Research Objectives

The primary objective of this work is to break down the barriers and address challenges limiting the entrance and utilization of NN pose estimation architectures in spacecraft systems. Referring back to the four challenges identified in section 1.1, this work's objectives are:

[6] In this context, the Hessian refers to the partial derivatives of a particular 2D layer's weights during gradient descent back-propagation with respect to the loss function.

1. To create, validate and open-source a complete spacecraft image pipeline accounting for all the physics and characteristics specific to the space environment.

2. To demonstrate NN enabled pose estimation on the limit cases. A GNC system is designed to operate within the boundaries defined by challenging and limit cases. Therefore, a NN enabled pose estimator must be able to work with high-noise, resource-restrictions and non-ideal targets.

3. To explore and quantify a NN architecture's sensitivity to compression. Terrestrial based state-of-the-art NN architectures continue to grow in complexity and size. Spacecraft hardware cannot support this trend.

Although not explored in this work, NN stability is identified as another challenge to be addressed before adoption into spacecraft GNC systems.

1.4 Outline

This dissertation consists of five chapters, three of which address challenges identified in section 1.1.

Chapter 2 characterizes the challenges associated with obtaining spacecraft imagery. Image rendering theory, as applicable to the space environment, is discussed and a complete render pipeline accounting for material properties, target motion, environment and sensor-related noise is developed. A physical model / experimental setup is used to validate corresponding digital twin rendering. In support of future work, a number of datasets are created.

Chapter 3 introduces the different available pose estimation methods as applicable to non-cooperative resident space objects. The JAXA Hayabusa 2 Minerva-II2 deep space deployment case and images are introduced. Each of the identified pose estimation methods are applied to the challenging Minerva-II2 images and the performance is compared. Additional attention is given to the pose estimation methods' vulnerability to handling a symmetric target.

Chapter 4 describes quantization theory and the validation work performed in collaboration with the École Polytechnique Fédérale de Lausanne (EPFL) Computer Vision Laboratory (CVLab). Finally, a chosen quantization method is applied to two different CNN pose estimation architectures; the architectures' sensitivity to aggressive quantization and the pose estimation performance of the quantized networks is assessed.

Chapter 5 summarizes the developments of this dissertation. The contributions, both novel and practical, are delineated. Ongoing work is briefly discussed and future research directions are identified.

Chapter 2
Spacecraft Pose Estimation Datasets

As introduced in chapter 1 and expanded upon in chapter 3, there are a number of different computer vision pose estimation strategies, but the majority of these approaches require vast quantities of images for training. When images are difficult to acquire, they must be created. In this section, theory associated with spacecraft dataset imagery is reviewed; a spacecraft render pipeline is proposed, detailed and validated; and multiple datasets are rendered. By the end of the chapter, the reader will be familiar with the challenges associated with spacecraft rendering and how to address them. The complete rendering pipeline including models, material assets, rendering code, automation scripts and example images are publicly shared as a Google repository [61].

2.1 Obtaining Spacecraft Imagery

An image of a spacecraft in proximity (less than $\sim$1 km) can be obtained via real imagery in-space, via ground-based experimental setups, or via synthetic rendering as shown in figure 2.1

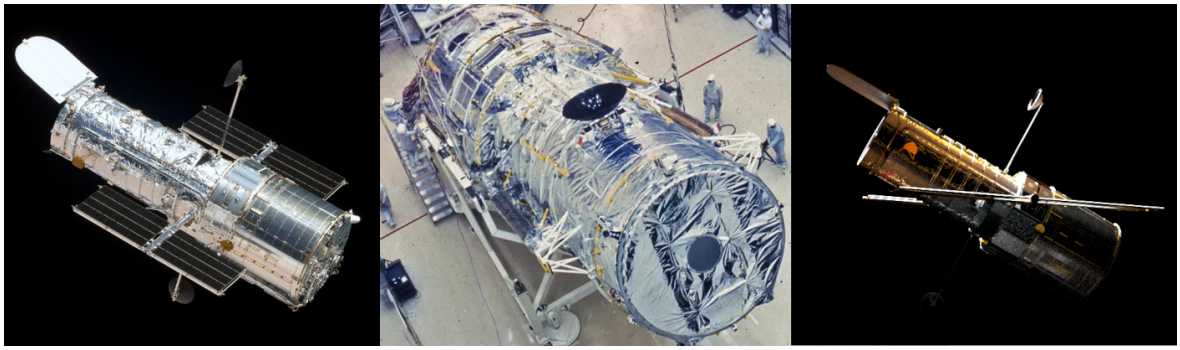
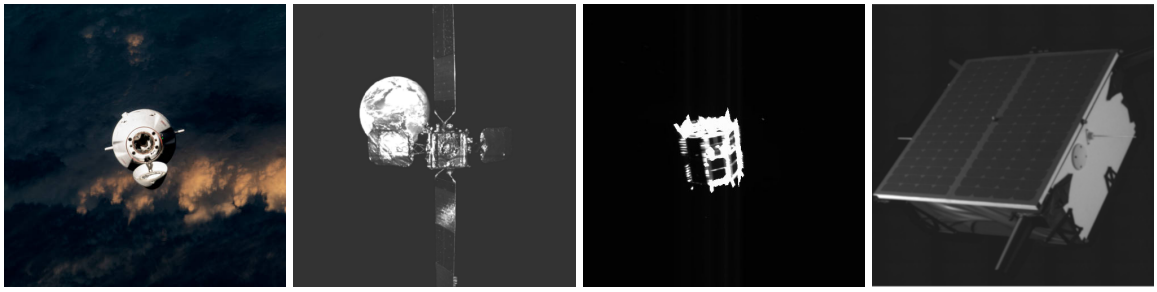


Figure 2.1: In-Orbit, Before-Launch (©Wikimedia) and Rendered Images of the Hubble Space Telescope

2.1.1 Real Imagery

Real imagery of resident space objects, while representative of the final use case, is expensive to obtain and often proprietary. Consider four example cases: a Low Earth Orbit (LEO), a Geostationary Equatorial Orbit (GEO), a deep space mission and a laboratory recreation as depicted in figure 2.2



(a) Exped. 69 ©NASA **(b)** Intelsat 901 [6] **(c)** Minerva ©JAXA **(d)** SPEED [62]

Figure 2.2: Real Spacecraft Images from Different Regions

The Expedition 69 image (a) features the SpaceX Dragon docking with the International Space Station (ISS) on June 6, 2023. Similar images and videos can be found in space agency archives, however the data is not annotated. The camera hardware used to capture the image is not defined and may even be operated by hand, resulting in unknown and possibly varying camera extrinsics. Computer vision pose estimation of the Dragon capsule demonstrations have been shown [2], however, to the extent of the author's knowledge, annotated images, camera intrinsics or extrinsic information are not publicly provided for these images. Relative pose estimation is possible, or through photometry and with target dimension information, an intrinsic/extrinsic calibration is possible; however this is not always feasible and annotating a sufficiently sized training dataset becomes an arduous task.

The Intelsat 901 image (b) was taken in February 2020 by a radiation hardened Charge-Coupled Device (CCD) camera system [6]; the image is single channel (i.e. grayscale). Single channel sensors can support a higher resolution on the same physical footprint, exhibit greater reliability to radiation exposure and are a logical choice for imaging extending past the human standard $400-700$ nm electromagnetic spectrum. Additionally, at a distance of $\approx 36,000$ km, the effect of Earth albedo[1] and atmospheric refraction are greatly reduced, resulting in less diffuse lighting conditions. Again, these images are not annotated.

Typical of deep space missions, the monochromatic trend continues for the Minerva-II2 image (c). However, for this hardware, the sensitivity ranged from only approximately 480-650 nm [63, 64]. Additionally, initial images were being transmitted at distances exceeding 10^9 km; at long distances, quantization noise and Gaussian noise can become more

[1] Albedo: The fraction of sunlight that is diffusely reflected by a body.

prevalent [65]. As discussed in greater detail below (Section 2.2.4), the Minerva-II2 images showcase additional local noise that must be accounted for as well.

The fourth example image (d) was captured in the Stanford Space Rendezvous Laboratory (SLAB) TRON. A robotic arm was used to position the model satellite in a dark room, a light box was used for lighting and post processing masking was done to crop out the robotic arm. Extensive effort was exerted to replicate the large radiometric dynamic range of in-orbit lighting [62]. The resulting images were of high fidelity but limited to a single target and distances of a few metres.

Thus it has been established that real spacecraft images are expensive, difficult to obtain, containing noise and often laborious to annotate. A practical solution is to resort to synthetic images [28, 39, 40, 41, 42, 43, 66, 67]. The challenge then becomes to train a method in one domain (i.e. rendered images) to then be tested in another domain (i.e. real images). In the following sections, a render pipeline is shared followed by analysis of datasets produced by the pipeline.

2.2 Render Pipeline

Perhaps counter-intuitively, the objective of a synthetic dataset is not necessarily to create hyper-realistic images, but to optimize pose estimation performance in the desired domain. While training image realism and testing domain performance are often correlated, this distinction is important when dedicating resources to a project.

In contrast to conventional ground-based pose estimation datasets, spacecraft renders incorporate a number of unique features including:

1. model symmetricity;
2. difficult to obtain, difficult to calibrate and restricted material properties;
3. unrestricted poses; and
4. unique noise, dynamic ranges and environmental backgrounds.

Each of these challenges will be addressed throughout Sections [2.2.1–2.2.4]. The render process flow is shown in figure 2.3.

2.2.1 Model and Material Properties

Synthetic images require a render environment, a render model and material properties. Several render environments are available with varying degrees of fidelity [68, 69, 70, 71]; for this work the Cycles engine is used [72].

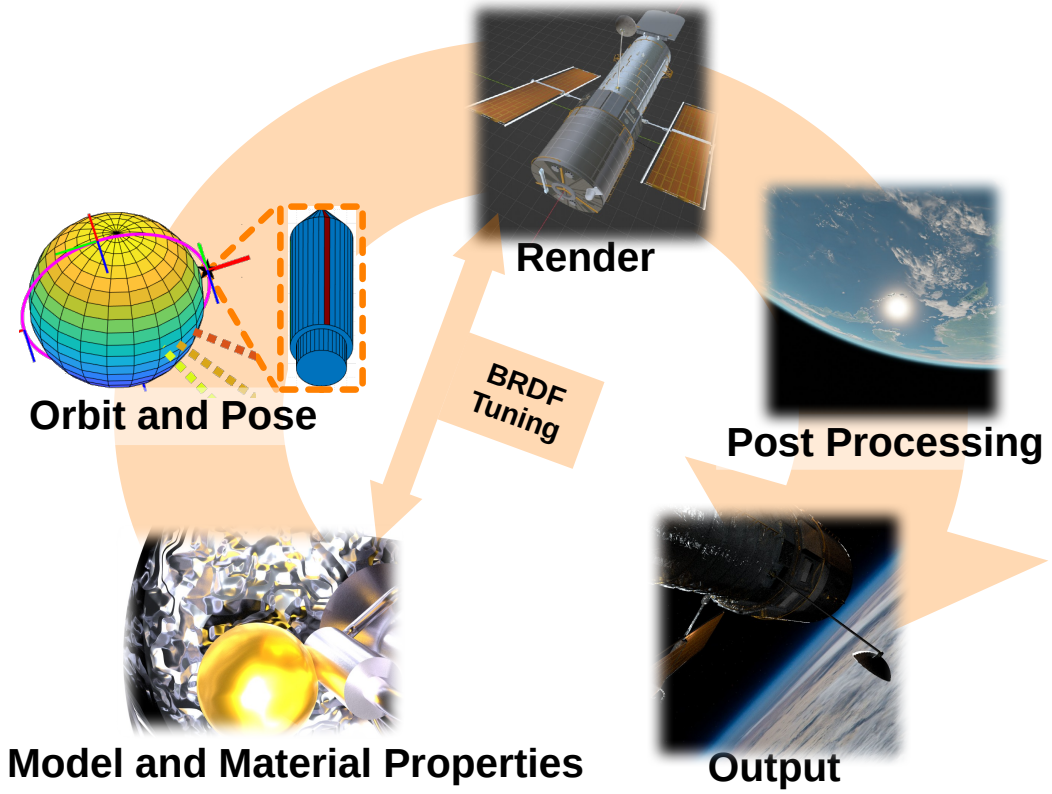


Figure 2.3: Spacecraft dataset Render Process Flow

Render Model

Spacecraft render models can be obtained from online databases [51, 61, 73] or constructed manually [67].

A symmetric model is especially challenging for pose estimation [74]. At the time of writing, no spacecraft pose estimation dataset, public or otherwise, has included a symmetric model. As will be discussed in Chapter 3, the symmetric Minerva-II2 model is a particularly challenging target for spacecraft pose estimation; the Minerva-II2 model is a D_{16} symmetric octagonal prism. The Minerva-II2 rover possesses three sets of identical faces leading to ambiguous pose solutions. To assess this, two Minerva-II2 models are rendered: the *original* physically correct model and the *modified* model with solar panels removed to create individually unique faces. The Minerva-II2 rover and the two models are shown in figure 2.4.

Material Properties

Obtaining appropriate material properties is more challenging. In many physics-based rendering environments, the Bidirectional Scattering Distribution Function (BSDF) is used [75]. The BSDF is a superset of the Bidirectional Reflectance Distribution Function (BRDF) and

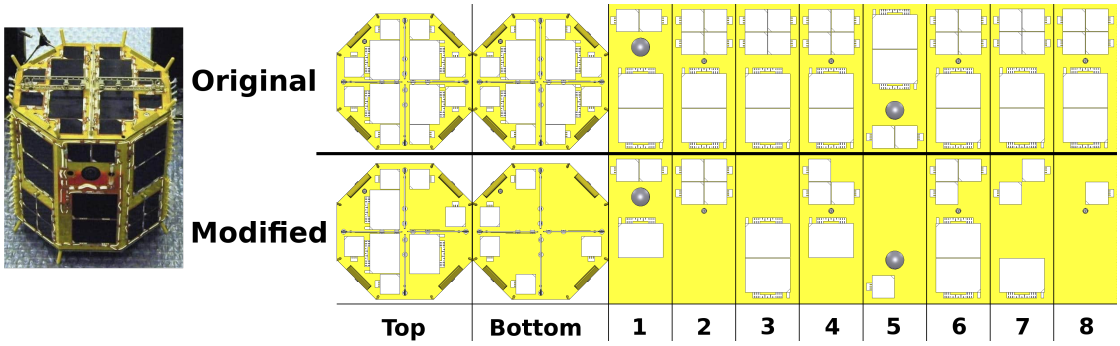


Figure 2.4: Minerva-II2 Symmetry, the *Original* Render Model, and the *Modified* Render Model

the Bidirectional Transmittance Distribution Function (BTDF) as depicted in figure 2.5. In practice, the BRDF dominates the render process [68].

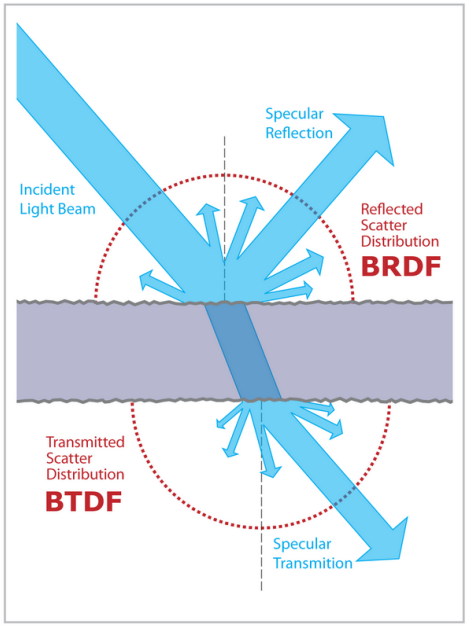


Figure 2.5: BSDF, a Generalization of the BRDF and BTDF (©Wikimedia)

In spherical coordinates, he BRDF is a 5 Degree of Freedom (DoF) parameterization, dependent on a 2 DoF incoming light vector, a 2 DoF outgoing light vector and the light wavelength, λ. From its first introduction [76], the BRDF is defined as:

$$BRDF_\lambda(\vec{v}_i, \vec{v}_o) = \frac{L_o}{E_i} = \frac{1}{L_i(\vec{v}_i)\cos\theta_i}\frac{dL_o(\vec{v}_o)}{d\vec{v}_i} \tag{2.1}$$

where

L is the radiance $[\frac{W}{m^2 sr}]$,
E is the irradiance[2] $[\frac{W}{m^2}]$,

[2]It can be helpful to think of radiance, L as the power of a ray of light and irradiance, E as the power of light in an area.

$\vec{v}$ is the light ray vector,
i is for the incident light,
o is for the outgoing light,
θ_i is the angle between the incident light and the surface normal,
λ is the wavelength.

In this way the BRDF is useful to know the partial radiance, dL_o for rendering a given pixel given the inbound light, L_i on vector, $\vec{v}_i$. Rendering the complete radiance for a single pixel then becomes the surface integral over all inbound light vectors as:

$$L_{o,\lambda} = \oint_{\vec{v}_i} BRDF_\lambda(\vec{v}_i, \vec{v}_o) L_i(\vec{v}_i) \cos\theta_i d\vec{v}_i \tag{2.2}$$

However, a material's BRDF is also frequency dependent, thus equation 2.2 returns the radiance of only one frequency. To handle the different frequencies, the total radiance must be integrated over all frequencies as:

$$L_o = \int_\lambda \oint_{\vec{v}_i} BRDF_\lambda(\vec{v}_i, \vec{v}_o) L_i(\vec{v}_i) \cos\theta_i d\vec{v}_i \tag{2.3}$$

which becomes challenging. Empirically obtaining a continuous frequency dependent BRDF function at different inbound light vectors for a material property is out of the scope of this work[3]. For the purposes of this render pipeline, a number of simplifying assumptions can be made.

1. The majority of spacecraft images of interest are captured by CCD camera systems tuned to the limited human light spectrum (recall figure 2.2). As such, the material BRDF will be assumed uniform over this limited spectrum; the dependency on λ will be neglected. For hyper-spectral or monochromatic images with significant illumination outside $400-700$ nm, this assumption is inadvisable.

2. Accurately defining the BRDF is a field of study in itself and beyond the scope of this work. Instead material BRDFs from available libraries will be used. If a material is unavailable, a set of parameters (e.g. specularity, roughness) are tuned to produce a desired $\widehat{BRDF}$.

3. Without a differential BRDF (see assumption 2), non-differential ray-traced based rendering is used. For this work the Blender Cycles renderer is used. Cycles is a *probabilistic* ray-tracing engine [72]; instead of summing over the complete set of $\vec{v}_i$, Cycles uses a semi-random subset of $\vec{v}_i$. As the render sampling parameter is increased, additional $\vec{v}_i$ are incorporated, with the limit converging to equation 2.2.

[3]Typically the BRDF is measured using gonio-photometry[77]. A light source and a detector are positioned at various angles to sample material reflectivity; a time consuming process which must be repeated for each material.

With the above three assumptions, this work's renderer radiance then becomes:

$$L_o \approx \sum_{rand\vec{v}_i} \widehat{BRDF}(\vec{v}_i, \vec{v}_o) L_i(\vec{v}_i) \cos\theta_i \tag{2.4}$$

where $\widehat{BRDF}$ signifies the sequence of tuned functions that represent the renderer's BRDF. The tuning of the $\widehat{BRDF}$ is an iterative process typically tied to a particular renderer software.

2.2.2 Orbit and Pose

For longer observation periods (1+ min), objects in LEO may experience significant environmental changes or attitude variations. To accommodate this, the orbits, the associated frame changes, the lighting and the tumble motion are simulated.

Orbit and Frame

Orbital parameters are defined using the Two Line Element set (TLE) specific to the National Aeronautics and Space Administration (NASA) perturbation models [78]. The orbit is mapped to an Earth Centered Inertial (ECI) frame and then discretized into points to calculate a local Hill frame[4] for each point. In this way, a tumble motion can be integrated in the ECI frame, mapped to the Hill frame and then rendered by a theoretical camera maintaining a corresponding relative pose in the Hill frame. For longer observation periods, the Earth albedo and the sunlight vector would vary dramatically throughout the orbit, resulting in high illumination dynamic ranges for the renders. An example orbit is depicted in figure 2.6 and the associated sunlight vector empemeris is shared in figure 2.7. For completeness, the source code is made available at [79].

Lighting

Notably, for an entirely convex object (no self-reflections) and no Earth albedo (a single sunlight source), a single render pixel will have a single inbound lightsource, L_{sun}. On average the sun is 1 standard astronomical unit ($150e^9$ m) distant from Earth [80] and can be considered a collimated light source; in the absence of atmosphere, the solar constant is $E_{sun} = 1361\ \frac{W}{m^2}$ [81] or $L_{sun} = \frac{1361}{\pi}\ \frac{W}{m^2 sr}$ for a collimated light source. Incorporating this solar constant, equation 2.4 further simplifies to

$$L_o = \widehat{BRDF}(\vec{v}_{sun}, \vec{v}_o) L_{sun}(\vec{v}_{sun}) \cos\theta_{sun} \tag{2.5}$$

which is an interesting result. The illumination of a given render pixel is therefore only a function of the known sun radiance, L_{sun} and the $\widehat{BRDF}$. These are ideal circumstances

[4]The Hill frame is defined as: positive X aligned with the vector from the center of mass of the central body to the spacecraft; positive Y tangent with the direction of travel; positive Z as the orbital plane normal vector.

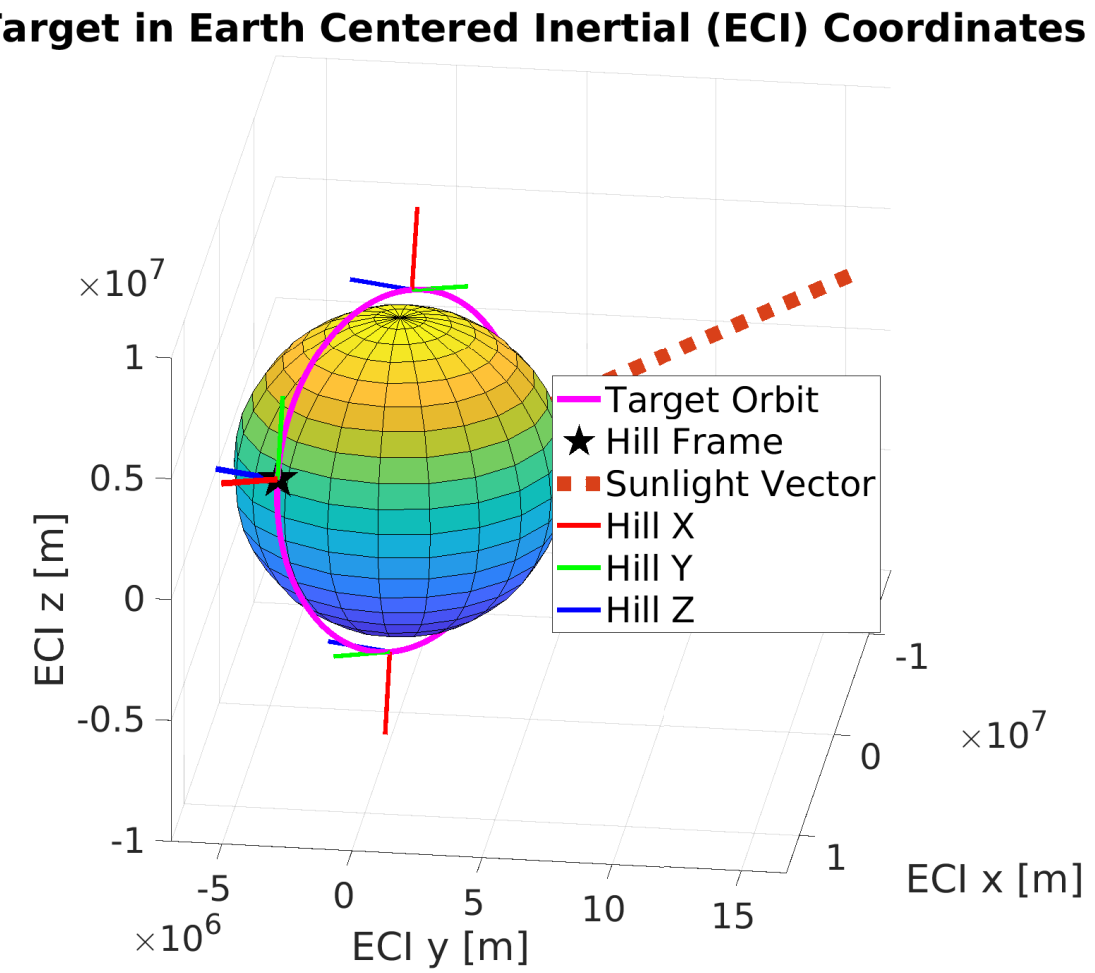


Figure 2.6: Simulated Polar Orbit During the Summer

for the calibration of a $\widehat{BRDF}$ after launch. Due to extensive exposure to solar radiation, and mechanical excitation during launch and manoeuvres, it is highly likely that the material properties and their associated BRDF would deviate from the values measured before launch. This idea will be further discussed in Section 5.4.

Conversely, the effect of Earth albedo is not collimated and is highly variational, being dependent on surface type, intervening atmosphere and incidence angle [82]. For the purposes of dataset rendering, the albedo is assumed to be isotropic and constant throughout the range of $400-700$ nm and set to an average factor of 0.3X the sunlight intensity [83].

Hyper-spectral wavelengths are not explored in this work. Although hyper-spectral imaging of the Earth from space is a mature field [84, 85, 86], in-orbit optical flow and hyper-spectral rendering of resident space objects in proximity is in its infancy. However, at the time of writing, there are the beginnings of hyper-spectral spacecraft rendering works as well [87]. This render pipeline can accommodate hyper-spectral rendering by replacing the Cycles render engine with a spectral render engine such as Mitsuba [68], but this will further

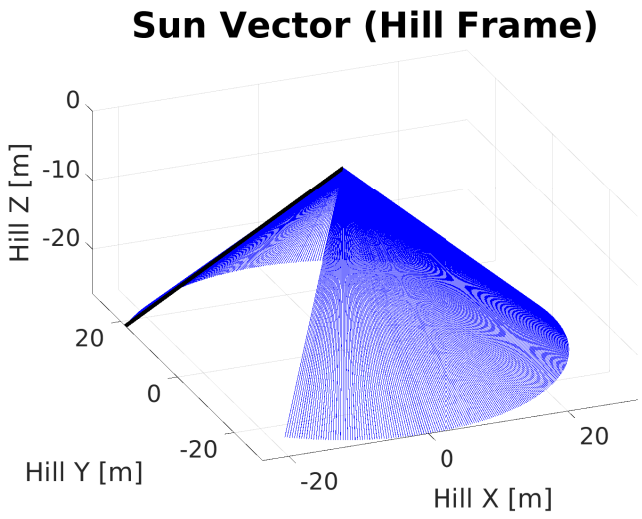


Figure 2.7: Sunlight Vector Ephemeris Plot

require the definition of *spectral* $\widehat{BRDF}$ functions.

Tumble Integrator

Orbital dynamics, trajectory estimation and control is a mature [88] and active field of research [89] with commercial software [90, 91] and academic simulation packages [92] existing. For the purposes of this render and pose estimation work, a simple rigid body motion is implemented; a fourth order Runge-Kutta integrator is coded from Euler's principal axes inertia rotation equations[5]:

$$\begin{aligned} I_1\dot{\omega}_1 + (I_3 - I_2)\omega_2\omega_3 &= M_1 \\ I_2\dot{\omega}_2 + (I_1 - I_3)\omega_3\omega_1 &= M_2 \\ I_3\dot{\omega}_3 + (I_2 - I_1)\omega_1\omega_2 &= M_3 \end{aligned} \tag{2.6}$$

with

- the inertia matrix, $\begin{pmatrix} I_1 & 0 & 0 \\ 0 & I_2 & 0 \\ 0 & 0 & I_3 \end{pmatrix}$ $[\frac{kg}{m^2}]$;
- the angular velocity, ω $[\frac{rad}{s}]$;
- the angular acceleration, $\dot{\omega}$ $[\frac{rad}{s^2}]$; and
- the external Torque, M $[\frac{kgm^2}{s^2}]$

for principal axes 1, 2 and 3. For completeness, the derivation of equation 2.6 from first principles and an explanation of applicable limitations is included in the Appendix; the coded integrator implementation may be found open-sourced at [79].

[5]The principal axes can be obtained for any inertia matrix [93]; thus the assumption that the observed render object's inertia matrix is diagonal is as valid as any other arbitrary canonical inertia frame.

In the case that the inertia matrix, $I = \int_m r^2 dm$, is unknown, mesh analyzer software [94, 95] can estimate a uniform density inertia matrix. If additional density information is available:

1. Uniformly sample N points from the mesh volume interior.
2. Calculate the Centre of Mass (CoM) $\vec{R}_{CoM} = \sum \frac{m_i r_i}{m_{total}}$.
3. Calculate the inertia matrix $I = \sum_i^N (r_i - R_{CoM})^2 m_i$.

where r_i is the location of point i and m_i is the mass of point i.

The tumble motion kinetic energy is monitored throughout the motion integration to ensure integrator stability and to check for energy leakage. An example visualization is provided in figure 2.8 using the initial conditions $I_1 = 2e^5$, $I_2 = 1e^5$, $I_3 = 5e^3$ $\frac{kg}{m^2}$ and $\omega = [0,4,4]^\top \frac{deg}{s}$.

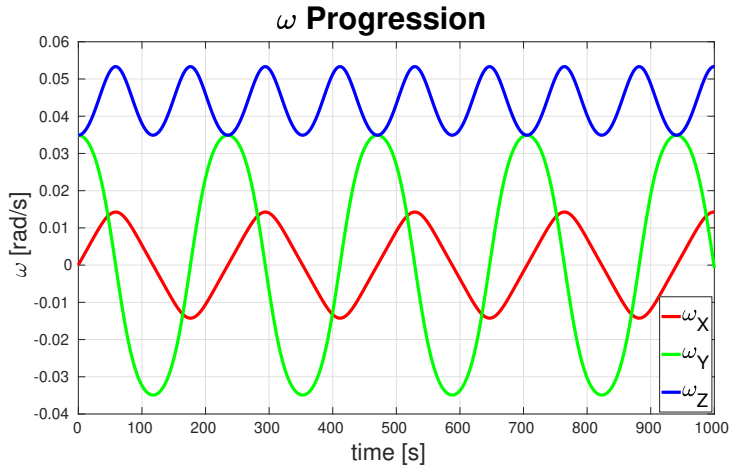


Figure 2.8: Tumble Integrator Example Visualization

Randomized Pose

Machine learning pose estimation algorithms benefit from a training dataset's increased size, variability and similarity to the testing domain. This includes exposure to representative poses; for spacecraft, any pose is conceivable. In addition to the tumble integrator attitudes, a sampling of all possible orientations are rendered. Uniformly sampling from the Euclidean space rotation Special Orthogonal group (SO(3)) is not trivial. The problem is akin to the energy minimization Thomson sphere problem [26, 96]; finding a uniform distribution of charged points, p

$$Energy = \sum_{i=1}^{n-2} \sum_{j=1+1}^{n} \frac{1}{|\vec{p}_j - \vec{p}_i|} \tag{2.7}$$

for the unit sphere geodesic distance function

$$|\vec{p}_j - \vec{p}_i| = arccos(\vec{p}_i \cdot \vec{p}_j) \tag{2.8}$$

As an example, the MATLAB randrot function [97] is used to generate random rotations which are applied to a unit vector and plotted in figure 2.9 left; the distribution is not uniform. Instead, for this work, a Gauss-Seidel [98, 99] iterative approach is adopted resulting in a more uniform distribution as viewable in figure 2.9 right.

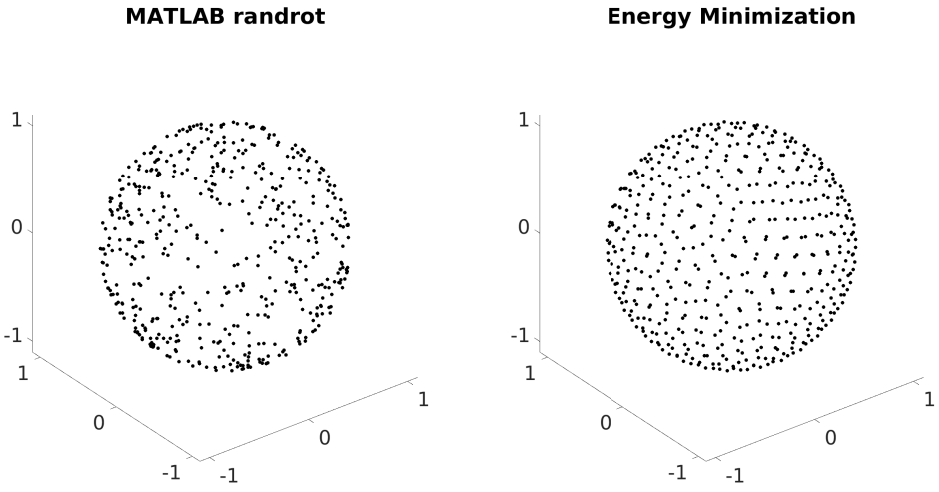


Figure 2.9: Sampling SO(3) Rotation Space for a Uniform Rotation Distribution

Locating the target in 3D space was not sampled uniformly. Instead camera Field of View (FoV) frustums of various sizes were created, with a heavier concentration of target poses included near the render camera as observable in figure 2.10.

In this way, a uniform sampling of random poses as well as physics based tumble motion poses are rendered.

2.2.3 Rendering

Renderer cameras do not require lenses, exposure time or a sensor ISO setting. To replicate such features, manual functions must be incorporated at the appropriate stage. For reference, the standard rectilinear[6] depth pinhole camera model [101], used by most render engines, is

[6]It is worth noting that not all renderers employ this model. The Blender Cycles renderer employs a spherical coordinate pinhole camera model instead of the standard rectilinear depth model. [100]

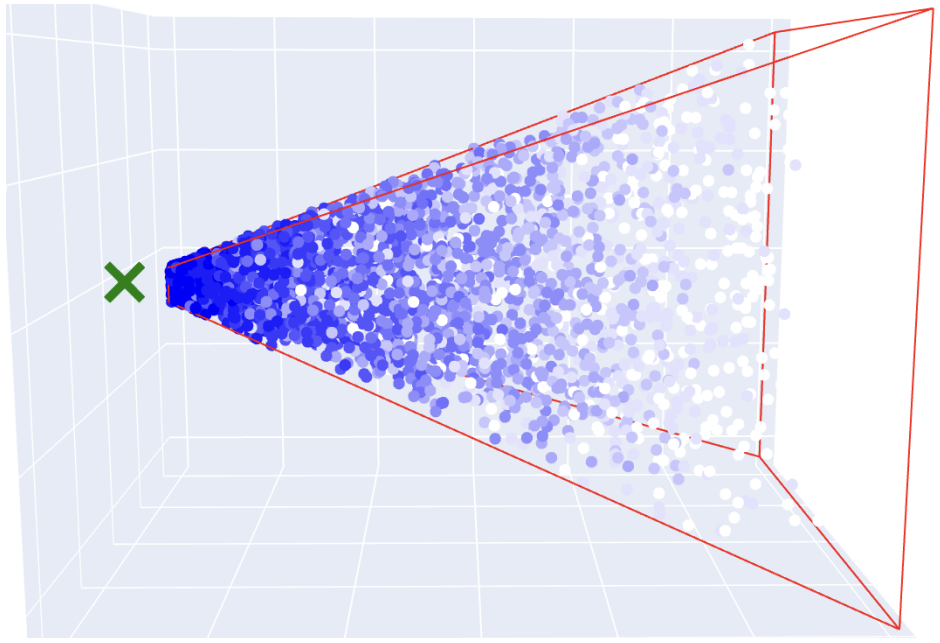


Figure 2.10: Render Camera FoV Frustum Visualization

written as:

$$z \begin{bmatrix} \frac{x}{z} \\ \frac{y}{z} \\ 1 \end{bmatrix}_{image} = \begin{bmatrix} f_x & 0 & p_x \\ 0 & f_y & p_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} Rot_{11} & Rot_{12} & Rot_{13} & T_x \\ Rot_{21} & Rot_{22} & Rot_{23} & T_y \\ Rot_{31} & Rot_{32} & Rot_{33} & T_z \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}_{world} \tag{2.9}$$

with

- pixel focal length, $f = \frac{F[m]}{pixelsize[m]}$ $[pix]$,
- principal point, p $[pix]$,
- extrinsic rotation matrix, Rot, and
- the camera location in world coordinates, $T[m]$.

Lens Distortion

It is common practice to calibrate the lens distortion of a physical camera so that an appropriate correction can be applied to the image. In the Hayabusa2 Minerva-II2 deep space mission however, the captured images were stored and transmitted without lens distortion correction [64]. In this case, the distortion model can be applied in reverse to achieve the desired distortion. Distortion is modelled by the polynomial [102]

$$\begin{bmatrix} x_u \\ y_u \end{bmatrix} = \frac{1 + k_1 r^2 + k_2 r^4 + k_3 r^6}{1 + k_4 r^2 + k_5 r^4 + k_6 r^6} \begin{bmatrix} x_d \\ y_d \end{bmatrix} \tag{2.10}$$

where

- x_u, y_u represent the undistorted image plane coordinates,
- x_d, y_d represent the distorted image plane coordinates,
- $r^2 = x_d^2 + y_d^2$, and
- k_i are the calibrated distortion parameters.

Exposure and ISO

Exposure and ISO affect the amount of light received by the sensor and the sensor sensitivity respectively. If this is required to be modelled in the training dataset, it can be included as a multiplicative gain, G_{exp}, in equation 2.4 after the summation of $\widehat{BRDF}$ incident radiance:

$$L_o \approx G_{exp} \sum_{rand\vec{v}_i} \widehat{BRDF}(\vec{v}_i, \vec{v}_o) L_i \cos\theta_i \tag{2.11}$$

Salt and pepper noise can be used to reproduce the type of noise associated with high gain ISO settings.

2.2.4 Post Processing

The practice of data augmentation has been shown to greatly improve neural network performance and to avoid over-fitting [103, 104]. Many data augmentation techniques (e.g. cropping, blurring etc.) are trivially implemented and thus will not be discussed here. However, unique to space-borne datasets are the environment background and certain sources of image noise.

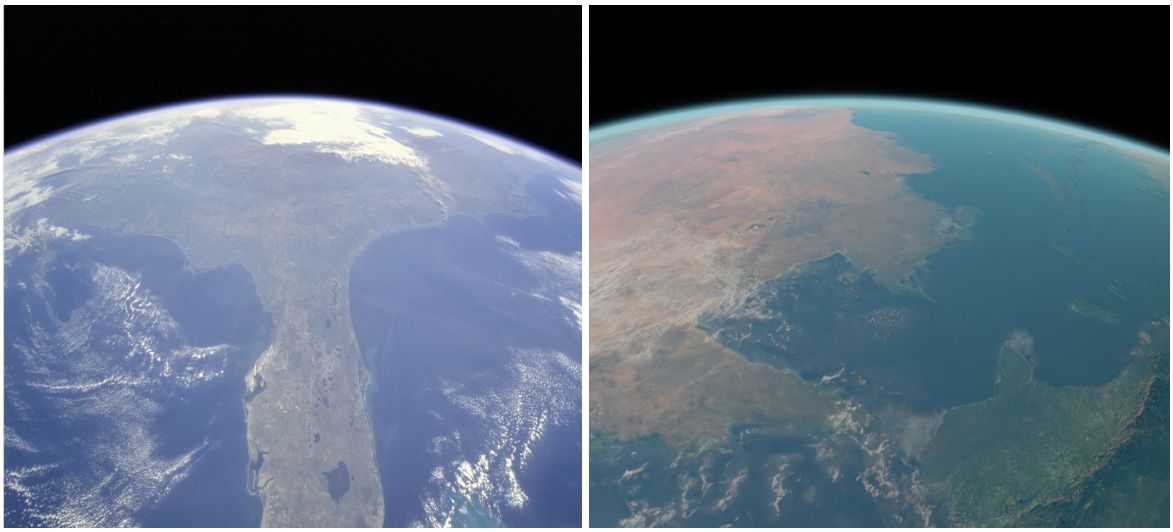
Background

It is common practice to insert real images of the Earth into the background of spacecraft datasets [28, 39, 41]. If a moving background is required, a roving crop of the Earth image is taken and inserted into the background. This results in a realistic background, but

1. the perceived motion does not take into account the Earth's spin or the viewing camera's orbit;
2. depending on the render environment, the domain gap between the Earth background image and the synthetic rendering may be perceivable; and
3. it is not possible to align the simulated Earth albedo lighting normal vector with the orientation of the background image Earth.

This work goes one step further by including a render model of the Earth; NASA-sourced Earth and Cloud textures uv maps [105] are layered to a sphere. In this way, more complex relative motion incorporating Earth spin, orbital motion, target tumbling and even occlusion is possible. To better help the pose estimation algorithms generalize, both rendered Earth backgrounds and real image Earth backgrounds are incorporated into the open-sourced

synthetic multi-object spacecraft dataset [61]. For comparison, two example background images are shown in figure 2.11.



(a) Real Earth Image **(b)** Rendered Earth Image

Figure 2.11: Example Earth Background Images

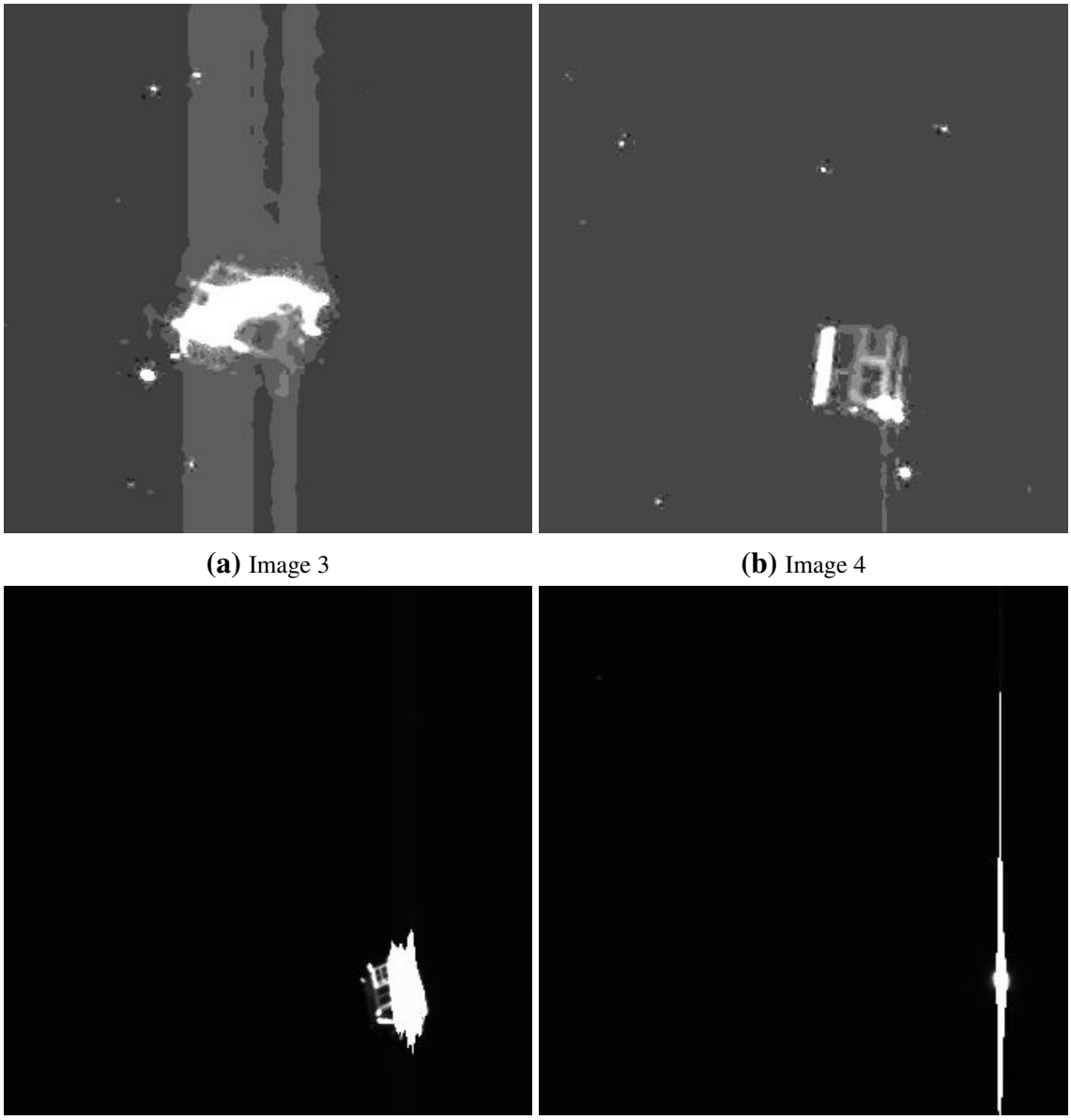
Noise

Space missions (in particular *deep* space missions) can exhibit significant sources of noise due to the lack of a protective atmosphere and long transmission distances. In the Minerva-II2 dataset, the inclusion of a number of physically-based noise post processing functions is necessary to extract pose information [43]. For context, four real Minerva-II2 deployment images can be referenced in figure 2.12. Notably, all three of the Hayabusa 2 optical navigation system cameras were CCD type with calibrated light responses of approximately uniform sensitivity in the range of 480–640 nm [64]. Two types of noise are modelled, sensor noise and transmission noise.

Sensor noise, transmission noise and their respective modelling functions are not commutative. That is, the noise occurs and corrupts the image in a particular order, partially described with figure 2.13. For example, physics related noise (e.g. motion blur) is followed by sensor capture noise (e.g. saturation) is followed by transmission noise (e.g. Gaussian). The following post processing noise functions are introduced in order of their real (and modeled) occurrence.

Sensor Noise

The sensor noise detailed here is generalizable to other spacecraft imagery; radiation hardened CCD sensors without colour filter arrays have generally been the optical sensor of



(a) Image 3 **(b)** Image 4

(c) Image 6 **(d)** Image 32

Figure 2.12: Cropped Raw Deployment Images of the Minerva-II2 Rover ©JAXA

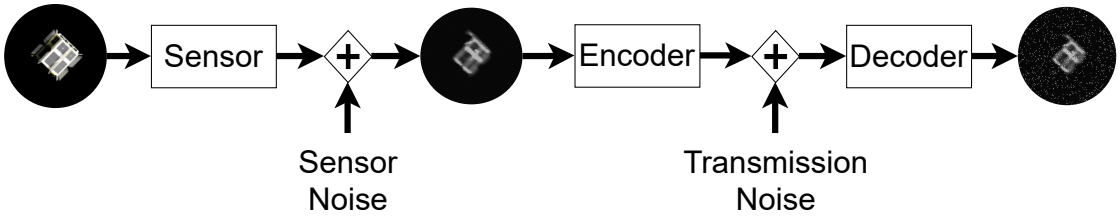


Figure 2.13: Noise Sources in Spacecraft Imagery

choice for deep space or long duration space missions [6, 64]. Moving forward, the noise modelling algorithms shared here can be adapted if the market shifts to radiation hardened

Complementary Metal Oxide Semiconductor (CMOS) sensors [106].

A CCD sensor is a 2D array of photoactive layered capacitors. When the aperture is opened, light is projected onto the photoactive region causing each capacitor to accumulate charge. Once the aperture is closed, the charge is transferred sequentially along the array (i.e. a shift register) and sequentially passed through a charge amplifier converting the charge into a voltage as shown in figure 2.14. The image is saved as a sequence of voltages [107].

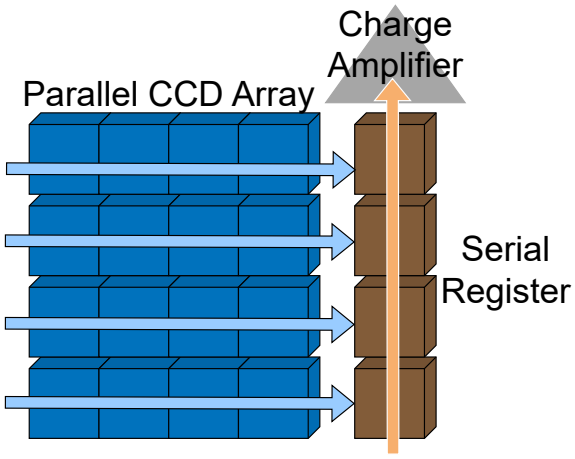


Figure 2.14: Simplified CCD Sensor Schematic

For the purposes of render image noise modelling, three features of the CCD sensor are of particular importance. Firstly, note the sequential connection of adjacent capacitors as illustrated in figure 2.14. Secondly, note that all image values are recorded as a voltage as measured from a single (or one of two) charge amplifiers; the voltage reference is the same for all pixels and thus the intensity of the entire image is interconnected. Thirdly, the final pixel values are recorded as voltages while passing through the control circuitry, then pass through an encoder and finally pass through a waveform transformer before being transmitted.

Motion Blur occurs when the aperture is left open long enough that features in the observed image pass through being projected onto different pixels before the aperture is closed. This problem is well understood but nonetheless occurs in challenging space environments. The lack of diffuse and reliable lighting produces difficult situations for the appropriate selection of ISO setting and shutter speeds; trade-offs must often be made. To model this in post processing, 1D Gaussian blurring is applied multiplicatively and globally to all pixel intensities along the direction of motion.

CCD Bloom occurs when a single CCD capacitor or set of capacitors are overcharged and the excess charge leaks into neighboring capacitors. This leakage only occurs to connected capacitor neighbors and results in a sequence of image pixels recorded at maximum intensity, $I = 1$, as observable in figure 2.12 (a) and (d). From these images, it is clear that the Hayabusa 2 CCD capacitors are connected in vertical arrays. To replicate this in post pro-

cessing, 1D Gaussian blurring is applied additively to saturated pixels (i.e. pixel intensities of $I = 1$).

Sensor Saturation is observable as a highly variable contrast. It is readily seen when comparing figure 2.12's images (a) and (b) to images (c) and (d). This is a result of overall sensor saturation. The combination of the camera ISO setting and aperture overexposure results in the entire image recording a higher base voltage; the sensor dynamic range is effectively reduced as the dynamic range lower limit is raised towards the upper limit (the saturation voltage). To model this for image pixel intensity, $I \in [0,1]$, in post processing, pixel intensities below the saturation threshold, s_{th}, are dropped (if $I_{i,j} > s_{th}$, $I_{i,j} \leftarrow s_{th}$). This effectively reduces the image dynamic range from [0,1] to [s_{th},1], for the saturation threshold $s_{th} < 1$.

Random Artifacts are an additional form of information corruption especially observable in figure 2.12 (c). Image corruption can occur due to thermal gradients or ephemeral radiation exposure. For the Minerva-II2 render pipelines a semi-random polygon artifact, scaled to the target's projected size is included.

Random Particles can be observed in figure 2.12 (a) and (b). These particles only occurred in the saturated (reduced dynamic range) images and are a result of dust motes and release mechanism particles being highlighted due to overexposure. A separate function is written to replicate these particles.

Transmission Noise

In addition to sensor noise, space missions (especially *deep* space missions) are challenged by extremely long transmission distances. Transmission noise is typically modelled with Gaussian and salt and pepper noise.

Gaussian Noise is a generic catch all for transmission noise. The assumption is that more disruptive perturbations to the waveform signal will occur less frequently, following the Gaussian probability density function

$$Gaussian(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \tag{2.12}$$

which is typically included via convolving a 2D kernel and the 2D image with variance, σ.

Salt and Pepper Noise or impulsive noise is characterized as sharp perturbations of a signal. It can occur during analog-to-digital conversion (decoders) but is more often due to a bit error during transmission. Both Gaussian and salt and pepper noise are generally included in most image analysis software [108].

2.3 Render Pipeline Results

With the material properties, pose, render engine and post processing defined, the full render pipeline is used to support a number of experiments and projects. For this paper, five rendered datasets are introduced.

1. The **Cubesat** dataset is first used to evaluate the domain gap between experimental real images and this render pipeline.
2. The **Rocket Body** dataset is then used to perform CNN based pose estimation on 6 DoF poses for 3 channel images. This dataset is developed in support of the Japanese Aerospace Exploration Agency (JAXA) Commercial Removal of Debris Demonstration (CRD2) project.
3. The **Modified Minerva-II2** dataset is used as a stepping stone to the more challenging Minerva-II2 CNN based pose estimation for single channel images.
4. The **Minerva-II2** dataset is used to estimate the pose and deployment velocity from real Hayabusa 2 Minerva-II2 images as compared to the JAXA orbit insertion analysis.
5. Finally, the **Multi-Object** dataset is introduced and shared publicly with the render pipeline. This contribution is provided open-sourced as a road map for future works and to lower the barrier of entry for other researchers requiring renders of spacecraft images.

The datasets are discussed in the following respectively numbered sections 2.3.1–2.3.5. A summary of the dataset parameters is shared in table 2.1.

Table 2.1: Render Dataset Summary

Dataset	Pose	Has Real Images	Channels	Model
Cubesat	Planar	Yes	3	Asymmetric
Rocket Body	SO(3)	No	3	Asymmetric
Modified Minerva-II2	SO(3)	No	1	Asymmetric
Minerva-II2	SO(3)	Yes	1	**Symmetric**
Multi Object	SO(3)	No	3	Asymmetric

2.3.1 Cubesat Dataset

The first dataset uses an experimental laboratory setup to capture images of a physical cube satellite model. These experimental images are used as a comparator to evaluate the performance of the proposed pipeline.

Cubesat Real Images

To obtain the experimental images, the Tohoku University Space Robotics Laboratory (SRL) 2D granite air-bearing testbed is placed into a dark room. A custom built parabolic reflector with a 1 kW light source located at its focal point is used as a collimated sunlight source. The camera is aligned so as to avoid primary reflections from the background. The experimental setup can be viewed in figure 2.15.

A micro-satellite physical model is constructed at 2X scale to a 2U satellite (dimensions 300x160x160 cm). A compressed air tank and 3 air-bearings ensure negligable friction between the micro-satellite and the granite table at test time. The micro-satellite surface is wrapped in satellite Multi-Layer Insulation (MLI). Two varieties of solar panels and four aluminum antennas are mounted on the exterior. Two Printed Circuit Board (PCB) CCD cameras are recessed into the casing. The cubesat physical model can be viewed in figure 2.15. Images are captured with a Realsense D535i camera [109].

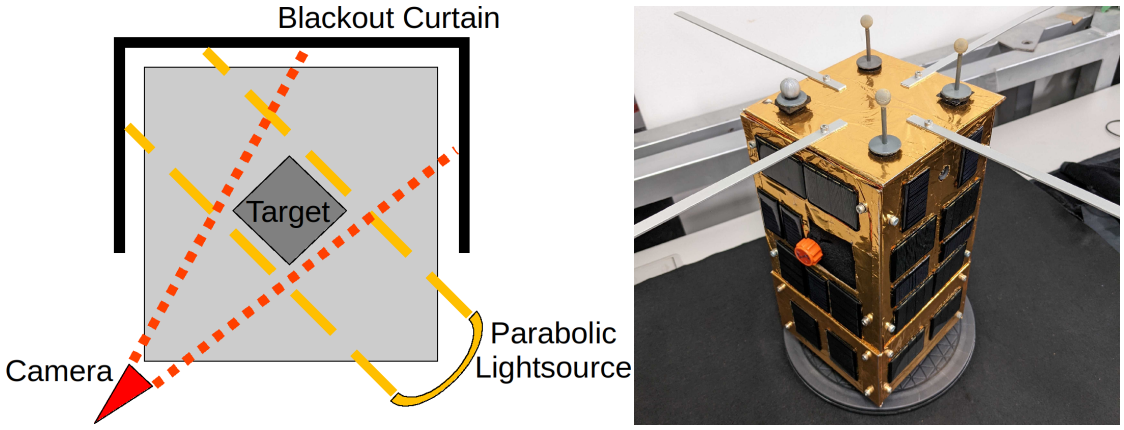


Figure 2.15: The SRL Dark Room Air-Bearing Testbed and the Cubesat Physical Model

Cubesat Renders

Following the render pipeline process flow:

1. **Model and Material Properties:** The physical model is measured and a geometric CAD model is manually constructed. Scans of the MLI and the two solar panels are used to manually tune the $\widehat{BRDF}$ material property functions as visualized in figure 2.16. A single collimated 1 kW white light source is used.
2. **Orbit and Pose:** Rendered poses are limited to 1D rotation and 2D translation poses as the experimental setup is limited to planar motion. No orbit is modelled as the orbit can not be replicated in the laboratory.
3. **Rendering:** The Realsense camera automatic ISO and shutter speeds adjustment are disabled. The Realsense camera intrinsic and extrinsic matrices are readily available. Approximately 300 images are rendered.

4. **Post Processing:** As an opportunity to validate the post processing step independently, two cases are investigated: (1) no post processing and (2) post processing sensor saturation is incorporated into the rendered images.

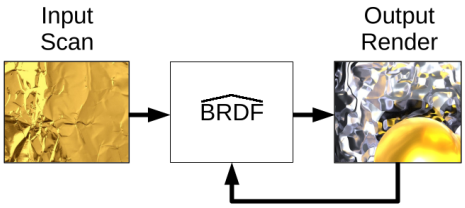


Figure 2.16: $\widehat{BRDF}$ Manual Tuning

Real vs render side-by-side comparison images are provided without post processing noise in figure 2.17.

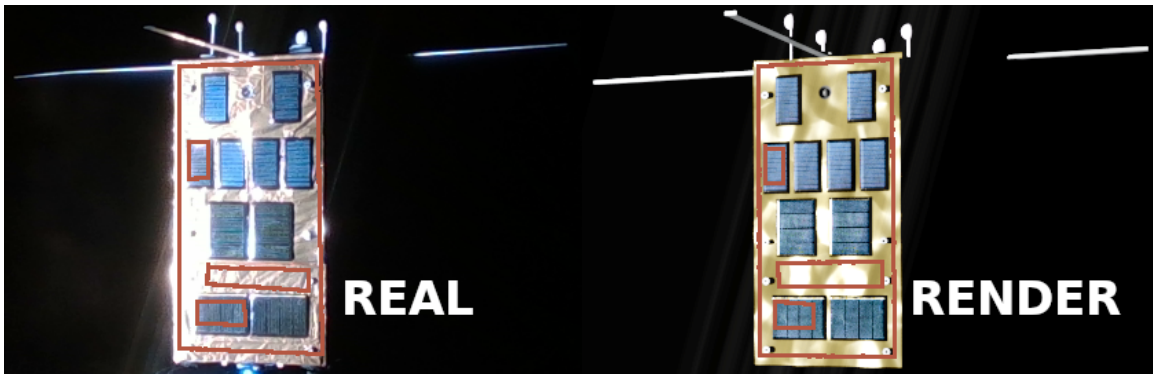


Figure 2.17: Real and Rendered Cubesat Images without Post Processing Noise Included

It can be seen that the real shadowing and solar panel hue are well represented in the render, but the real blooming sensor saturation effect is not well represented in the render image. This is expected, as sensor saturation and bloom are not included in the render illumination (equation 2.11), but instead incorporated as post processing sensor noise. For a quantitative comparison between the two images, the Battacharya distance function [110] is introduced here as

$$d(H_1,H_2) = \sqrt{1 - \frac{1}{\sqrt{\overline{H}_1\overline{H}_2 N^2}} \sum_{i=1}^{N} \sqrt{H_1(i)H_2(i)}} \tag{2.13}$$

where H_1, H_2 are the image pixel intensity histograms, $\overline{H}_1, \overline{H}_2$ are the respective image histogram means, and N is the total number of histogram bins. For a multi-channel Red Green Blue (RGB) image, separate distances can be calculated for each channel.

Discrete regions of interest are defined for real vs render comparisons as indicated by the orange bounding boxes in figure 2.17. Specifically, an MLI region, a solar panel region

and an overall region of interest are defined. The 3 channel image histograms of the real and rendered images without post processing are shown in figure 2.19's top row. The Battacharya distance between the real image and the rendered image for each region is shown in table 2.2 as the first two rows.

Observing the figure 2.19 image histograms:

- It is clear that the **solar panel** hue is well matched. The 3 channel image intensity distribution functions are well aligned, but the real image variance is smaller. Possibly the real solar panel BRDF exhibits anisotropic behaviour compared to the render solar panel $\widehat{BRDF}$.
- The rendered **MLI** histogram intensity is actually higher than the real MLI histogram with a distinct peak at intensity, $I = 1$, for all 3 channels, but this is more a function of the MLI bounding box location. It is clear that the real image is experiencing more sensor saturation than the render image in other locations as viewable in the overall histograms. The rendered image MLI appears more yellow than gold; the render blue channel distribution does not match the real blue channel distribution. Evidently the render MLI $\widehat{BRDF}$ requires additional tuning.
- The real image **overall** histogram is dominated by saturation ($I = 1$), whereas the intensity peaks at the 0.75 bin for the render. Again, this is expected as sensor saturation is modelled as a post processing step in this render pipeline.

Next, post processing sensor saturation is applied to the render images. The real vs render side-by-side comparison images are shown in figure 2.18.

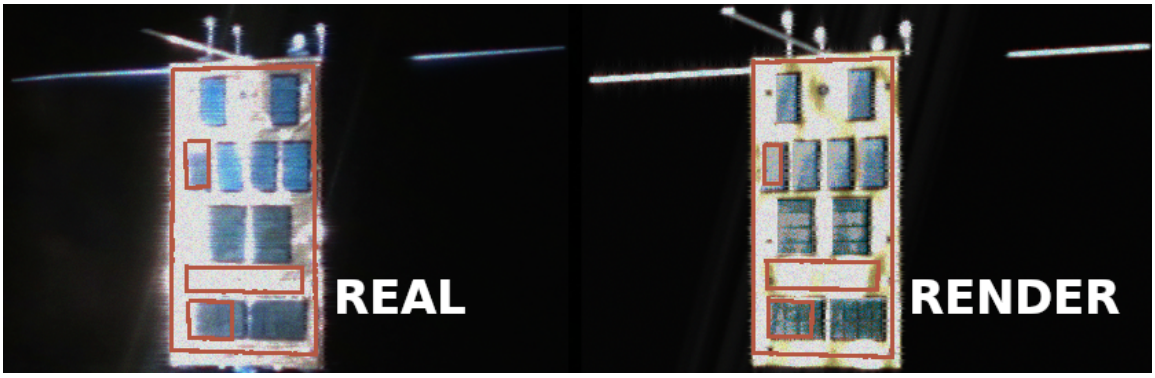


Figure 2.18: Real and Rendered Cubesat Images with Post Processing Noise Included

Upon inspection, it is clear that the modelling of sensor saturation has reduced the difference between the two images. Again, image histograms are shown in the bottom row of figure 2.19 and the Battacharya distance between the real and the rendered images with post processing is shown in table 2.2.

It is clear that with post processing, the histogram distributions are better matched. The Battacharya distance was reduced for all regions of interest as well. If further improvement is

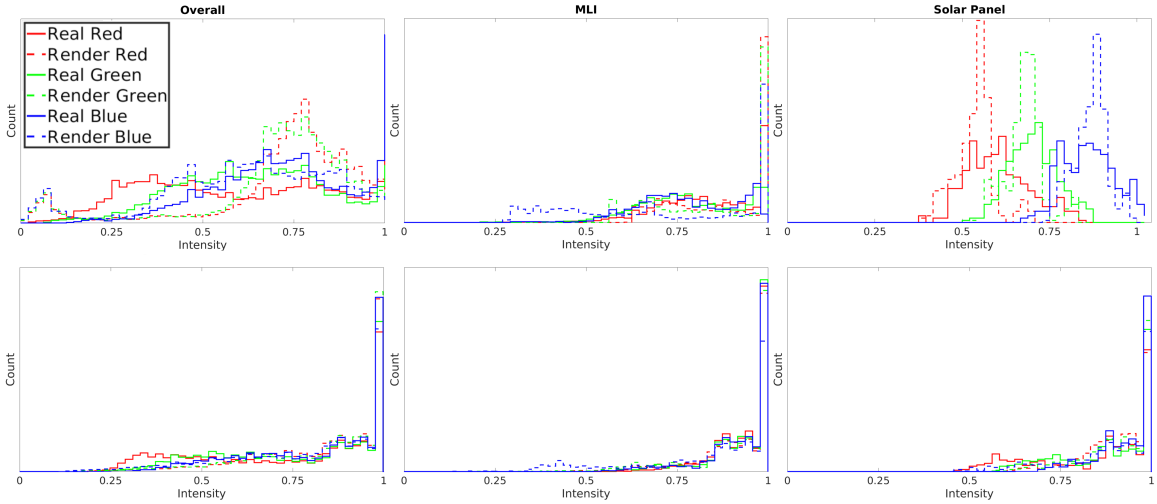


Figure 2.19: Cubesat Real vs Render Channel Pixel Intensity Histograms
Top: no post processing
Bottom: with post processing sensor saturation

Table 2.2: Battacharya Distance Between the Real and Rendered Images (lower is better)

Histogram	Overall	MLI	Solar Panel
RGB	.32, .30, .25	.33, .37, .39	.45, .34, .30
mean	**0.291**	**0.359**	**0.362**
Post Processing *RGB*	.23, .17, .13	.11, .11, .23	.29, .25, .22
Post Processing *mean*	**0.176**	**0.148**	**0.254**

required, more tuning of the MLI $\widehat{BRDF}$ may be necessary. Notably, applying transmission noise to both images has saturated many pixels and removed surface texture information for a NN to learn. The advantage of this approach is that the hypothetical NN will be required to learn more macroscopic/geometric features as opposed to surface texture features. As seen in the Minerva-II2 images, surfaces texture features can be unreliable in deep space imagery. As will be shown in chapter 3, this render result was sufficient to perform pose estimation with a variety of CNN architectures.

2.3.2 Rocket Body Dataset

With the render pipeline validated, the next step is to produce a spacecraft dataset with 6 DoF poses. In this example, the JAXA H-IIA launch vehicle upper stage is used. Following the render pipeline process flow:

1. **Model and Material Properties:** A geometric CAD model was developed in-house from publicly available images. Material property $\widehat{BRDF}$s were constructed via pub-

licly available image scans.

2. **Orbit and Pose:** A (1) simple flat spin and a (2) conic motion are integrated. A polar orbit is selected to showcase the effect of a roving sunlight vector. (3) Random orientations with random sunlight vectors are also rendered for training.
3. **Rendering:** There are no camera requirements. The sunlight source is set to the in-orbit theoretical value of $L_{sun} = \frac{1361}{\pi} \frac{W}{m^2 sr}$ [81]. 16,845 images are rendered including random orientations and pre-defined tumble motions.
4. **Post Processing:** Without specific reference images to replicate, no post processing is incorporated.

The effect of the roving sunlight vector is clearly visible in figure 2.20. The effect of modifying the sunlight intensity is shown in figure 2.21. Finally, a sampling of the conic motion is shown in figure 2.22.

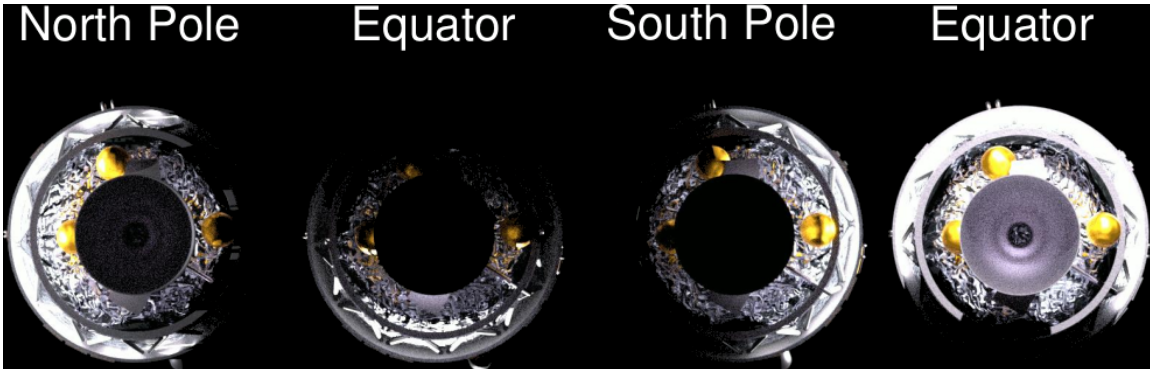


Figure 2.20: Rocket Body at Different Locations in Orbit

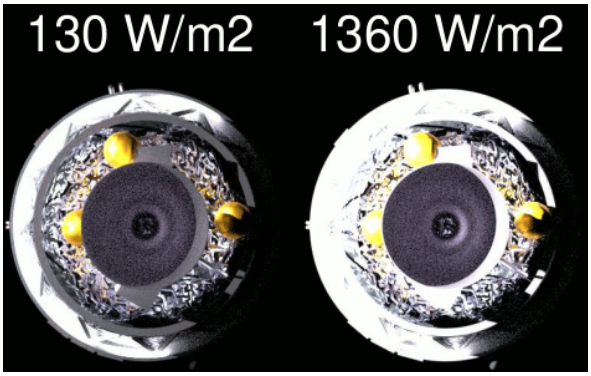


Figure 2.21: Rocket Body with Different Sunlight Intensities

2.3.3 Modified Minerva-II2 Dataset

For the challenging Minerva-II2 deployment case, two datasets are created: an (1) asymmetric CAD model and a (2) realistic symmetric CAD model. Both datasets use the same

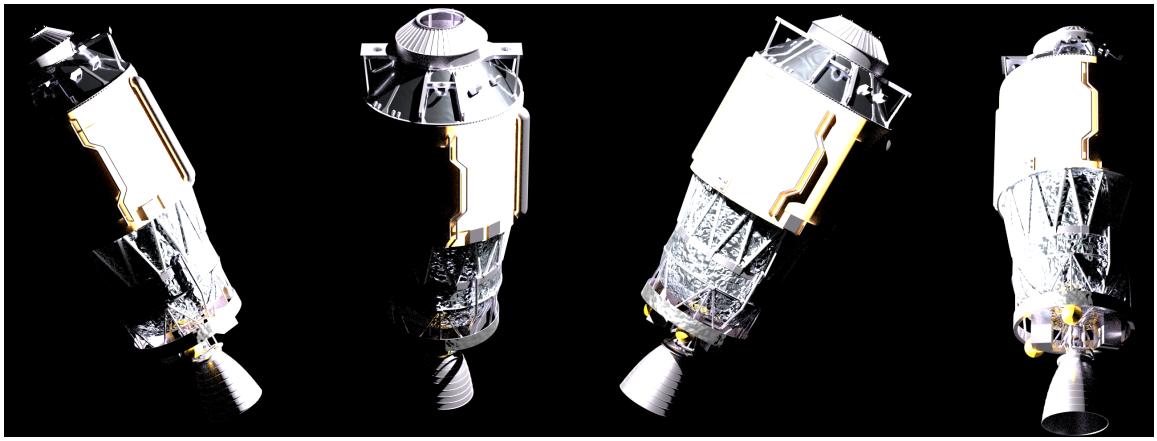


Figure 2.22: Rocket Body Conic Motion

render-pipeline settings with the exception of the CAD model.

1. **Model and Material Properties:** A CAD model from the original developers of the Minerva-II2 rover is available; material properties are not available.
2. **Orbit and Pose:** A (1) generic tumble and (2) randomized poses are rendered. As this was a deep space deployment, no orbit is simulated.
3. **Rendering:** The Hayabasa2 camera intrinsic matrix is readily available [64], however extrinsic information is not. Without telemetry, the precise sunlight vector is unknown. Instead, from the NASA Horizons System [111], the Hayabusa2, Ryugu asteroid and Sun emphemerides are used to estimate the sun vector as seen in figure 2.23. $\pm 6°$ are applied to the estimated sunlight vector. The real images are also adjusted for barrel distortion instead of applying distortion to the rendered images. 20,300 images are rendered with poses including random orientations and generic tumble motions.
4. **Post Processing:** As real images are available for references, the full suite of post processing sensor and transmission noise is incorporated.

A snapshot of the modified Minerva-II2 renders are shown in figure 2.24. The snapshot contains 10 sequential images of general integrated tumble poses.

2.3.4 Minerva-II2 Dataset

The unmodified (i.e. realistic and symmetric) model dataset is used to train a network for pose estimation inference on the real Minerva-II2 deployment images. A comparison of (a) the render without post processing, (b) the render with post processing, and (c) a real image is shown in figure 2.25.

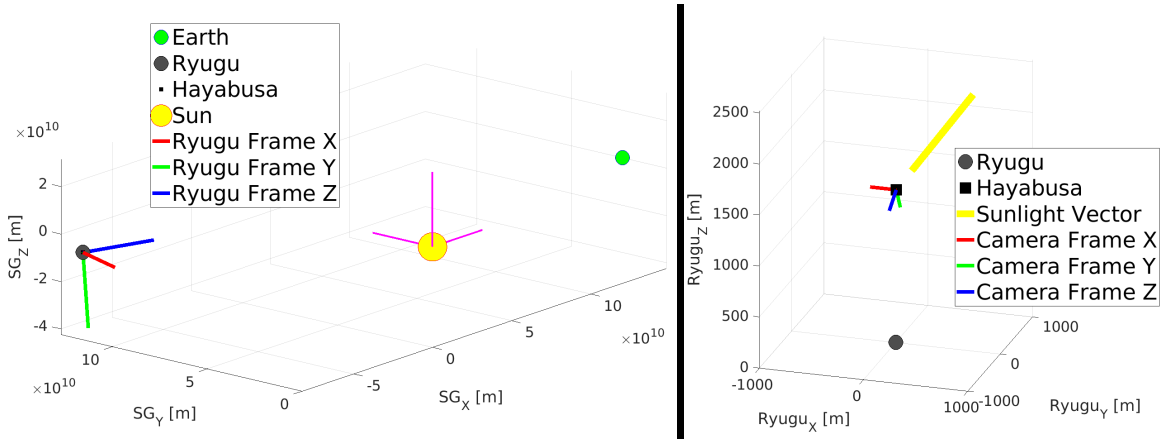


Figure 2.23: Minerva-II2 Deployment Sun Vector Estimation
Left: Sun Geocentric Frame
Right: Ryugu Asteroid Local Frame

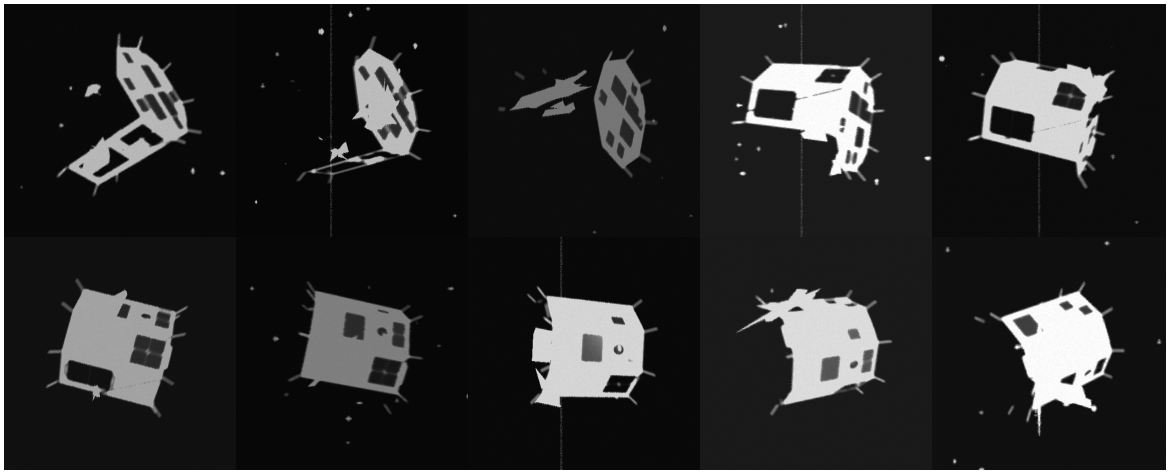


Figure 2.24: Modified (asymmetric) Minerva-II2 Render Images of an Integrated Tumble

(a) Render **(b)** with Post Processing **(c)** Real

Figure 2.25: Unmodified (symmetric) Minerva-II2 Renders and Real Image Comparison

2.3.5 Multi-Object Dataset

Finally, the most recent render dataset is shared. This dataset contains multiple models in support of unseen object pose estimation for future work. The render pipeline settings are defined as:

1. **Model and Material Properties:** Previously developed, open-sourced public and ESA SciFleet models are included. No calibrated material properties are used.
2. **Orbit and Pose:** Several (1) generic tumbles and (2) randomized poses are rendered. Randomized orbits and Earth albedo are included.
3. **Rendering:** There are no camera or lighting requirements. 40,000+ images are rendered, but the project is ongoing. More models are being incorporated and more renders will be made.
4. **Post Processing:** Without specific reference images to replicate, no post processing is incorporated.

A sampling of the multi-object rendered images are shown in figure 2.26. Additional renders for various datasets are available in the Appendix.

2.4 Summary

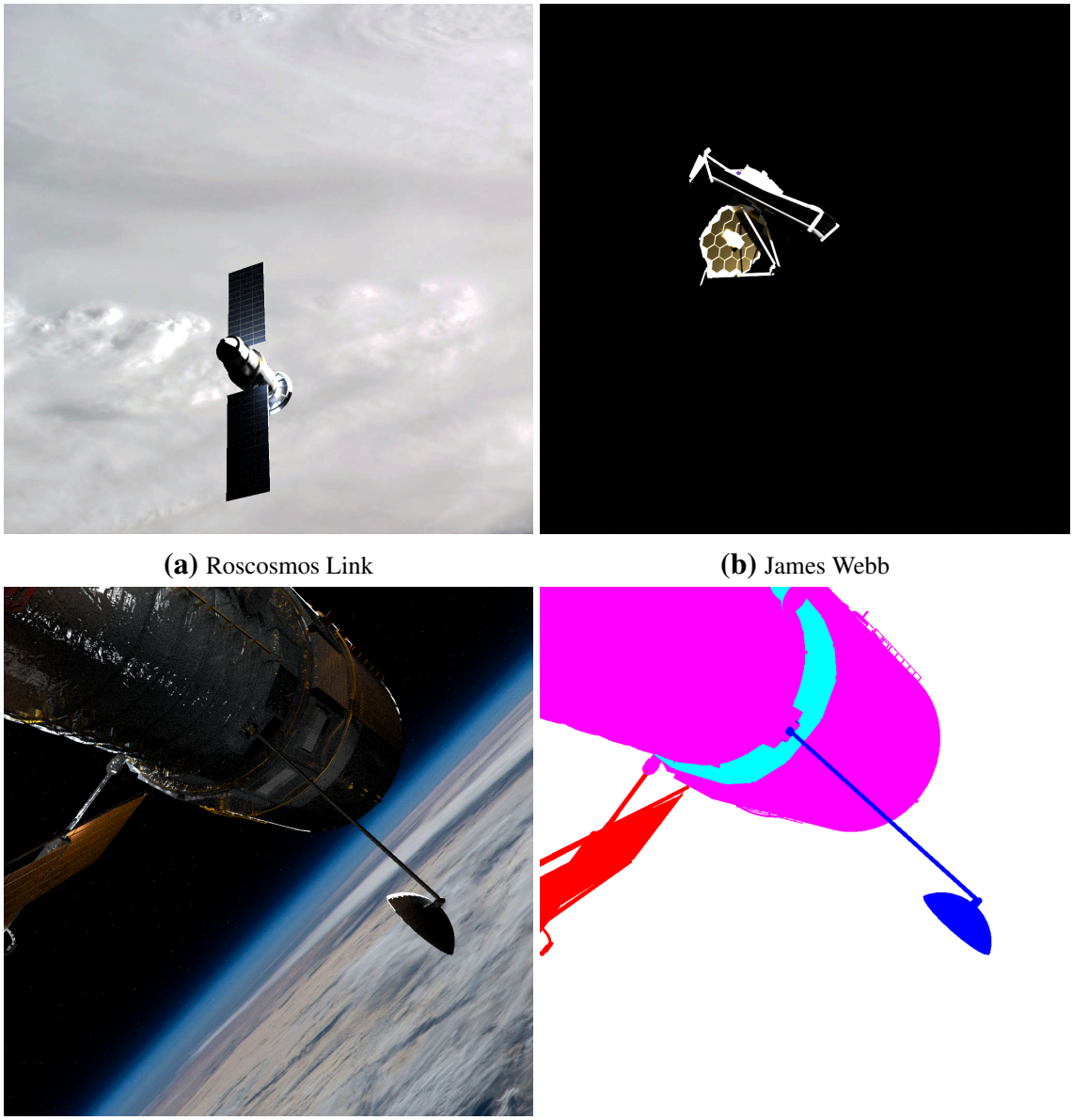
In this chapter, the challenges of obtaining spacecraft imagery are acknowledged. Depending on the domain, images are available, but frequently not annotated and without camera intrinsic or extrinsic information. Synthetic spacecraft datasets are an obvious solution but are fragmented across the community due to proprietary restrictions, difficult to obtain material properties and unreleased render pipelines.

To address this, a full and complete spacecraft render pipeline is proposed, detailed and validated. The render pipeline is broken down into four steps:

1. **Model and Material Properties:** The target model geometry and material property BRDFs are defined. In the event that material properties are unavailable, $\widehat{BRDF}$s are manually tuned in the render environment.

2. **Orbit and Pose:** Spacecraft motion is not restricted to planar motion or subjected to reliable diffuse lighting conditions often observable in Earth-bound datasets. To address this, a rigid body tumble integrator; an orbit integrator; and a solution to produce uniformly distributed rotations in SO(3) are defined and shared.

3. **Rendering:** Rendering cameras do not require lenses, exposure time or an ISO setting, but these parameters can be addressed when rendering images for comparison to real-world images.



(a) Roscosmos Link

(b) James Webb

(c) Hubble

(d) Hubble Segmentation Map

Figure 2.26: Multi-Object Dataset Renders

4. **Post Processing:** A number of noise sources unique to spacecraft environments are identified and physically modelled. The noise sources cannot be applied commutatively and must follow the appropriate ordering.

With the render pipeline defined, it is used to render a number of datasets. First the **Cubesat** dataset is used to validate the render pipeline and perform analysis on the domain gap between the renders and a real world laboratory setting. Second, a **Rocket Body** dataset is constructed to explore orbits and unrestricted poses. Third and Fourth, two **Minerva-II2** datasets are rendered to support the pose estimation and deployment analysis in Chapter 3. Finally, a **Multi-Object** dataset is shared in support of future work.

Chapter 3
Spacecraft Pose Estimation

As discussed in chapter 1, spacecraft close proximity navigation using monocular camera systems has historically relied on algorithms that minimize the fit error between handpicked image features and the corresponding 3D model's features. However, recent works have introduced CNN architectures as a candidate for monocular pose estimation. CNN pose estimation strategies require large amounts of images to train. In this chapter a number of pose estimation strategies enabled by the chapter 2 dataset rendering will be explored, culminating with the challenging Hayabusa 2 Minerva-II2 deployment images. By the end of this chapter, the reader will be familiar with different monocular pose estimation strategies, their limitations and the most promising architectures for the challenges of spacecraft pose estimation.

3.1 Monocular Pose Estimation Strategies

As reviewed in chapter 1, a variety of different monocular pose estimation strategies exist. For the purposes of this work, they can be coarsely categorized into straight-shot and correspondence based. **Straight-shot** methods refer to NN architectures which output the pose directly or otherwise provide pose information directly. **Correspondence** methods identify a number of 2D features in the image and the *corresponding* feature on a 3D model of the target. Correspondence methods assume an injective 2D-3D feature relationship and then minimize the error between the reprojected real features and the image-identified features.

A description and introduction of common pose estimation strategies is provided in the following sections 3.1.1–3.1.3. If the reader is already familiar with the different pose estimation approaches, they are invited to skip to section 3.2 for the application and experiments of this work.

In the context of the following strategies, a number of assumptions are made:

- **Available Model:** The target geometry is known (i.e. the object is known or *seen*).
- **Lost in Space:** A-priori knowledge of the target pose is unavailable.
- **Noncooperative Target:** Unique identifying features such as light emitting diodes or matrix barcodes are not present on the target spacecraft.
- **Known Camera:** The camera intrinsic matrix is known.

3.1.1 Correspondence Identification

Correspondence methods can use conventional computer vision methods or CNN architectures to identify the feature correspondences.

Conventional Feature Identification

A conventional feature identification strategy will first extract a feature and then will build the correspondences.

Feature Extraction: From the image, features are identified. A common feature identifier is an edge detector [112], although features distinct to a particular filter kernel are also possible. In [17], the author achieves good results with a sequence of commonly available techniques, namely: low-pass filtering, Canny-edge detection and the Hough transform [113]. The output of such a sequence would be a grouping of *significant* lines corresponding to the target object's primary geometry. For context, an idealized theoretical line segment grouping is hand selected on a real Minerva-II2 image and provided in figure 3.1; the hand selected line segment features are shown in red.

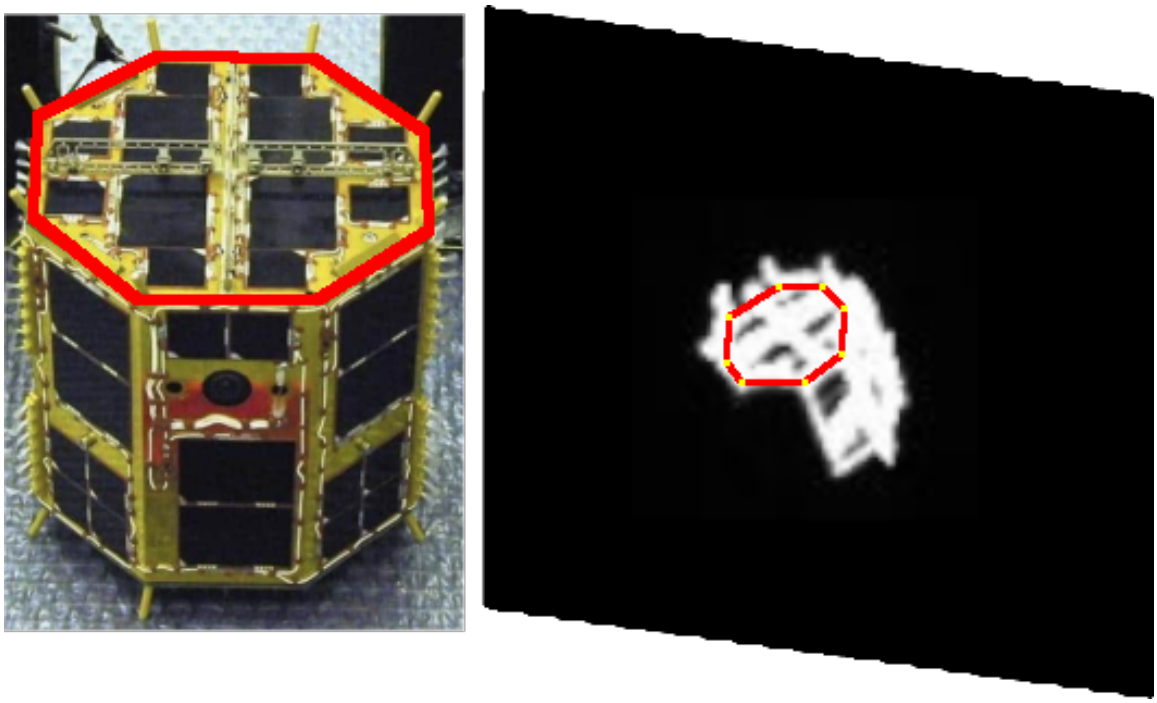


Figure 3.1: Idealized Hand Crafted Feature Selection on a Real Minerva-II2 Image

Correspondence Building: Next the identified image features must be associated with the corresponding features of the target model. Uniquely identifying line features is not trivial; a line segment projected onto the image plane has an infinite number of associated line segments in Euclidean space as showcased in figure 3.2.

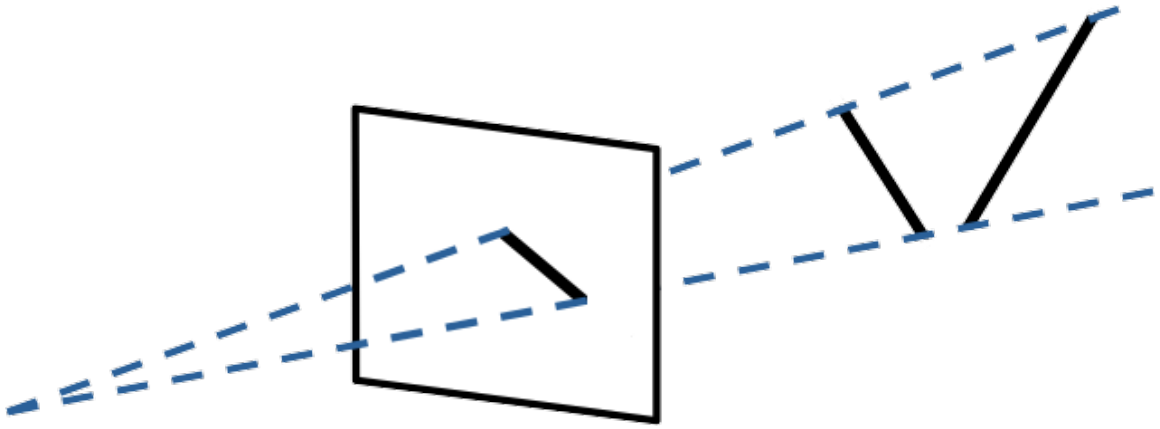


Figure 3.2: Infinite Re-projection Solutions for a Line Segment

Therefore, to associate correspondences between the image edges and the reprojected pose edges, an initial pose guess is required. Strategies exist for choosing an initial pose [114], but in practice a grid search over possible poses may provide the best initial guess while observing the reprojection error[1], e_{reproj}:

$$e_{reproj} = \sum_{i=1}^{N} \sqrt{(x_{proj_i} - x_{feat_i})^2 + (y_{proj_i} - y_{feat_i})^2} \tag{3.1}$$

for all N identified image feature points, x, y_{feat} as compared to the reprojected points, x, y_{proj} for the initial pose guess. The pose guess with the lowest reprojection error is selected, an injective 2D–3D correspondence relationship is constructed and pose estimation (i.e. correspondence fitting) can then occur.

CNN Feature Identification

Although recent CNN pose estimation architecture trends are tending to denser feature correspondences [116], the underlying strategy remains the same: identify 2D image features corresponding to 3D model features. In contrast to the conventional feature identification approach, the CNN feature identification selects injective 2D–3D correspondences in advance and then trains a network to extract the correspondences; specifically the network is trained to identify 2D *keypoints* for preselected 3D correspondences. Various network architectures exist, but the current trend of keypoint detector CNNs typically consist of an Encoder-Decoder-Head framework as shown in figure 3.3.

1. An **encoder** or *backbone* extracts feature maps of gradated granularity from an input image. The extracted feature maps are referred to as an embedded *feature space*.

[1]The Average Distance Indistinguishable (ADI) metric [115], discussed in chapter 4 is also a suitable error function for pose error minimization.

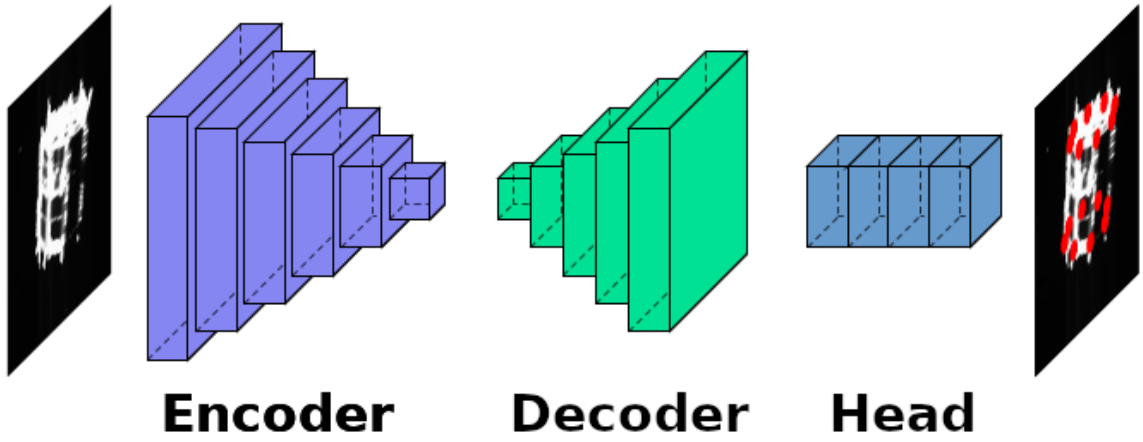


Figure 3.3: Generic Keypoint Extraction CNN Architecture

Convolutional layers[2] dominate the majority of the encoder's operations. In practice most backbones perform multiple convolutions at each layer and thus the number of channels (i.e. the *depth*) tends to grow while the feature map height/width shrink. The result is an embedded feature space with dimensions that do not match the original input image.

2. A **decoder** interprets the embedded feature space. Often some of the reduced height/width dimensions are recovered as keypoint prediction is a spatial task. Similar to the encoder, the decoder's operations are dominated by convolutional layers.
3. One or more **head(s)** then regress the desired information from the spatially correlated decoder output. Head operations are typically dominated by fully-connected layers[3].

In theory, any set of 2D–3D keypoints can be chosen, including keypoints located off of the 3D model or a dense array of keypoints. In the Minerva-II2 case, 16 keypoints corresponding to the major geometric vertices are selected, as shown in figure 3.4.

Different loss functions can be used to train the network. One approach is to use the ground truth pose to reproject the 2D keypoint estimations into 3D space for comparison to the ground truth keypoints. Another approach is to define the loss function in the 2D image plane. In either case, a simple L1 or L2 norm can be used

$$Loss = \sum_{i=1}^{N} \{L_1 | L_2\} | p_i^{GT} - p_i^{est} | \tag{3.2}$$

[2]In machine learning, a convolutional layer refers to the two dimensional convolution of two matrices. An input image, feature map or *channel*, $C \in \mathbb{R}^{m \times n}$, is convolved with filter kernel, $F \in \mathbb{R}^{k \times k}$. Similar to one dimensional convolution, padding and stride can be adjusted. There are typically multiple filter kernels at each layer resulting in a total number of output channels, $C_{out} = num(C_{in}) \times num(F)$.

[3]In machine learning, a fully connected or *dense* layer refers to linear layers where every input node is directly connected to every output node with a unique weight.

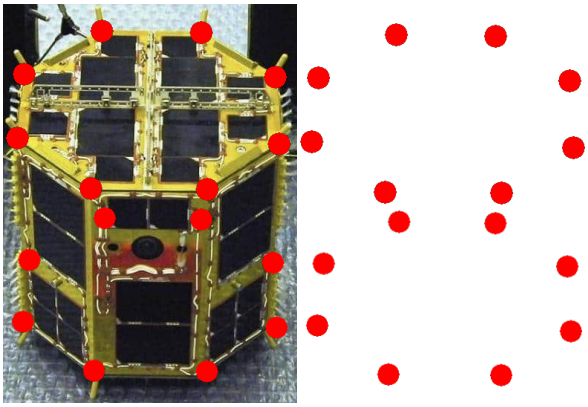


Figure 3.4: Minerva II2 Keypoints

for all N ground truth keypoints, p^{GT}, and the network estimated keypoints, p^{est}. In the author's experience, mapping each point to a heatmap can help with particularly challenging datasets (idea adopted from [27]). If two keypoints are far away, the heatmaps may still overlap which can help the network converge. An image of two keypoints with their associated overlapping heatmaps is shown in figure 3.5.

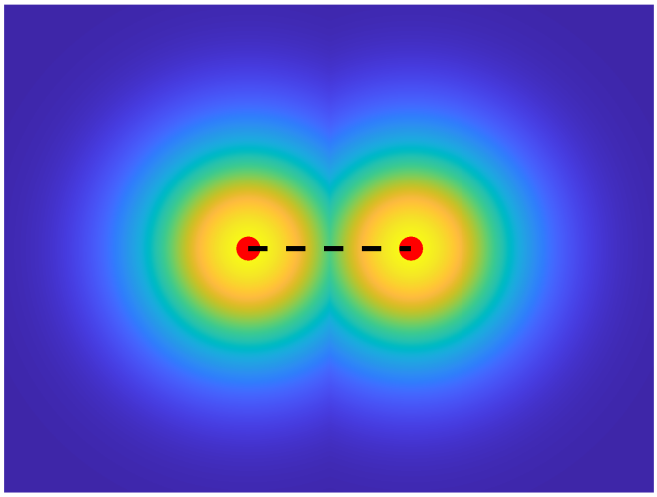


Figure 3.5: Keypoints Projected onto Heatmaps for Loss Functions

The heatmap standard deviations can be reduced during training to increase accuracy. For this formulation, the loss function becomes the minimization of the heatmaps, $h(p_i)$ as

$$Loss = \sum_{i=1}^{N} \{L_1 | L_2\} | h(p_i^{GT}) - h(p_i^{est}) | \tag{3.3}$$

3.1.2 Correspondence Fitting

Once injective 2D–3D feature correspondences are identified, a pose fitting algorithm is used to minimize the fit error, $e_{fit} = \Delta_T + \Delta_{RoT}$, for equation 2.9's translation error, $\Delta_T = T_{real} - T_{proj}$ and attitude error, $\Delta_{Rot} = |Rot_{real} - Rot_{proj}|$. Two popular strategies are the Newton-Raphson method and the PnP problem formulation.

The **Newton-Raphson** method linearizes the fit error about an initial pose estimation. At each iteration step, i, the linearized fit error derivatives are evaluated at the current pose and the pose is reprojected for step, $i+1$ [17]. In this way the fit error will be iteratively reduced. The Newton-Raphson method requires a good initial pose guess for a good fit, otherwise it can converge to a local minimum. A strength of the Newton-Raphson method is its robustness at handling noisy keypoints.

The **Efficient Perspective-n-Point (EPnP)** assumes that the correspondence points can be expressed as a weighted sum of four virtual control points [117]. The reprojection equations are then reformatted into a set of linear equations with the four unknown virtual control points (3 DoF each) as $[\hat{x}]_{12x1}$. The linear system of equations is then solved for $M\hat{x} = 0$. The EPnP methods requires that the correspondence points not be co-linear and also suffers if the correspondences are of poor quality (high noise). PnP solutions also require at least 4 points to converge to a solution.

EPnP and the Newton-Raphson method can be used in tandem, first determining an initial pose estimation with EPnP and then improving the accuracy with Newton-Raphson refinement. However, as explored in [26] the Newton-Raphson method's computational runtime is almost an order of magnitude higher than EPnP. As will be explored in chapter 4, there is a great desire to explore less computationally demanding pose estimation solutions.

3.1.3 Straight-Shot Approach

With the recent surge in machine learning, a number of novel CNN pose estimation architectures have appeared. As discussed in section 1.2.2, some of these architectures avoid the usage of correspondences altogether in favour of a different problem formulation. The most successful straight-shot architecture from the Spacecraft Pose Estimation Dataset (SPEED) challenge was URSONet [2, 118]. As seen in figure 3.6, URSONet implements a simple Residual Network (ResNet) [119] as the backbone to extract features from the image, which are directly fed into a location regression head and an orientation head.

The location regression head can regress the translation vector, $T_{[3x1]}$, directly with a number of fully connected layers. The orientation estimation is less straightforward. A direct regression of an orientation quaternion or Euler angles is possible but not well suited to CNN architecture problem formulations: quaternions are non-injective; Euler angles are not commutative and cyclical; and rotation matrices do not formulate into well defined regression

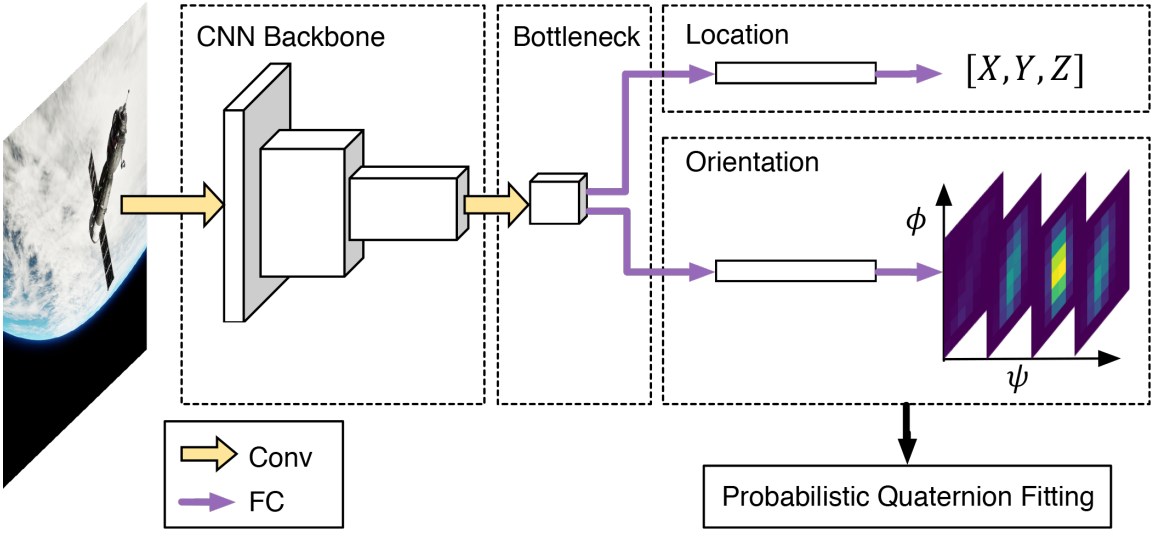


Figure 3.6: Example Straight-Shot CNN Architecture [2]

problems.

Another approach is to reformulate the orientation estimation into a classification problem and to select from predefined orientations. The orientation head can output an activation weight, $w_i \in [w_1, .., w_N]$ associated to a set of N bins, $b_i \in [b_1, .., b_N]$. Each bin corresponds to a discrete rotation. At inference time, the network outputs the bin activation weights, and an orientation estimation quaternion is fitted to the activations by minimizing the weight least squares:

$$q^{est} = \arg\min_{q} \sum_{i=1}^{N} w_i (1 - b_i^{\top} q^{GT})^2 \tag{3.4}$$

In this way, no specific reference to a feature correspondence is made. Increasing the number of bins, N, reduces the quantization error but increases the branch size by $O(N^3)$ resulting in a much bigger network head to train. Without incorporating post processing, the straight-shot CNN network size increases dramatically for higher precision orientation classification. In practice, the formulation of the orientation estimation is the weakest link in straight-shot CNN pose estimation pipelines [17].

3.2 Minerva-II2 Pose Estimation Experiments

The MIcro Nano Experimental Robot Vehicle for Asteroid (Minerva) rovers were deployed from the Japanese Hayabusa 2 deep space probe to investigate the asteroid Ryugu [120]. The Hayabusa 2 deployed the Minerva-II2 rover on October 3rd, 2019. Unfortunately, due to technical difficulties with the Minerva-II2 rover and due to the nature of the release mechanism, the rover telemetry and its Ryugu insertion trajectory were unavailable [3]. Consequently, the Minerva-II2 was released as a known mass for Ryugu microgravity observations

using the Hayabusa2 Onboard Navigation Camera (ONC) telescopic camera [63, 64]. For context, plots[4] from JAXA's orbit insertion strategy report are shared in figure 3.7.

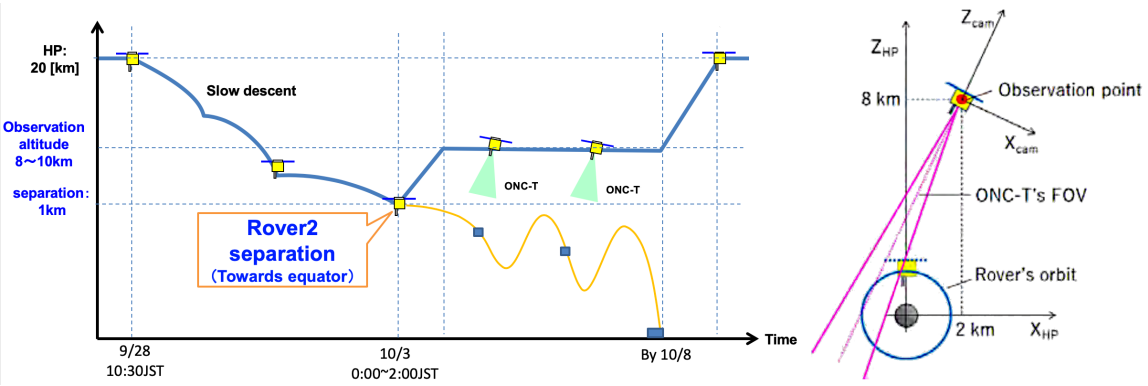


Figure 3.7: JAXA Minerva-II2 Rover Deployment Summary [3]

JAXA used images captured by the Hayabusa 2 ONC telescopic camera to confirm orbit insertion simulations. These observations were then back-propagated to estimate the initial release velocity of the Minerva-II2 rover. Example images of the Minerva-II2 rover, as captured by the Hayabusa2 ONC telescopic camera, are shown in figure 3.8.

Figure 3.8: Minerva-II2 Rover Deployment Telescopic Camera Images [3]

3.2.1 Minerva-II2 Problem Definition

In addition to the telescopic camera images, JAXA also recorded images using the ONC Wide Angle 2 (W2) camera during deployment; the W2 camera images were not used in their orbit insertion velocity analysis. Cropped real W2 Minerva-II2 images were previously shown in figures 2.12 and 2.25. For context again here, a collage of the first 20 Minerva-II2 rover images captured by the Hayabusa 2 ONC W2 camera system are shown in figure 3.9.

The images were captured at 3 second intervals for a total of 61 images. It is evident that after image 21, the Minerva-II2 rover has receded so far from the camera, that it appears

[4]The HP coordinate system refers to a Ryugu-specific coordinate system as shown previously in figure 2.23.

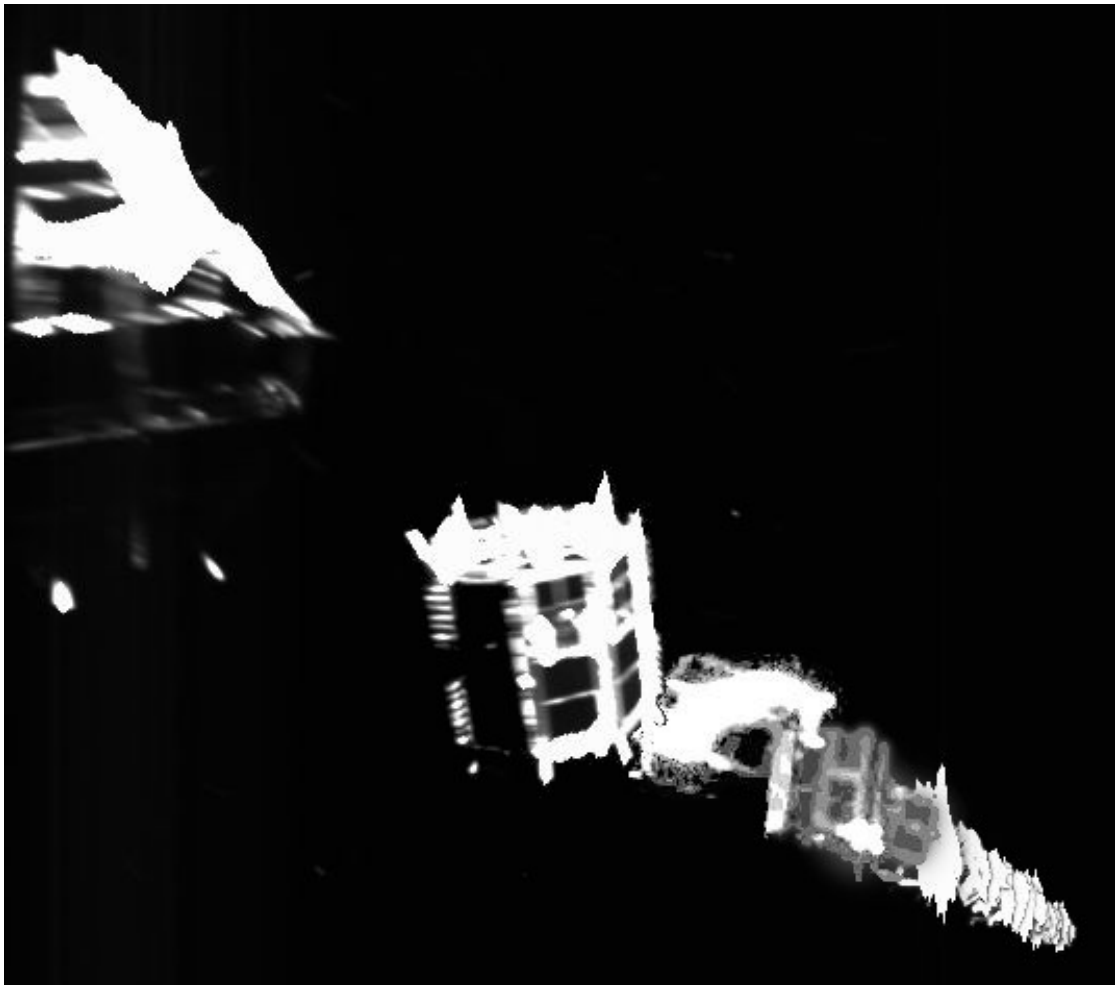


Figure 3.9: Minerva-II2 Rover Deployment Wide Angle Camera Image Collage © JAXA

as little more than a featureless white dot. The problem, as it was given, was to estimate the deployment velocity of the Minerva-II2 rover using only the W2 camera images as an independent verification of the JAXA telescopic orbit insertion analysis. A secondary objective was to estimate any pose information, if possible.

The pose estimation strategies described in the previous section 3.1 are applied in the following sections. Results for a **conventional** correspondence matching approach, a **straight-shot CNN** approach and finally a **CNN FE** with post processing fitting approach are shared.

3.2.2 Conventional Correspondence Matching Approach

Features are identified using two methods: a manual **Hand Selection** procedure and by a customized, but automated, **Edge Extraction** procedure.

Hand Selected Features

As there are only 61 deployment images, arguably 20 with discernable features, it is feasible to hand select features on the images. Due to its relatively uniform dimensions, the Minerva-II2 rover can be approximated as a sphere, the advantage of a sphere being that the widest sphere dimension perceived in an image corresponds to the sphere's diameter. Furthermore, the normal vector of the image plane containing the perceived diameter is the same as the position vector from the camera to the sphere CoM. Therefore, the photo-metric trigonometry is greatly simplified and no initial pose is required. For clarity, a small camera-sphere trigonometry schematic is shared in figure 3.10. The modelled sphere and hand selected points defining the sphere's diameter for W2 deployment image 2 are shown in figure 3.11.

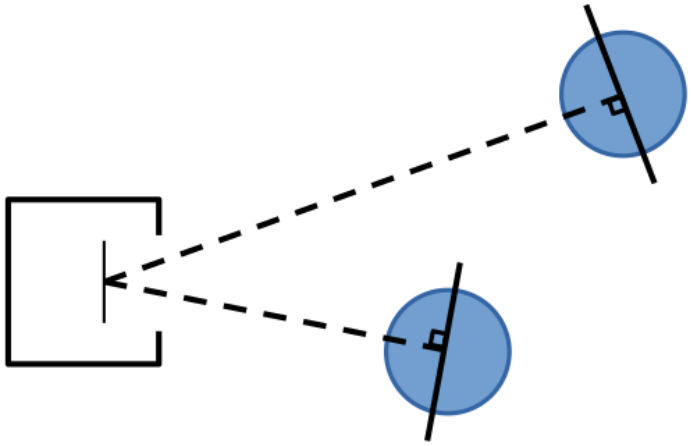
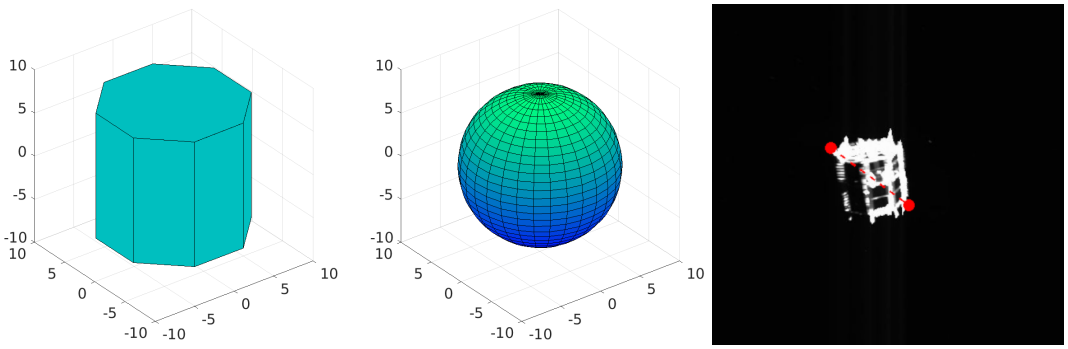


Figure 3.10: Simplified Camera Trigonometry for a Sphere



(a) Dimensions of Modelled Sphere [cm] **(b)** Im2 Hand Selected Points

Figure 3.11: Minerva-II2 Localization by Hand

Edge Extraction Features

Conventional feature extraction techniques (section 3.1.1) such as an edge detector can also be used to identify features on the image. The output of three different edge detectors applied

to the Minerva-II2 rover images are shown in the top row of figure 3.13. The two mutually farthest features are then automatically selected to define the modelled sphere diameter.

Location Estimation

With these selected features, the distance of the sphere, *dist* can be simply estimated using similar triangles:

$$\frac{f_{cam}[pix]}{d_{proj}[pix]} = \frac{dist[m]}{d_{sphere}[m]} \tag{3.5}$$

where the sphere diameter, $d_{sphere}[m]$ is known, the perceived projected diameter, $d_{proj}[pix]$ is being measured and the calibrated camera focal length, $f_{cam}[pix]$ is known. Once the distance is calculated (and distortion has been accounted for), the Euclidean location can be calculated using the calibrated camera's known FoV as shown in figure 3.12 with:

$$\frac{\varphi[rad]}{FoV[rad]} = \frac{r[pix]}{FoV[pix]} \tag{3.6}$$

where FoV is the calibrated camera's known FoV across a specific dimension in radians (and measurable in pixels). The distance from the camera's principal point is then measured as r $[pix]$ and the off-center angle, φ can be calculated in radians. The problem here is formulated in cylindrical coordinates to better accommodate barrel distortion effects, but once the target's location is known, any coordinate system can be used.

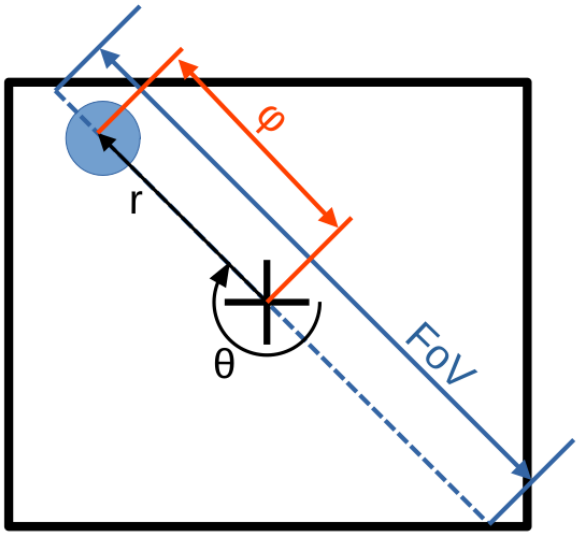


Figure 3.12: Location of Target in Camera Cylindrical Coordinates

Velocity Estimation

Using the hand selection approach and the edge detector selection approach, the deployment velocity of the Minerva-II2 rover is estimated using a least squares fit to the 1^{st} order

polynomial:

$$\hat{\vec{X}}_k = \vec{X}_0 + \vec{\alpha} t_k \tag{3.7}$$

where

- $\hat{\vec{X}}_k = \left[\hat{X}_k, \hat{Y}_k, \hat{Z}_k\right]^\top$ is the projective geometry measured target location for image k;
- $\vec{X}_0 = \left[X_0, Y_0, Z_0\right]^\top$ is the target location at 0 s;
- the deployment velocity, $\vec{\alpha} = \left[V_X, V_Y, V_Z\right]^\top$, is the parameter to be fitted; and
- t_k is the time at which image k was recorded.

Velocity estimation results are shared and compared later in table 3.1.

Pose Estimation

In addition to the velocity estimation, the target pose can theoretically be fitted to identified edge features as discussed in section 3.1.2. Feature fitting is a non-trivial task in optimal conditions [17]. For the noisy and symmetric W2 Minerva-II2 deployment images (see figures 2.12 and 3.9), this challenge is increased; image features corresponding to real features on the Minerva-II2 rover are unable to be consistently extracted. As observable in figure 3.13, only the Canny pipeline extracts *some* lines corresponding to a real Minerva-II2 dimension. Consequently, an automated pose fitting pipeline for the Minerva-II2 W2 deployment images cannot be created using conventional feature extraction methods.

Nonetheless, a manually supported pose fitting is attempted for the highest quality deployment image. The performance of three different edge detectors is shown in figure 3.13. An initial pose is selected manually. With the initial pose, the 2D–3D correspondence injective relationship could be re-established and a PnP solver [121] is employed to obtain the final pose. The associated fitted pose is shown in figure 3.14. The 2D keypoint fit error is shared later in table 3.2.

As this process requires significant human intervention, automated pose estimation using conventionally extracted edges is not feasible for this Minerva-II2 case study. A neural network approach is next explored.

3.2.3 CNN Feature Extractor Approach

The CNN FE approach is shared here. Due to significant image noise and to avoid significant image resizing losses, a larger high resolution CNN architecture is selected: HRNet [122][5]. HRNet was initially developed for an object detection and recognition dataset, COCO [123]. Different versions of HRNet were later adapted for keypoint extraction [27].

[5] A multi-stage network architecture featuring an object detection for Region of Interest cropping followed by a smaller keypoint extraction network would also avoid image resizing losses.

Figure 3.13: Image 2 Minerva-II2 Edge Detector + Hough Transform Comparison

Figure 3.14: Image 2 Pose Fit Example (Manual Supported PnP)

The network is trained using the section 2.3.4 unmodified (symmetric) synthetic Minerva-II2 dataset. Typically a keypoint extraction network has several outputs, one corresponding to each keypoint. Due to the Minerva-II2's symmetry it is not possible for the network to

differentiate one vertex from another, resulting in network training that does not converge. Therefore, keypoints are selected as shown in figure 3.4, but instead of 16 keypoint outputs, a single output with all 16 keypoint heatmaps superimposed is used; the injective 2D–3D correspondence information is dropped in favour of identifying any keypoint. This is effectively a hybrid conventional-CNN approach. Thus the loss function (equation 3.3) is no longer a summation of the individual keypoint errors but a single heatmap comparison.

$$Loss = |h(p^{GT}) - h(p^{net})|_{L_2} \tag{3.8}$$

The heatmap size (standard deviation, σ) is modified throughout training starting with $\sigma = 10$ at epoch 1 and decreased piecewise to $\sigma = 2$. Due to the size of HRNet, the network is quick to overfit to the synthetic training data, resulting in worse performance on the real images. Comparing models trained to different durations, a noticeable tradeoff is observed. More training results in less keypoints being identified, but the identified keypoints are of higher quality. Training duration vs the total identified keypoints summed across all 20 images is shown in figure 3.15 (i.e. the network trained for one epoch identified 110 keypoints total across the 20 test images). Example keypoint outputs for a single real test image are shown in figure 3.16.

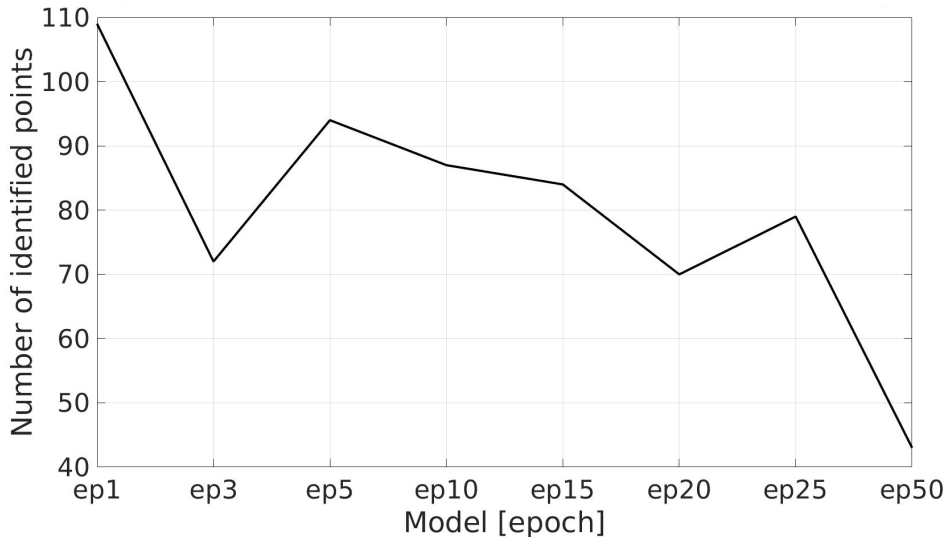
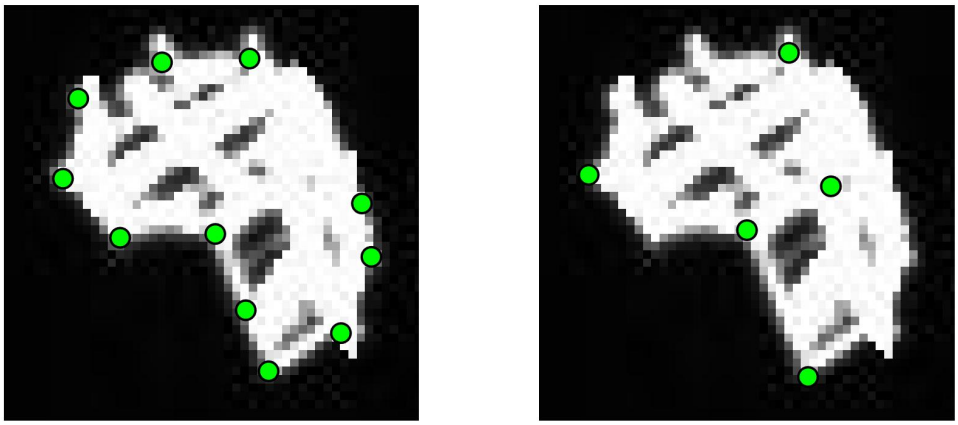


Figure 3.15: Network Model Training Duration vs
the Number of Identified Keypoints Summed Across All Test Images

A network model with less training (lower epochs) has less defined heatmaps, but after running a peak localization function it can be seen that several keypoints are defined. The keypoints seem to correspond to high contrast and obtuse geometry. The less-trained network model is also more prone to incorrectly labelling locations as keypoints. As seen in figure 3.16 left, 11 keypoints are identified, two of which are not real keypoints.

In contrast, a network model with more training (higher epochs) has sharply defined heatmaps, but with fewer keypoints defined. The keypoints are of higher quality (a sharply



(a) 1 Epoch Training **(b)** 50 Epoch Training

Figure 3.16: Network Model Training Duration Keypoint Output Comparison

defined heatmap peak is effectively a higher confidence output) and are not limited to regions of high contrast as seen in figure 3.16 right: the rightmost keypoint is effectively surrounded by sensor saturation. The identification of the rightmost keypoint signals that the network model is effectively using knowledge of the target's geometry to contextually locate keypoints. However, as discussed in section 3.1.2, PnP solutions require a minimum of 4 keypoint correspondences to converge. In the current case, a network model training for 50 epochs will not provide sufficient keypoints for each of the test images. Instead an ensemble fusion approach is adopted. If a higher trained network (e.g. 50 epochs) supplied 4+ keypoints for a particular image, those keypoints are taken, otherwise additional keypoints from lower trained networks (e.g. 25 epochs) are included to obtain 4+ sufficiently high confidence keypoints.

Without injective 2D–3D correspondence information, a section 3.1.2 post processing pose fitting algorithm cannot be directly applied. An initial pose guess is again required to re-establish injective 2D–3D correspondences. The initial pose guess is first supported by estimating the Euclidean location similar to the conventional approach shown in section 3.2.2. Assuming the spherical model geometry, the two most separated 2D keypoints are selected and equation 3.5 is applied to estimate the radial distance. Equation 3.6 is used to estimate the 3D location. Again, a least squares velocity fitting of this 3D location is applied to refine the 3D locations. The CNN FE velocity fitting results are shared and compared later in table 3.1.

With a refined 3D location, a geometric model of the Minerva-II2 can be fitted to the keypoint features, similar to the conventional approach. However, now there is a distinct advantage, as every keypoint feature is known to belong to a vertex of the geometric model. Detected edge features not corresponding to geometry do not need to be manually rejected.

Instead the entire process can be automated.

First an exhaustive 3D matrix grid search is performed at Euler angle intervals of 30 degrees resulting in 12 angles to investigate for each degree of freedom creating a $12^3 = 1728$ matrix to search. For each element of the search, the root mean square error of a keypoint and the nearest neighbour projected keypoint is calculated. The 10 lowest error results are selected and next a PnP solver is run to obtain the pose. The best fit error results for image 2 are compared to the conventional fit results in table 3.2.

It is worth noting that an initial pose guess can be selected manually for the first few images, thus circumventing the first grid search. However, as seen in figure 3.9, the image quality deteriorates rapidly preventing manual assistance. Given a sufficient framerate and images, a resolved tumble propagation could theoretically be able to provide initial pose guesses for refinement on subsequent images. However, due to the target's symmetry and the W2 $\frac{1}{3}$ Hz framerate, a rigid body tumble integrator cannot be initialized to converge to the real Minerva-II2 estimated attitudes. The framerate of $\frac{1}{3}$ Hz does not sufficiently constrain the harmonics of possible rotation rates as per the Nyquist sampling theorem [124]. An expanded discussion, theoretically fitting a motion estimation to the Real Minerva-II2 deployment images, is available in appendix **??**: *Minerva-II2 Deployment Tumble Motion Fitting Limitations*.

3.2.4 Straight-Shot CNN Approach

For the CNN straight-shot approach, an architecture similar to what was introduced in section 3.1.3 is adopted. To maintain a fair comparison with the CNN FE approach, HRNet is again adopted as the backbone (see figure 3.6). However, due to the symmetry of the Minerva-II2 rover, a standard orientation estimator head is unable to converge. Instead, the orientation head is replaced with a head similar to the location head to regress a 3x1 vector defining the axis of symmetry. The network is trained for 10 epochs with the loss function

$$Loss = \sum_{i=1}^{N} \Big(\{L_1|L_2\}(T_{GT} - T_{net}) + \{L_1|L_2\}(|ax_{GT}| - |ax_{net}|) \Big) \tag{3.9}$$

where

- $T_{GT,net}$ represent the ground truth and network prediction locations respectively; and
- $|ax_{GT,net}|$ represent the ground truth and network axis of symmetries respectively.

In this way, the network is trained to output two 3x1 vectors. Similar to the conventional and the CNN FE pipelines, the location estimation is included in a least squares fitting velocity estimation with the results shared in table 3.1. The axis of symmetry for image 2 is compared against the two previous methods in table 3.2.

3.2.5 Results and Comparison

Finally, the results from all the previously discussed pose estimation methods are compared and discussed.

Minerva-II2 Deployment Velocity Estimation

Recall that JAXA performed an orbital insertion analysis of the Minerva-II2 rover deployment using the ONC telescopic camera. The estimated orbit was then back-propagated to determine the initial deployment speed of **0.1924** $\frac{m}{s}$. This work's deployment velocity estimation results are shared in table 3.1.

Table 3.1: Minerva-II2 Deployment Velocity Estimation Results

Parameter	Hand Selection	Edge Extraction	CNN FE	Straight-Shot
Velocity $\frac{mm}{s}$	$[6.7, 19.8, 189]$	$[6.8, 19.8, 190]$	$[4.0, 30.5, 191]$	$[4.5, 30.2, 191]$
Velocity Norm $\frac{m}{s}$	**0.1900**	**0.1908**	**0.1934**	**0.1936**
RMSE (↓ better)	0.3385	0.2477	0.3333	0.3389

The CNN FE approach delivered the most accurate velocity estimation with a speed of 0.1934 $\frac{m}{s}$ and deviation of 0.01 $\frac{m}{s}$ from the JAXA estimate. It is also interesting to note that the hand selection of points resulted in the worst velocity estimation and the largest Root Mean Squared Error (RMSE) (lowest accuracy and lowest precision). The straight-shot CNN results had a smaller deviation but a larger RMSE than the edge matching (higher accuracy but lower precision).

It is of particular interest to observe the RMSE values across all four methods. The edge extraction approach exhibited the lowest RMSE, however it must be clarified that the hand selection and edge extraction methods were optimized for an adjacent problem: estimating the diameter of a sphere of similar size to the Minerva-II2 rover. In this way, the hand selection and edge extraction methods did not select features corresponding to a particular geometry, but any geometry that was mutually farthest separated. Worded another way, the hand-selectiton and edge-extraction methods were solving a simpler parallel problem and consequently the RMSE value for edge extraction was the lowest.

In contrast, the CNN FE was trained to identify 16 specific features. While it was likely that 2 of the 16 features would be located mutually farthest apart, it was not always guaranteed to identify two specific keypoints. As stated above, because the injective 2D-3D correspondences were dropped, it was still necessary to perform a first order approximation using the sphere model to estimate the Minerva-II2 location. In this way, the CNN FE pose fitting process was directly linked to the real deployment velocity estimation problem as opposed to solving the parallel sphere model problem.

The straight-shot CNN outputs a single estimation of the location as opposed to multiple extracted features. Using multiple features for a single location estimation would potentially create smaller deviations in the estimations to achieve a lower RMSE value (i.e. averaging across multiple sources of information results in a better estimate).

Minerva-II2 Deployment Axis of Symmetry Estimation

The JAXA orbital back-progation was unable to provide orientation information because the analysis was limited to observing the location of single point (see figure 3.8). Therefore, the orientation estimations performed in this work must be compared amongst themselves. Again, note that the conventional feature extraction approach is only able to provide a pose estimation with significant manual labour. Therefore the estimated poses are only compared for the W2 deployment image 2. Finally, due to the symmetry of the Minerva-II2 rover, 16 poses are possible and the true SO(3) attitude is unknown. Therefore the estimated axes of symmetry are shared in table 3.2. The RMSE refers to the error between the extracted 2D keypoint feature and the estimated orientation reprojected keypoint. As the straight-shot approach did not estimate 2D keypoints, no keypoint loss (equation 3.2) is available for comparison.

Table 3.2: Image 2 Axis of Symmetry Estimation Comparison

Parameter	Edge Extraction	CNN FE	Straight Shot CNN
Axis of Symmetry	$[.285, -.852, 0.438]$	$[.054, -.855, -0.515]$	$[.050, -.858, -0.511]$
Eq. 3.2 Loss [pix]	8.34	1.33	N/A

The CNN FE approach demonstrated a significantly lower equation 3.2 loss. Furthermore, the straight-shot CNN and the CNN FE methods produced closer estimations than the edge-extracting method. A collage of CNN FE axis of symmetry estimations (blue) with one of the 16 possible orientations selected is shown in figure 3.17.

It is worth noting that the straight-shot CNN estimation pipeline did not include any post processing and no small perturbation pose refinement. This represents a dramatic reduction in computational cost. As will be discussed in chapter 4, spacecraft hardware is computationally restricted and thus pose estimation network memory size and throughput speed are important factors to consider when selecting a pose estimation pipeline.

Synthetic Asymmetric Minerva-II2 Results

As discussed previously, the Minerva-II2 rover deployment analysis is uniquely challenging for two reasons: (1) significant image noise and (2) object symmetry. While manageable, this combination represents a worst case scenario for 6D pose estimation that is often avoided.

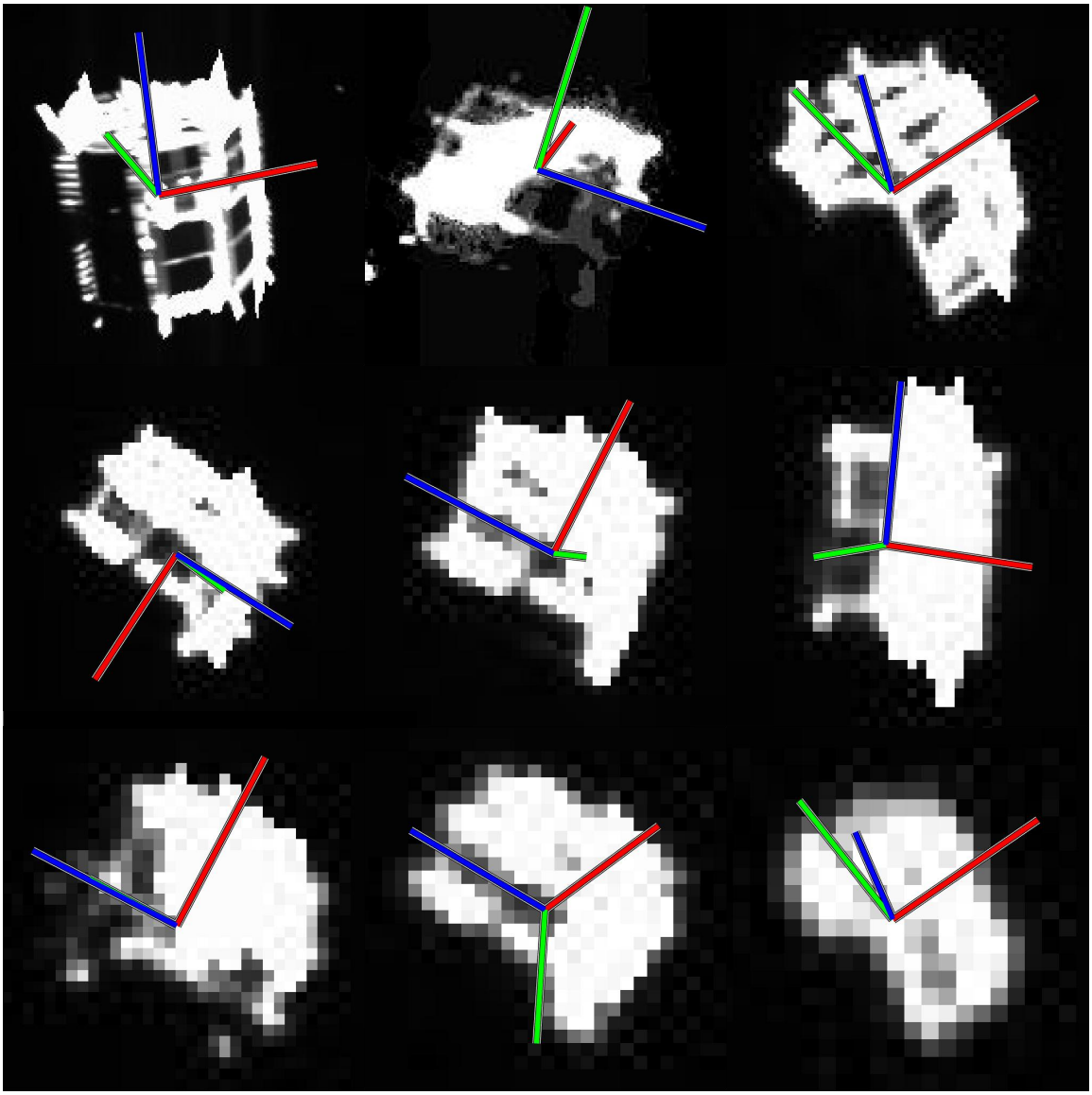


Figure 3.17: CNN FE Axis of Symmetry Estimations (Blue) for W2 Deployment Images

As discussed in section 2.3.3, a modified Minerva-II2 dataset with an asymmetric model was also created. Pose estimation results for the CNN FE pipeline are shown in figure 3.18.

As the training and testing domains are the same (both synthetic), the network model is trained for 50 epochs with little concern of overfitting. The injective 2D–3D correspondence heatmap loss function is used for training (equation 3.3). The dataset still contained significant noise, so again the heatmap size (standard deviation, σ) is modified throughout training starting with $\sigma = 10$ at epoch 1 and decreased piece-wise to $\sigma = 2$. The network is trained to output 16 unique keypoints. A PnP solver can be directly applied to the keypoint output.

The CNN FE is tested on a rigid body tumble motion test dataset. An example collage of pose estimation results are shown in figure 3.18. The CNN FE tumble pose estimations are plotted against the ground truth tumble progression in figure 3.19. Evidently, the pose

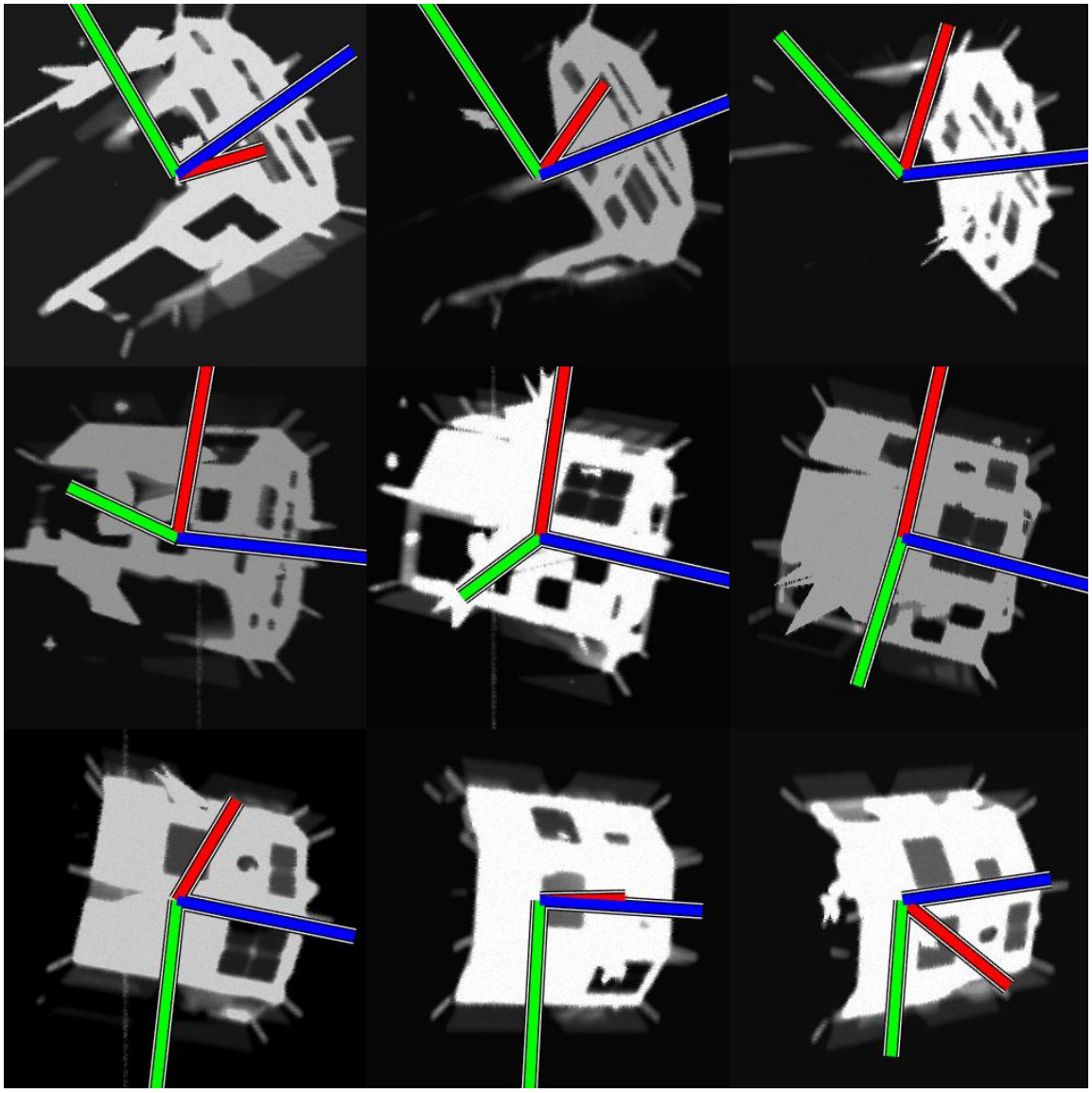


Figure 3.18: CNN Feature Extractor Pose Estimation Results for an Asymmetric Target

estimation of an asymmetric object with a consistent distance to the camera, is a largely simplified case.

3.3 Rocket Body Pose Estimation Experiments

The necessary theory was introduced in section 3.1 and applied to the challenging real Minerva-II2 rover deployment images in section 3.2. In support of chapter 4's network compression work, an additional case is introduced here: the rocket body dataset. The section 2.3.2 3-channel, SO(3) pose and asymmetric rocket body dataset contains a polar orbit (figure 2.6) which results in dynamic lighting conditions. Additionally, the dataset contains a flatspin, a conic motion and uniform random poses.

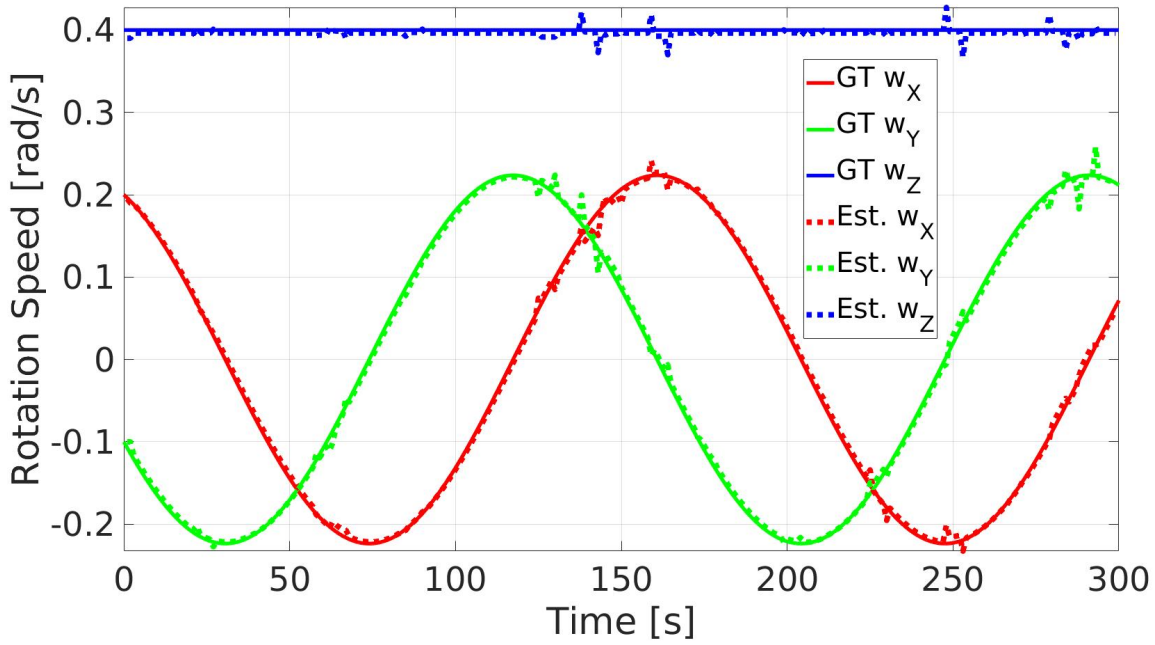


Figure 3.19: Tumble Progression. Ground Truth (GT) vs the Network Model Estimate (Est.)

In contrast to the previous challenging Minerva-II2 case, the pose estimation network size will also be taken into consideration. The two pose estimation pipelines are shared here.

3.3.1 CNN Feature Extractor Approach

In the previous Minerva-II2 section 3.2, the selected CNN FE architecture was large and not feasibly implemented onto smaller hardware. Therefore, a more compact CNN FE architecture is adopted here: Wide Depth Range (WDR) [28]. WDR features an encoder-decoder-head structure as generally depicted in figure 3.3. The WDR encoder or *backbone* is Darknet-53 [125], the decoder is a Feature Pyramid Network (FPN) and the head regresses 3D keypoint locations for a following PnP post processing step.

WDR is trained on the rocket body dataset for 30 epochs. Additional compression experiments are trained for 100 epochs. As the rocket body dataset represents an easier dataset, keypoint heatmaps were not required. A simple L_2 keypoint loss function is used during training (equation 3.2). Results are shared and discussed in chapter 4.

3.3.2 Straight-Shot CNN Approach

An architecture similar to that introduced in section 3.1.3 is adopted; the relatively small ResNet 50 backbone[6] is maintained as this was recommended by the original author [2].

[6]In many studies Resnet 50 [119] and Darknet 53 [125] have been evaluated against one another as backbones for various tasks. In [126], it was observed that Darknet 53 was generally a higher performing encoder. Therefore, this may be an additional opportunity for exploration in this straight-shot CNN pipeline.

The location head is unmodified from section 3.1.3. As a post-processing averse choice, the orientation head is formatted into a regression problem outputting an 11x1 vector representation, v [67] as shown in table 3.3 and figure 3.20. The 3 bit quadrant value indicates which quadrant the target is "pointed" towards.

Table 3.3: Ground Truth Vector Encoding. IP - image plane WF - world frame

Vector	Description	Reference
$[x, y, z]$	Location	WF
θ_{in}	Angle about z-axis	IP
θ_{out}	Angle about x-axis	IP
Θ	Pitch	WF
Ψ	Roll	WF
Ω	Yaw	WF
$[111]$	Quadrant Indicator	WF

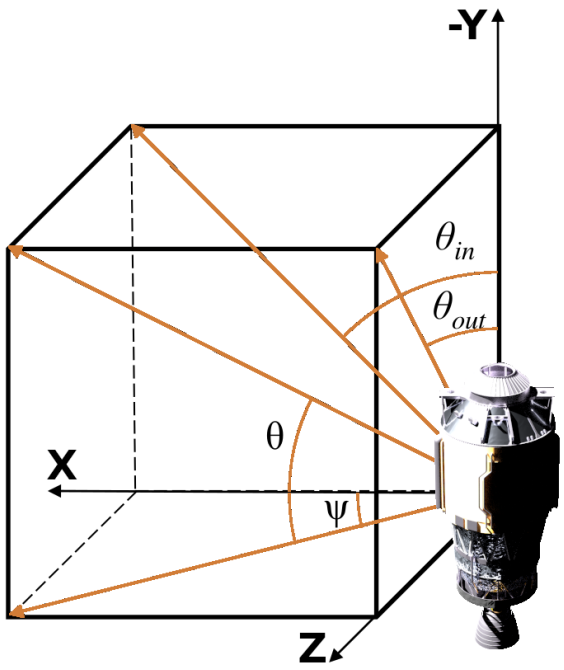


Figure 3.20: Schematic Representation of the Rocket Body Straight-Shot Output Vector

A weighted Mean Squared Error (MSE) loss function with control weight vector α is used during training:

$$L(v^{GT}, v^{net}) = \alpha(v^{GT} - v^{net})^2 \tag{3.10}$$

where v^{net}, v^{GT} are the the network output and ground truth. The index weights within α are adjusted throughout training. A higher weight is assigned to the quadrant value at the

start of training to help the network converge, but then a higher weight is assigned to the image plane values near the end of training to improve accuracy. The Euler angles, although typically a poor regression choice due to their order of operations, maintain utility as they can be directly calculated from the image plane representation as:

$$\theta_{\text{in}} = \arctan\left(\frac{\cos\Theta\cos\Psi}{\sin\Theta}\right) \tag{3.11}$$

$$\theta_{\text{out}} = \arctan\left(\frac{\cos\Theta\sin\Psi}{\sin\Theta}\right) \tag{3.12}$$

To reference components of the 11x1 output vector, refer to figure 3.20. Similar to the section 3.3.1 CNN FE WDR network mentioned previously, this straight-shot CNN architecture is trained for 30 epochs on the rocket body dataset. Additional compression experiments are trained for 100 epochs. The results are shared and discussed in chapter 4.

3.3.3 Architecture Comparison Note

As a final note, it is worth mentioning that during the Stanford SPEED competition [39], the parent of this straight-shot CNN architecture, URSONet [2] outperformed this CNN FE architecture, WDR [28]. However, the author of this work is unable to recreate URSONet's performance. Additionally, when applying URSONet to this rocket body dataset (arguably more challenging than SPEED), it become necessary to increase URSONet's classification heads' size to improve performance. The result is a network both larger than WDR and with worse performance. It is for this reason, that a customized regression vector head was fitted to the straight-shot CNN architecture. As will be seen in chapter 4, the regression head performs well on the conic motion estimation.

3.4 Summary

In this chapter, the preliminaries of computer vision techniques for monocular pose estimation are discussed in the context of spacecraft pose estimation. Three pose estimation procedures are classified and demonstrated:

1. a **Conventional Feature Extraction** method,
2. a **CNN Feature Extraction** method, and
3. a **Straight-Shot CNN** method.

All three methods are applied to the noisy and symmetric Minerva-II2 rover deployment images and compared against an independent JAXA study. The chapter 2 render pipeline is again validated as the CNN pipelines are trained on synthetic images, but tested on real W2 deployment images with favourable results. In this way, the first CNN pose estimation

from deep space images is performed[7]. Additionally, the CNN FE and the straight-shot CNN methods are also tested on a 3-channel asymmetric rocket body dataset with available ground truths.

A summary of the pose estimation methods' applicability to challenging spacecraft images is provided in table 3.4. The **CNN Feature Extraction** method proved the most reliable and most accurate.

Table 3.4: Comparison of Pose Estimation Methods

Method	Location Estimation	Orientation Estimation	Sensitivity to Symmetry	Accuracy
Hand-Selection	Possible	Not Possible	N/A	Poor
Edge-Extraction	Possible	Possible	High	Poor-Mid
CNN FE	**Automatable**	**Automatable**	High	**Best**
Straight-Shot	**Automatable**	**Automatable**	**Variable**	Mid-High

Although largely unaddressed in spacecraft dataset literature [28, 39, 40, 41], the symmetric case was addressed in this work. It is observed that when injective 2D–3D correspondences can not be maintained, the feature extraction methods become more cumbersome as additional grid searches or post processing initial pose algorithms are required to rebuild the 2D-3D correspondences. The **Straight-Shot CNN** architecture proves less difficult to adjust to accommodate symmetric targets.

It is clear that neural network architectures provide significant benefits over conventional methods in terms of accuracy and estimation automation. A symmetric target increases the pose estimation difficulty substantially, but depending on the formulation of the straight-shot CNN, useful information can be regressed without difficulty. Again, the CNN FE approach demonstrates the highest and most consistent accuracy, but required significant post processing compared to the straight-shot CNN approach. Selecting an optimal approach will be application specific.

In the following chapter 4, this idea of network size is addressed quantifiably.

[7]To the best of the author's knowledge, this is also the first symmetric *spacecraft* CNN pose estimation as well. However, there are several symmetric *non-spacecraft* pose estimation results in literature [127].

Chapter 4
Neural Network Quantization

As summarized in chapter 1, not all compression methods can be directly applied to CNN architectures. However, quantization compression methods have no such limitation. In this chapter quantization theory is reviewed; a novel quantization algorithm is validated in collaboration with EPFL CVLab; and experiments with the section 2.3.2 rocket body dataset are shared.

4.1 Quantization Theory

Quantization is the process of mapping continuous or higher precision number representations to a smaller discrete set of lower precision number representations. Quantization is adaptable with many tunable parameters including symmetric/asymmetric quantization, level of granularity, clipping range and different quantization algorithms [128]. The objective of quantization is to maintain network inference accuracy (performance) in the target domain while reducing the network's memory footprint and latency (i.e. maintain the network's accuracy; decreasing the network's size; increase the network's speed).

4.1.1 Quantization Fundamentals

The general formulation of the quantization function, Q, can be written as [129]:

$$Q^b(x) = Int\left(\frac{x}{S}\right) - Z \tag{4.1}$$

Int: The rounding operation, which rounds to the nearest available quantized integer,
$Q^b(x)$: The quantized number (i.e. the b bit precision representation of x),
S: The scaling factor,
x: The high precision number representation (i.e. a CNN activation or filter kernel), and
Z: The quantized output bias (i.e. what zero maps to).

In this way, a high precision number, x is scaled, rounded to an integer and mapped to the quantization output. The scale factor, S, is defined as:

$$S = \frac{\beta - \alpha}{2^b - 1} \tag{4.2}$$

α: The lower clipping range, $\alpha \in x$,
β: The upper clipping range, $\beta \in x$, and
b: The quantization bit width (i.e. $b = 4$ for 4 bit quantization);

where the values of x outside $[\alpha, \beta]$ are first clipped to the range $[\alpha, \beta]$. The range need not be symmetric (i.e. $-\alpha \neq \beta$); however, as this work explores aggressive quantization (i.e. $Q^2(x) \in [-1, 0, 1]$), a non-symmetric range would result in exploding activations[1].

For this work, a symmetric range with bias, $Z = 0$, and a non-uniform[2] bit-width $Q^b(x) = \pm 2^{-b}$ is adopted. The clipping range is selected as $-\alpha = \beta = max(|x|)$. A visualization of the quantization mapping used in this work is provided in figure 4.1; recall the quantized output bias, $Z = 0$, here.

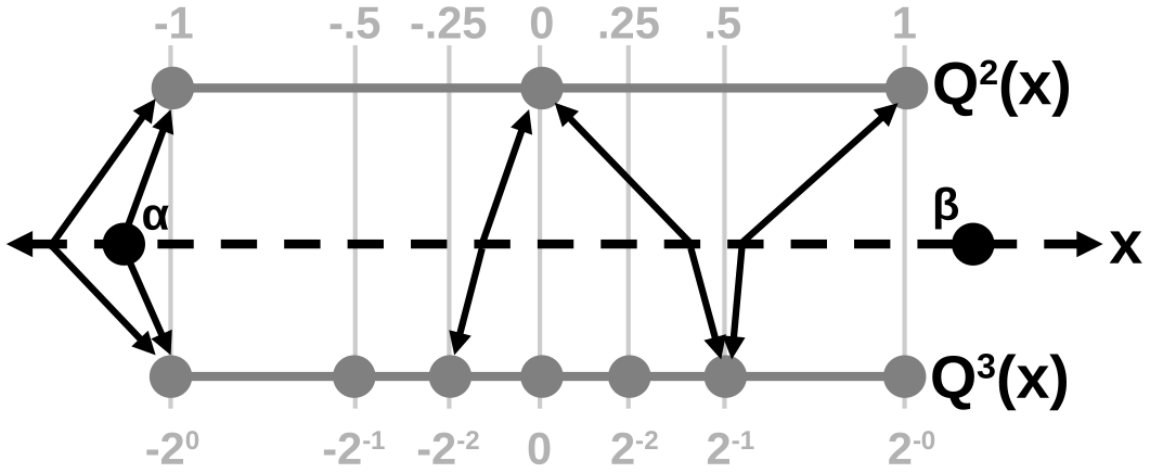


Figure 4.1: Illustration of the Quantization Mapping Function for $Q^2(x)$ and $Q^3(x)$

With setting $-\alpha = \beta = max(|x|)$, it is interesting to observe the relationship between batch normalization[3] and quantization. Activations with a mean, $\mu = 0$, and standard deviation, $\sigma = 1$, are ideal for aggressive symmetric quantization. However, as the batch normalization statistics vary from batch to batch, this can introduce inconsistent jitter before the quantization operation is performed. In [129], the authors address this by introducing a running averaging window over the batch normalization. However, while using the INQ

[1] A non-symmetric ternary bit-width representation such as $Q^2(x) \in [0, 1, 2]$ would result in all the input activations increasing in size. The following layers would require negative ranges to counteract a non-symmetric ternary quantized layer.

[2] In this context, uniform refers to the spacing of the quantization function output. An output of $Q(x) = \pm[0, 1, 2, 3]$ is *uniform* as it has consistent linear spacing. An output of $Q(x) = \pm 2^{-b} = \pm[0, 0.25, 0.5, 1]$ is *non-uniform* as it does not have consistent linear spacing. A nonuniform spacing can be better fit to the distribution of input numbers, x (i.e. a normal distribution may better represent the information in x).

[3] In the context of deep learning, batch normalization refers to the process of standardizing activations by rescaling the activations to a normal distribution with known mean, μ, and standard deviation, σ [130]. Batch normalization can be expressed as $x_{BN} = \frac{x-\mu}{\sigma} + bias$.

algorithm for this work's experiments, the effect of the jitter is observed as negligible and thus batch normalization is completed normally without an averaging window.

To perform back-propagation[4], every operation in a NN must be differentiable, however, the quantization operation is not differentiable. To account for this, an additional approximation must be made using a Straight Through Estimator (STE) [131] during the back-propagation step (i.e. during learning). The derivative of the input-output pair for the quantization layer is treated as the identity function in the clipping range $[\alpha, \beta]$ and 0, otherwise. A schematic illustration is shared in figure 4.2.

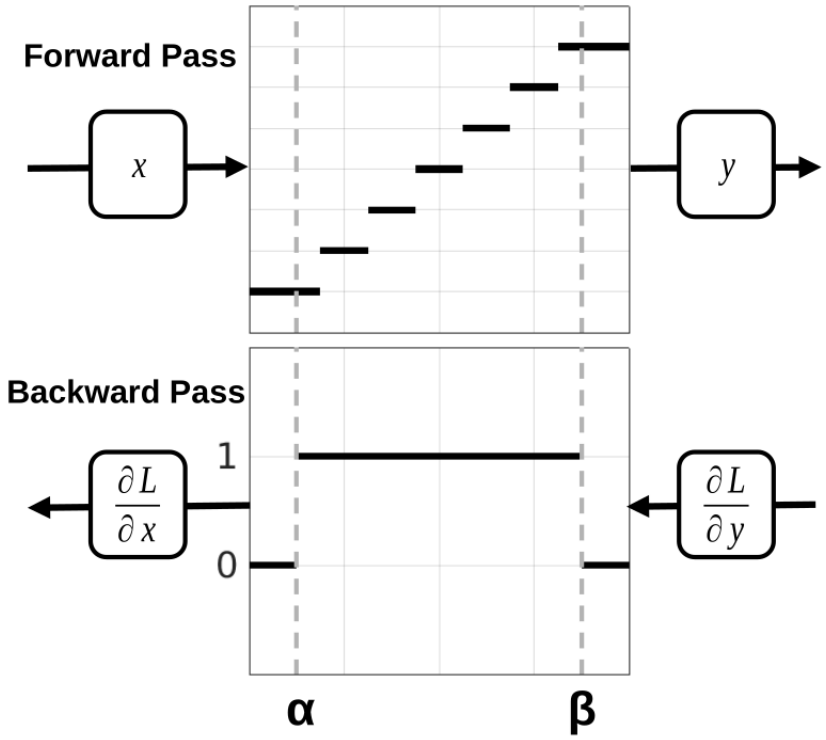


Figure 4.2: Straight Shot Estimator

4.1.2 Incremental Network Quantization (INQ)

With the fundamentals established and the quantization parameters defined, a quantization algorithm can now be applied. As reviewed in section 1.2.3, a number of different quantization strategies exist with advantages and disadvantages. Performance trade-offs are shared in the experimental section 4.2.1. Here, the relatively recent INQ method [132] is introduced; a method that can be applied concurrently with network training to yield excellent results [131].

[4]Back-propagation refers to the learning step after a feed-forward operation. The derivative of the loss function with respect to the input and hidden layers is calculated. The NN weights of a particular layer are updated using the gradient between the corresponding layer's input-output pair.

INQ performs quantization incrementally and cycles between retraining and quantization steps. The process is shared as algorithm 1. The advantage of INQ is that the unquantized portions of the network are better able to adapt to the quantization error being introduced. This is in contrast to a quantization algorithm that quantizes every parameters in a network simultaneously, providing little consistency for the network to maintain good inference accuracy.

Algorithm 1 Incremental Network Quantization (INQ) [132]

Input: Training data
Input: $\sigma_1, \sigma_2, ..., \sigma_N$
Input: $W_l : 1 \leq l \leq L$
for $n = 1, 2, ...N_{epochs}$ **do**
    Update learning rate and policy
    for $l = 1, 2, ...L_{layers}$ **do**
        From σ_n update A_l, A_l^q, T_l
        Quantize the weights in A_l^q
    end for
    Calculate the feed-forward loss
    Update A_l values with gradient descent (retrain)
end for
Output: $W_l^q : 1 \leq l \leq L$

σ_n is the fraction of weights to be quantized at epoch n
W_l is the full precision model (untrained)
W_l^q is the quantized model (trained)
N is the number of epochs to train
L is the number of layers in the network
A_l is the unquantized weights at layer l
A_l^q is the quantized weights at layer l
T_l is a binary matrix specifying quantized weights, where
$[W_l][T_l(0)] = A_l$
$[W_l][T_l(1)] = A_l^q$, and
$count([T_l(1)]) = \sigma_n$

A quantization schedule, $\sigma_n = [\sigma_1, \sigma_2, ..., \sigma_N]$ is created for the number of training epochs, N. As a larger and larger fraction of the network is quantized, the remaining unquantized fraction becomes less and less able to adapt during retraining. For this reason, and similar to [131], a pseudo-logarithmic quantization schedule is adopted as shown in figure 4.3. First The network is quantized in steps of 0.2, but by the end of the training schedule,

the quantization occurs in steps of 0.0125; these quantization steps correspond to lines 7 and 8 of algorithm 1.

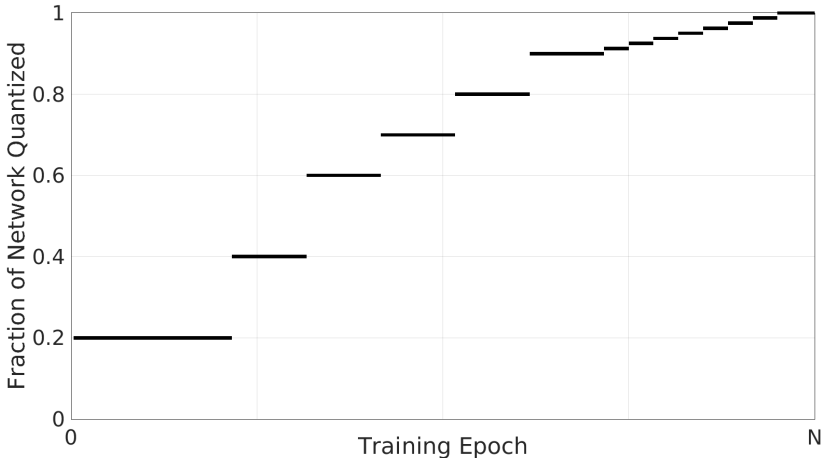


Figure 4.3: INQ Quantization Scheduling

Additionally note that quantization can occur in a structured (entire kernels) or unstructured (kernel fractions) manner; here unstructured quantization is adopted. More explicitly, given the weights of a single layer (kernel), $W_l(i,j) = A_l \cup A_l^q$, the values A_l^q are quantized and the values A_l are retrained before being quantized at a later epoch, n. No spatial restrictions are placed on which parameters are included in A_l^q (i.e. unstructured).

4.1.3 Module-wise Quantization (MQ)

In collaboration with EPFL CVLab, a novel approach to quantization is developed[5]. The network to be quantized is first partitioned into a number of modules that typically correspond to naturally occurring structures in a CNN pose estimator (e.g. encoder, decoder, head). For example, the architecture depicted in figure 3.3 might be partitioned into $K = 3$ modules. A quantization algorithm (e.g. INQ) is then performed on each module sequentially. In this way, MQ behaves as a sort of wrapper for another quantization method. This approach is novel for two reasons.

Module-wise Quantization (MQ) Novelty

Firstly, as introduced in section 1.2.3, training-concurrent quantization methods are categorized as *uniform* or *mixed-precision*. A uniform quantization method quantizes the entire

[5]For clarity, the idea for Module-wise Quantization (MQ) originated at EPFL CVLab [1]. Here, the author contributes by formalizing the idea into the MQ algorithm definition (algorithm 2), and by applying MQ to spacecraft specific network architectures and datasets. Unless otherwise specified, the MQ experiments shared here are performed by the author of this work; extensive additional validation experiments performed at EPFL CVLab can be found in [1].

network to the same bit precision (e.g. 2 bit, 4 bit etc.). A mixed-precision quantization method quantizes individual layers to different bit precisions. A mixed-precision approach has greater flexibility to meet compression/accuracy trade-offs, but this comes at the expense of being more difficult to implement during training and deployment. MQ is the first algorithm to quantize individual modules to different bit precisions, but within the module itself, the bit precision remains consistent. This increases flexibility for meeting compression/accuracy trade-offs with very little increase in training and deployment complexity.

Secondly, MQ addresses the quantization of different modules, sequentially. Even the mixed-precision quantization methods do not perform quantization sequentially [133]. In [1], CVLab makes the observation that this sequential quantization order matters; the process is not commutative. There is an optimal quantization order for each subdivision of the network into K modules. Algorithm 2 contains MQ as defined by this work's author[6].

Module-wise Quantization (MQ) Algorithm

The quantization algorithm has two main steps: the determination of the **quantization flow** (lines *5:19*) and the **optimal bit precision** search (lines *20:32*).

The quantization flow defines the non-commutative order to quantize the modules. An optimal order exists and is not negligible. An optimal order is demonstrated for a network subdivided into $K = 3$ in section 4.2.1. Currently, order optimality for a particular network subdivided into K modules is obtained via an exhaustive grid search requiring extensive GPU training. This is not always feasible, thus the quantization flow search recommended in algorithm 2 first iteratively performs aggressive quantization (i.e. $Q^2(x)$ via INQ) independently for all k modules (lines *5:11*). If the an inference accuracy improvement is obtained via aggressive quantization of a particular module, that model is kept.

Next, the quantization flow is defined by arranging the remaining modules from least parameters to most parameters. The same logic applies to that of the INQ quantization schedule depicted in figure 4.3; by first quantizing modules with a smaller number of parameters, the remaining network will have more parameters and flexibility to handle aggressive quantization.

With the quantization flow in hand, the optimal bit precision search is performed next. If a specific accuracy threshold, τ, is defined the algorithm proceeds by iterating through the bit-width search, $B = [2,4,6...]$ by using INQ to quantize the $flow$ defined module, m_k. If the network inference accuracy does not meet the accuracy threshold, the bit-width is increased and the process repeats. As there are no specific hardware or accuracy requirements for this work, instead the compression accuracy trade-off is demonstrated for different bit precisions in the following experiments section.

[6]Note that EPFL CVLab have continued to iterate on the MQ algorithm. As of the time of writing, the most recent version can be found in [1], rebranded as Informed Modular Quantization (IMQ).

Algorithm 2 Module-wise Quantization (MQ)

Input: Training data
Input: A NN architecture model M, subdivided into modules $m \in [m_1, m_2, ...m_K]$
Input: The model Accuracy Metric $ac(F)$ and Accuracy Threshold τ
Input: Bit-width search $b \in B = [2,4,6...]$
for $k = 1,2,..K$ modules **do**
    $M_k^2 \leftarrow Q_k^2(F)$
    if $ac(M_k^2) > ac(M) \wedge ac(M_k^2) > ac(M_{best})$ **then**
        $M_{best}^2 \leftarrow M_k^2$
        $k_{best} \leftarrow k$
    end if
end for
$flow = [\,]$
if M_{best} **then**
    $M \leftarrow M_{best}$
    $flow \leftarrow index(SORT_SIZE([m_1, m_2, ...m_K])$
    $flow.pop(k_{best})$
else
    $flow \leftarrow index(SORT_SIZE([m_1, m_2, ...m_K]))$
end if
$i, j = 1$
while $i \leq len(flow) \wedge j \leq len(B)$ **do**
    $b = B_j$
    $M \leftarrow Q_{flow[i]}^b(M)$
    if $(ac(M) > \tau)$ **then**
        $i = i + 1$
        $M \leftarrow M^Q$
        $j = 1$
    else
        $j = j + 1$
    end if
end while
Output: Optimized bit-width quantized model (M^Q)

$flow$: Sequence of quantization order of modules.
k_{best}: Index of the module for which quantization increased performance.
K: Number of modules
M_k^b: Model with only module k quantized to b bits.
Q_k^b: Quantization applied to module m_k with b bits.

4.2 Quantization Experiments

Finally the results of MQ are explored experimentally. First an exhaustive quantized module search is performed by colleagues at EPFL CVLab to demonstrate the existence of a non-negligible quantization order optimality. Next MQ is directly compared to one of the very limited spacecraft quantization literature works. Finally, MQ is applied to two different 6D pose estimation methods on this work's rocket body dataset.

To compare consistently with previous works evaluating on the SwissCube spacecraft dataset, here the ADI metric is introduced as:

$$e_{ADI}(p^{est}, p^{gt}) = \frac{1}{V}\sum_{i=1}^{V} \min_{j \in V} \| p_i^{est} - p_j^{gt} \|_2 \tag{4.3}$$

where

p_i^{est} is the i^{th} 3D Euclidean bounding box keypoint for the estimated pose,
p_j^{gt} is the j^{th} 3D Euclidean bounding box keypoint for the ground truth pose, and
V is the total number of bounding box keypoints (or *vertices*).

In this way, CNN FE methods and straight-shot CNN methods can be compared. The 3D bounding box of a target object is projected into Euclidean space using the estimated pose, p^{est}. The normalized sum of the distances between the i^{th} estimated bound box keypoint and the nearest j^{th} ground truth bounding box keypoint is calculated[7]. Then the errors are grouped into error bins proportionally scaled to the target object's diameter. For example, ADI-0.1d would refer to the proportion of inference errors that exceeded 10% of the target's diameter. In this way, the ADI metric does not punish errors on objects with larger diameters.

4.2.1 EPFL CVLab MQ Experiments

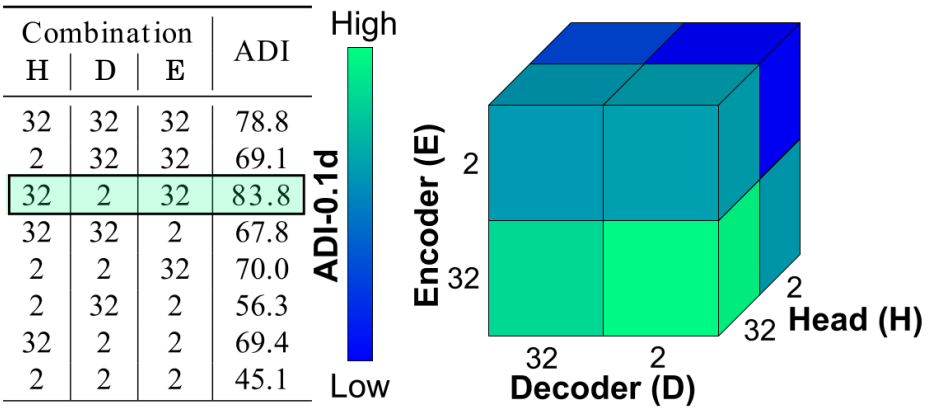
Two experiments performed by colleagues at EPFL CVLab are briefly shared for completeness to illustrate the advantages of MQ.

MQ Order Optimality

To confirm that a modular quantization order exists and is not negligible. Aggressive quantization (i.e. $Q^2(x)$) is performed on a CNN FE network subdivided into $K = 3$ modules. This is an $O(2^K)$ search and is completed using the SwissCube spacecraft dataset. The results are shared in figure 4.4. Notably, the highest accuracy is obtained only when the decoder is first

[7]The ADI metric was designed for image sets with symmetric objects and, consequently, indistinguishable poses. ADI is, in fact, a lower bound of another metric: Average Distance Distinguishable (ADD) [115]. Theoretically, ADD would be the suitable metric for the asymmetric SwissCube dataset. However, as previous works evaluated SwissCube using ADI, ADI is again adopted here.

quantized. Any other quantization case results in worse performance. Therefore, for this network subdivided into $K = 3$ modules, it is best to quantize the decoder first.



Combination H	D	E	ADI
32	32	32	78.8
2	32	32	69.1
32	2	32	83.8
32	32	2	67.8
2	2	32	70.0
2	32	2	56.3
32	2	2	69.4
2	2	2	45.1

Figure 4.4: EPFL CVLab Order Optimality Grid Search [1]

MQ Comparison to State-of-the-Art

Additionally MQ is directly compared to other state-of-the-art quantization methods: uniform method INQ [132] and mixed-precision method HAWQ [133]. Again, the experiment is performed using a CNN FE network subdivided into $K = 3$ modules and evaluating on the SwissCube spacecraft dataset. The results are shared in figure 4.5. MQ handily outperforms state-of-the-art quantization methods on the same NN architecture, dataset and at comparable compression factors.

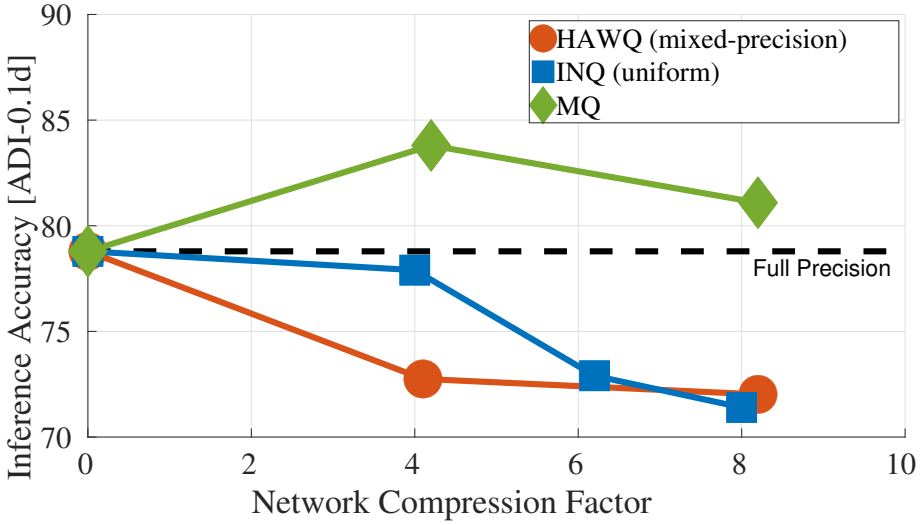


Figure 4.5: EPFL CVLab MQ Comparison to State-of-the-Art [1]

4.2.2 MQ Comparison to Spacecraft Quantization Literature

As discussed in section 1.2.3, quantization as a compression method is largely unexplored for spacecraft 6D pose estimation networks. However, CA-SpaceNet [29], were one of the first (and only) to develop a CNN FE framework for spacecraft pose estimation and to subsequently perform quantization on the framework. Notably the CA-SpaceNet architecture is again comparable to the generic encoder-decoder-head architecture depicted in figure 3.3. The most notable difference being that CA-SpaceNet uses two parallel decoders which are treated as a single module for the purposes of MQ.

In their paper, CA-SpaceNet defined three quantization "modes", but in this work's vernacular, these three "modes" would be in-completed quantization flows. Similarly, CA-SpaceNet identifies $K = 3$ modules but they do not explore individual module bit-widths or sequential module-wise quantization. Instead CA-SpaceNet perform 7 tests: (1) No quantization, (2) quantize only the encoder to 8 bits, (3) simultaneously quantize the encoder and decoder to 8 bits, (4) simultaneously quantize the entire network to 8 bits, and (5-7) repeat (2-4) but to 3 bits. Additionally, CA-SpaceNet never quantize the first or last layer of the CNN FE pipeline.

Here the CA-SpaceNet CNN FE pipeline is recreated and their results on the SwissCube spacecraft dataset are replicated. MQ is then applied to CA-SpaceNet. The quantization order of decoder-head-encoder is adopted and the decoder is quantized to 2 bits. Two different combinations of bit widths are also applied to the head and encoder. The CA-SpaceNet quantization results' accuracy and final compression factor are shared in table 4.1 to compare against the MQ results.

Table 4.1: MQ Applied to CA-SpaceNet vs CA-SpaceNet Published Quantization

Quantization Mode	ADI-0.1d	Compression
CA-SpaceNet Full Precision	79.4	1x
8 bit CA-SpaceNet E	76.2	2.2x
8 bit CA-SpaceNet ED	75.0	3.2x
8 bit CA-SpaceNet EDH	74.7	4.0x
MQ Applied to CA-SpaceNet	**82.7**	4.7x
3 bit CA-SpaceNet E	75.1	2.9x
3 bit CA-SpaceNet ED	74.5	5.9x
MQ Applied to CA-SpaceNet	**80.2**	8.2x
3 bit CA-SpaceNet EDH	68.7	10.6x

With MQ applied, the accuracy improved over the full precision CA-SpaceNet at both 4.7x compression and 8.2x compression. Notably, CA-SpaceNet did not demonstrate an accuracy improvement for any of the quantization tests. CA-SpaceNet did notice a significant

accuracy drop off between the 3 bit encoder-decoder case vs the 3 bit encoder-decoder-head case (a decrease of 5.8%). This is interesting as the head module had relatively fewer parameters compared to the encoder. Based on MQ theory and the section 4.2.1 experiments, CA-SpaceNet would likely benefit from not quantizing the encoder if one module is to avoid quantization. These results are also shared in the collaboration paper [1].

4.2.3 Quantization Applied to the Rocket Body Dataset

Introduced in section 2.3.2, the rocket body dataset features a JAXA H-IIA launch vehicle upper stage in a polar orbit around earth. The dataset includes sequential images of various motions (e.g. figure 2.22) as well as randomized poses. The two applied 6D pose estimation pipelines were introduced in section 3.3. Notably the CNN FE with post processing fitting utilizes a NN with 52 million parameters (this does not include the post-processing fitting resource utilization), while the straight-shot CNN has 11 million parameters. In this section the inference accuracy results are shared, but additionally MQ is applied.

A number of different quantization cases are evaluated. The straight-shot CNN and the CNN FE pose estimation network parameters are quantized to both 4 bit and 2 bit precisions using MQ. As the target is asymmetric, the ADD metric is used to evaluate the networks (identical to equation 4.2, but removing the min function). The ADD accuracy results are shared for two subsets of the rocket body dataset: (1) a pre-rendezvous scenario, and (2) a post-rendezvous scenario.

Pre-Rendezvous Scenario

The pre-rendezvous case concerns a target object of unknown distance and orientation. Presumably, the chaser satellite has confirmed that the target object exists and is within their camera FoV. For context, example images from the pre-rendezvous subset of the rocket body dataset are shown in figure 4.6. The full-precision and quantized neural network results are shown in table 4.2.

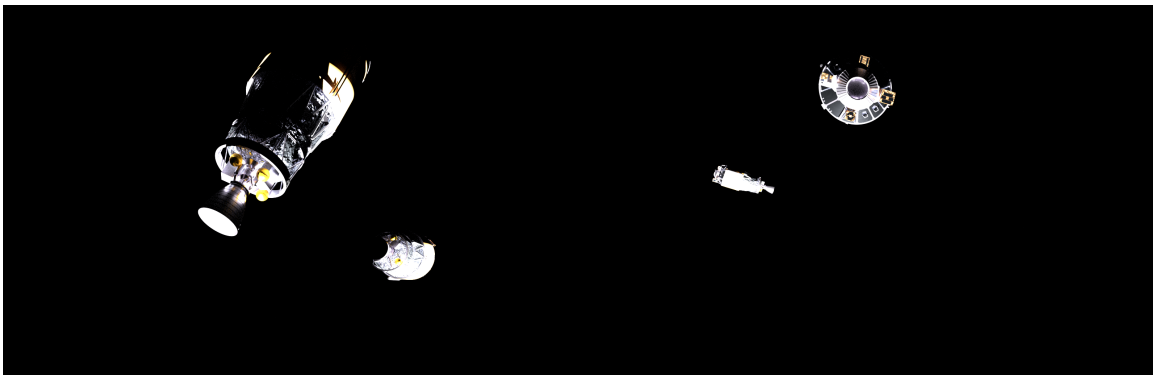
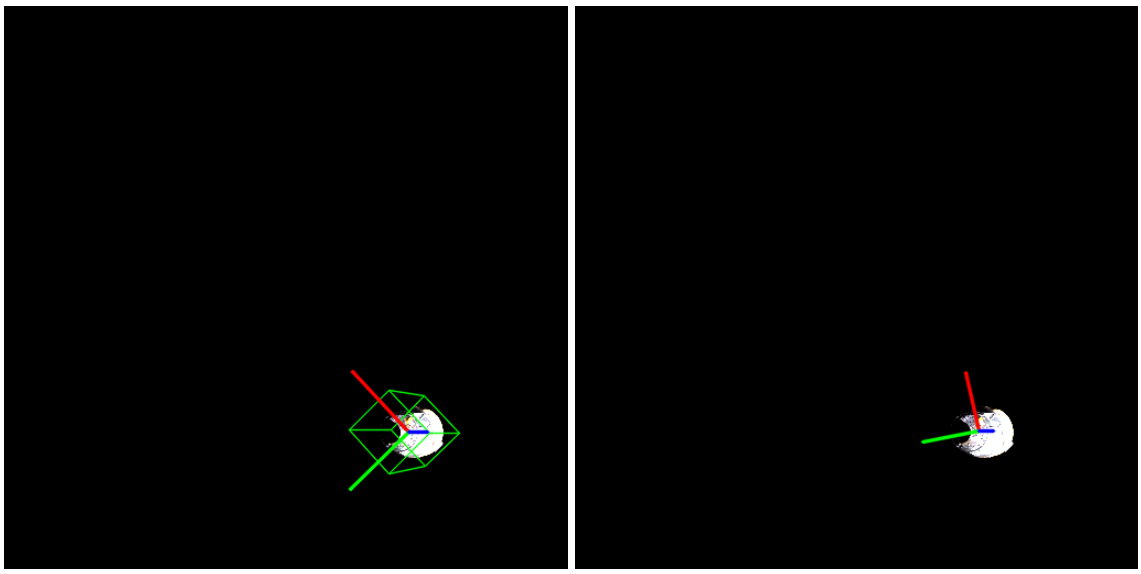


Figure 4.6: Pre-Rendezvous Pose Estimation Scenario

Table 4.2: Quantized Neural Networks Evaluated on the Pre-Rendezvous Scenario

	CNN FE		Straight-Shot CNN	
Quantization	0.05d	0.1d	0.05d	0.1d
Full precision	74.7	88.3	24.5	52.4
MQ 4 bit	76.5	91.2	19.9	46.9
MQ 2 bit	75.1	89.8	15.8	42.2

Evidently, the CNN FE network with post processing fitting exhibits higher accuracy for all cases. Additionally, the CNN FE maintains accuracy throughout quantization. The majority of the out-of-tolerance pose estimation cases included images of a distant target with a particularly difficult orientation. In these cases, it becomes especially challenging to estimate the target's roll angle. This roll error is especially observable in figure 4.7, with the ground truth and CNN FE network estimations both shown.



(a) Ground Truth **(b)** CNN Feature Extractor Pose Estimation

Figure 4.7: Example Rocket Body Random Pose Ground Truth and Estimation

In contrast, the straight-shot CNN exhibits relatively poor results that further declined in performance with quantization. This straight-shot CNN architecture would not be a suitable choice for pre-rendezvous pose estimation activities.

Post-Rendezvous Scenario

The post-rendezvous case concerns a target object with a fixed location and attitudes restricted to a steady-state tumble motion. Presumably, the chaser satellite has located the target object and is now maintaining a fixed holding position; any observed attitudes of the

target are thus a function of the target's own dynamics and not a coupling of the target and chaser's motion. This is a simpler pose estimation case, but is an equally valid scenario[8]. For context, example images from the post-rendezvous subset of the rocket body dataset are shown in figure 4.8. The full-precision and quantized neural network results are shown in table 4.3.

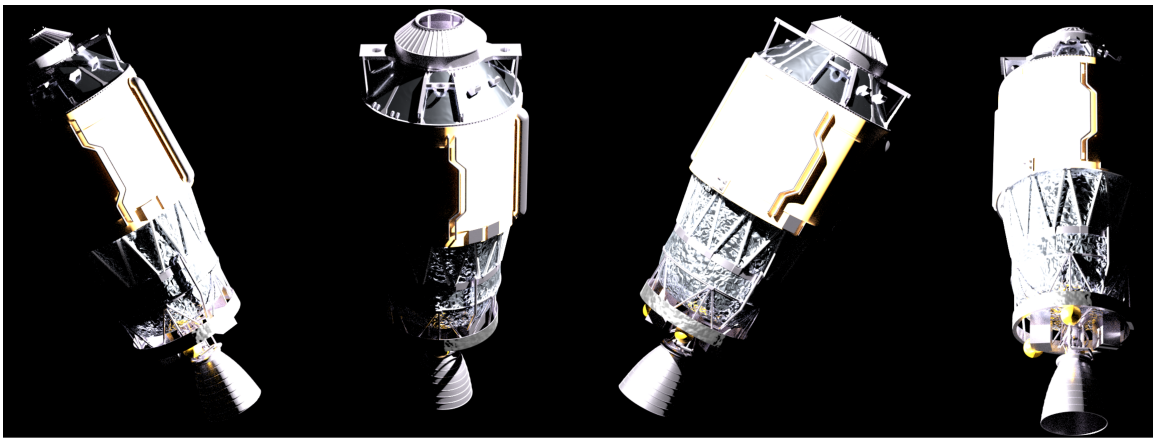


Figure 4.8: Post-Rendezvous Pose Estimation Scenario

Table 4.3: Quantized Neural Networks Evaluated on the Post-Rendezvous Scenario

	CNN FE		Straight-Shot CNN	
Quantization	0.05d	0.1d	0.05d	0.1d
Full precision	100	100	100	100
MQ 4 bit	100	100	100	100
MQ 2 bit	100	100	98.9	100

Notably, this post-rendezvous scenario contains a relatively trivial set of poses for these networks; the target is properly framed and at a convenient distance of 40 m centered on the camera optical axis. Even quantized, both networks achieved errors less than 0.05d[9]. By all counts, the CNN FE with post processing fitting architecture outperforms the straight-shot CNN network.

Motion Prediction

As the rocket body conic motion is sinusoidal, the CNN pose estimations can be fitted to Euler angle rotations about different axes (see figure 3.20):

$$\theta(t) = a * \sin(bt + c) + d \tag{4.4}$$

[8]This post-rendezvous scenario and the accompanying target motion were both provided by an industrial partner actively working on an in-space project.

[9]The rocket body is large with a bounding box diameter of $d = 13.39$ m, resulting in $ADD_{0.05d} = 0.67$ m.

where

a is the magnitude term,
b is the frequency term,
c, d are offset terms,
t is the time of a particular tumble render, and
$\theta(t)$ is the rotation about a given axis at time, t.

It goes without saying that the CNN FE network outperforms the straight-shot CNN pipeline, but again at a cost of 4X the network size. It is for this reason, that the lower bound performance of the smaller straight-shot CNN is further investigated here.

Euler angles for the first 200 network predictions from the Straight-shot CNN pipeline are shared in figure 4.9. The figure includes the ground truth values from the render annotations, the predictions from the 32 bit (full precision) network and the two quantized networks (4 bit and 2 bit). It can be seen that the 4 bit quantized network predictions closely follow the 32 bit full precision network predictions. The 2 bit quantized network additionally exhibits good results, but with noticeable higher errors at the limits of motion. For completeness, the prediction errors are also shared in figure 4.10. The prediction errors are largely uncorrelated, with the exception that the 2 bit quantized network prediction error maxima coincide with the limits of the conic motion.

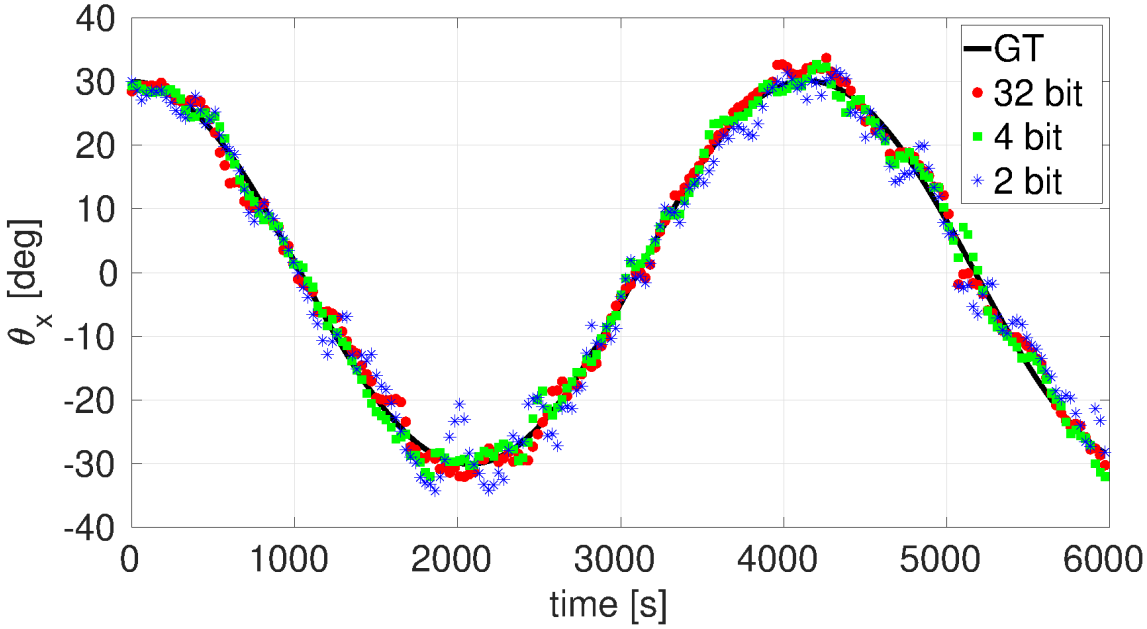
In addition to the raw predictions, the predictions fitted to equation 4.4, as well as their fit errors are shared in table 4.4 and table 4.5. Notably, the 4 bit quantized network exhibits negligible reduction in fit accuracy with similar RMSE fit errors to the full precision network for both θ_x and θ_z. The 2 bit quantized network exhibits a higher RMSE fit error but still estimates magnitude and frequency terms within 4% error of the ground truth.

It is worth acknowledging that the straight-shot CNN network adopted for this work has approximately 11 million parameters. At 32 bit precision, this equates to approximately 44 MB to hold the network in memory. Reducing the network from 32 bit to 4 bit (or 2 bit) results in a theoretical network size reduction down to 5.5 MB (or 2.75 MB)[10]. Depending on the accuracy requirements, the 2 bit network might be suitable. Furthermore, by every metric considered in this section, the 4 bit quantized network matched the 32 bit network performance.

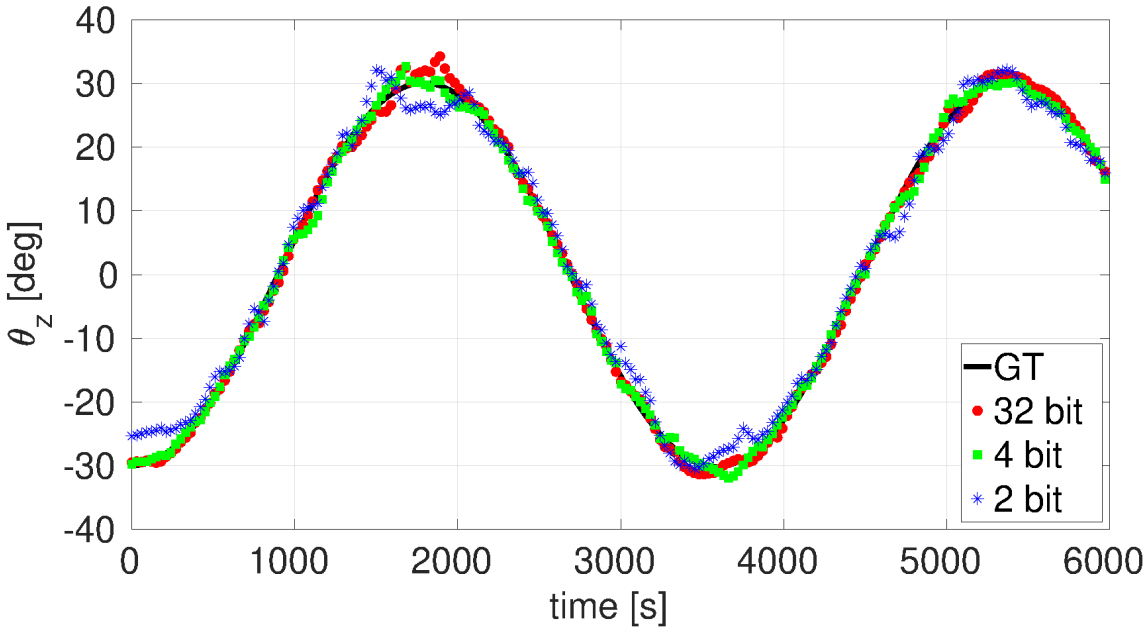
4.3 Summary

In this chapter, the NN compression method of quantization is introduced. Quantization theory is reviewed:

[10]To test latency, the network would need to be deployed onto specific hardware. Although the network is in fact quantized, the parameters are still stored in 32 bit precision on the GPUs used for training and testing. Thus the throughput or latency improvements of quantization cannot be quantified here.



(a) θ_x Network Predictions

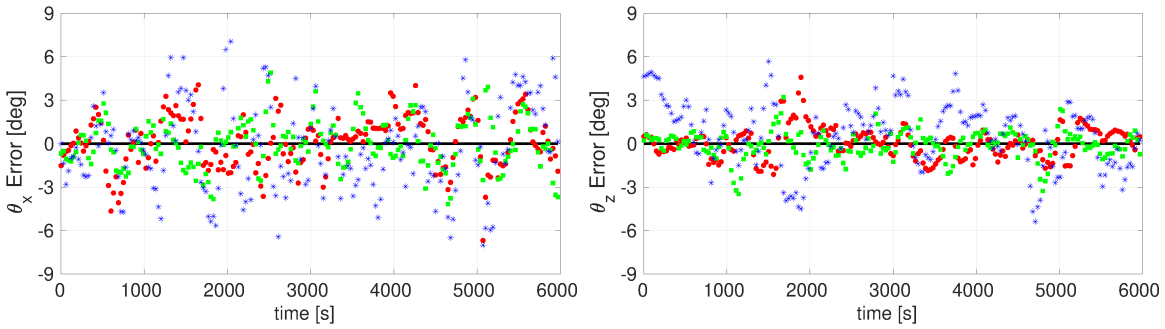


(b) θ_z Network Predictions

Figure 4.9: First 200 Straight-shot CNN Network Predictions

1. Quantization fundamentals are reviewed and a quantization mapping function equally appropriate for higher precisions (e.g. $Q^8(x)$) and lower precisions (e.g. $Q^2(x)$) is defined.

2. The modern and competitive INQ method is discussed, and it is incorporated into this chapter's experiments.



(a) θ_x Network Prediction Errors

(b) θ_z Network Prediction Errors

Figure 4.10: First 200 Straight-shot CNN Network Prediction Errors

Table 4.4: Quantized Network Prediction: θ_x Motion Fit

Fit Value	Ground Truth (GT)	32 bit	4 bit	2 bit
a [°]	30.6	31.0	30.3	29.5
$1/b$ [s]	571.4	571.1	571.1	572.1
R^2 ($\uparrow +$)	-	1.00	1.00	0.99
RMSE ($\downarrow +$)	-	0.8	0.9	1.8

Table 4.5: Quantized Network Prediction: θ_z Motion Fit

Fit Value	Ground Truth (GT)	32 bit	4 bit	2 bit
a [°]	30.5	30.6	30.3	30.2
$1/b$ [s]	660.6	659.2	660.1	659.6
R^2 ($\uparrow +$)	-	0.99	0.99	0.98
RMSE ($\downarrow +$)	-	1.8	1.8	3.0

3. In collaboration with EPFL CVLab, the novel quantization algorithm, MQ is introduced.

Next a number of experiments demonstrating the superiority of MQ to other quantization methods in literature are shared in section 4.2.1. An experiment demonstrating the superior performance of MQ against the limited quantization spacecraft literature is performed in section 4.2.2. The chapter finishes with a series of experiments on the section 2.3.2 rocket body dataset.

These final experiments culminate in the incorporation of everything studied in this dissertation: the rocket body dataset was developed in chapter 2; the CNN pose estimation

architectures were selected based on lessons learned in chapter 3; and the quantization experiments employ the theory and novel algorithm introduced in chapter 4. A straight-shot CNN spacecraft pose estimation network of size 44 MB is compressed to 2.75 MB, and a CNN FE spacecraft pose estimation network of size 206.2 MB is compressed to 12.9 MB. Both compressed networks are still readily able to fit to the target's motion.

If a hypothetical spacecraft's hardware is capable of running the 12.9 MB network, it will be able to perform pose estimations robustly for random orientations. If 12.9 MB (not including the iterative post processing fitting algorithm) is too large, motion estimation can still be reliably performed with a 2.75 MB network. Naturally, with specific hardware and for a specific application, networks can be tuned and the size further modified. The prevailing point is that pose estimation networks can be greatly compressed while maintaining performance.

Chapter 5
Conclusions

In-space rendezvous with non-cooperative targets is a key enabling technology for asteroid exploitation, in-orbit construction and orbital debris removal. The capability to remotely derive relative location, attitude and motion (i.e. *pose estimation*) is identified as a necessary and core competency to enable spacecraft rendezvous. A number of sensors and strategies exist, but this work focuses on the monocular camera system due to the widespread inclusion of single cameras in spacecraft systems.

5.1 Summary

The research in this dissertation explores the opportunities and limitations of CNN architecture solutions to non-cooperative spacecraft pose estimation.

Chapter 1

In chapter 1, four primary barriers restricting the incorporation of NN pose estimation architectures into spacecraft GNC systems were identified:

1. NN architectures require access to annotated space image datasets, but space datasets are largely limited and their usage is often restricted.
2. NN pose estimation architectures have been applied to typical spacecraft images, but the limit cases of high-noise, target symmetry and unseen targets are largely unexplored.
3. NN pose estimation architectures are trending towards increased complexity and size, a trend not supportable by spacecraft hardware.
4. NN architecture performance cannot be disputed, but the demonstration and theoretical proof of NN pose estimator stability must be addressed.

The progress and limitations of current state-of-the-art research for each of the four barriers was reviewed; objectives to address three of the barriers were delineated.

Chapter 2

The first barrier, spacecraft dataset limitations, was addressed in chapter 2. A complete spacecraft pose estimation render pipeline was theorized, developed, validated and open-sourced. The dataset render pipeline accounted for a number of space environment idiosyncrasies:

- Spacecraft material properties were defined, even when physically obtained material properties were nonexistent or access restricted.
- A tumble integrator was created from first principles, as spacecraft are not restricted to planar motion.
- Earth-orbit environmental lighting is dominated by direct sunlight and Earth albedo, thus both were modelled. Additionally, orbits were simulated to account for the dynamic changes in lighting.
- Sensor noise from the real Minerva-II2 deep space imagery was modelled and incorporated as post processing into the render pipeline. It was shown that the incorporation of sensor noise reduced the distance between real and rendered images.

Using the render pipeline, five datasets were created.

1. The **Cubesat** dataset was used in conjunction with real laboratory images to validate the dataset render pipeline.
2. The **Minerva-II2** (symmetric) dataset was used to train NN pose estimation architectures for testing on the real Minerva-II2 deployment images in chapter 3.
3. The **Modified Minerva-II2** (asymmetric) dataset was used to assess the performance of CNN pose estimation methods in the absence of object symmetry. Comments on the methods' sensitivity to object symmetry were made.
4. The **Rocket Body** 3-channel dataset was used to enable the chapter 4 network compression studies.
5. Finally, the **Multi-Object** dataset was shared in support of ongoing work: the testing of pose estimation architectures for *unseen* objects.

The full render pipeline scripts, example images and tutorial are shared from [61].

Chapter 3

The second barrier, addressing limit and challenging pose estimation cases, was addressed in chapter 3. Pose estimation theory was introduced and strategies were categorized into three groups: (1) **Conventional** feature extractor pose estimation, (2) **CNN feature extractor** pose estimation, and (3) **Straight-shot CNN** pose estimation methods. All three methods were applied to the noisy and symmetric Jaxa Hayabusa2 Minerva-II2 rover deployment images. The CNN FE speed estimation of $\mathbf{0.1934}\ \frac{m}{s}$ most closely matched the independent telescopic camera JAXA speed estimation of $\mathbf{0.1924}\ \frac{m}{s}$. Furthermore, the methods' sensitivity to object symmetry was discussed with the observation that the straight-shot CNN method was the

most easily adaptable to symmetric targets. Additionally, CNN FE and straight-shot CNN pose estimation methods were also applied to the chapter 2 rocket body dataset in support of the chapter 4 compression studies.

Chapter 4

The third barrier, addressing NN size and incompatibility with spacecraft hardware, was addressed in chapter 4. Quantization theory was reviewed and, in collaboration with EPFL CVLab, a novel quantization algorithm was introduced. The MQ algorithm was shown to outperform other state-of-the-art quantization algorithms, both uniform and mixed-precision, both in general and in space-specific literature.

Finally, this dissertation's work culminated in a series of experiments on the chapter 2 rocket body dataset, using the chapter 3 CNN pose estimation architectures and the chapter 4 NN quantization techniques. A CNN architecture of size 44 MB was compressed down to 2.75 MB and was still able to track the target's motion with less than 4% magnitude error compared to the ground truth annotations.

5.2 Statement of Contributions

The primary contribution of this research has been to address barriers inhibiting the entry of NN pose estimation architectures into spacecraft systems. The barriers encompass a diverse range of topics bridging from orbital dynamics into computer vision research. This work includes the following *novel* contributions:

1. The modelling and inclusion of physics based sensor noise into an image render pipeline, improving render fidelity and reducing the domain gap between the rendered images' domain and a targeted real images' domain.

2. The first NN pose estimation analysis of *deep* space images. This work overcame high noise imagery, limited access to real images (approximately 20 of utility) and a highly symmetric target; all of which was, as of yet, largely unaddressed in space-related literature.

3. In collaboration with CVLab [1], the validation of a novel compression algorithm and the assessment of different spacecraft pose estimation architectures' sensitivity to compression.

In addition to the above novel contributions, the large task of developing and open-sourcing a spacecraft image render pipeline should be acknowledged. While several spacecraft image datasets exist, methodology has only been discussed, rarely shared. This has resulted in almost every dataset render pipeline being developed from scratch with little opportunity

for incorporation of previous works or new 3D spacecraft models. This work also shares the coded solutions to a number of engineering challenges associated with image rendering including appropriately defining uniformly spaced quaternions, estimating the sun vector from deep-space ephemeris and implementing novel noise sources.

5.3 Ongoing Work

Currently two-related projects are actively ongoing.

Unseen Spacecraft Pose Estimation. All of the methods from this work and from space-related pose estimation literature are framed around *seen* instance pose estimation. In practice, if orbital debris removal is to be made economically feasible, a chaser spacecraft will need to be able to rendezvous with multiple different non-cooperative targets. If the target is a fragment of a larger satellite, or has been left in space for so long that its material properties have been significantly altered, there will be no accurate physical model of the target available; it will be *unseen*. In support of this, Tohoku SRL, EPFL CVLab and the ESA Advanced Concept Team (ACT) are collaborating in publicly supporting the development of *unseen* spacecraft pose estimation methods. The spacecraft render pipeline developed from this dissertation's work was selected to enable the ESA-EPFL-Tohoku collaboration project.

Neural Network Sensitivity to Compression. A promising quantization method and the compression sensitivity of two spacecraft pose estimation architectures was explored in chapter 4, however the development of earth-based neural network pose estimation architectures is rapid; novel and promising architectures can become obsolete in less than a year. For this reason, a survey of the sensitivity of additional earth-based neural network pose estimation architectures is necessary to develop additional recommendations for the space community.

5.4 Future Scope

Due to long exposure to solar radiation, it is likely that the material properties of a particular resident space object will change over time. Therefore, it may become difficult to train a pose estimation architecture on one set of material properties, to then test on a different set of material properties in space. It was observed in chapter 2 that it is possible to obtain ideal illumination conditions for the calibration of a $\widehat{BRDF}$ after launch (equation 2.5). This prompts follow up ideas.

In-orbit $\widehat{BRDF}$ Calibration. If a chaser satellite is equipped with a spectrally calibrated camera (i.e. the response sensitivity as a function of electromagnetic frequency is known), it would be possible to take several images of a resident space object to downlink to Earth. The combination of known framerate, known sunlight intensity and known cam-

era sensitivity would enable the render process to be backpropagated to infer the spectral and spatial $\widehat{BRDF}$ for particular material properties. With the $\widehat{BRDF}$ available, new higher fidelity images could then be rendered to train a parallel NN and the weights could be uploaded to the chaser satellite to perform additional pose estimation or rendezvous tasks.

Light-Curve Analysis. Similar to astronomical minor body analysis techniques, the temporal light-curves of an observed rotating body can be used to uniquely identify said body [134]. Initial works indicate that a similar taxonomic classification of resident space objects via earth-based observatories may also be possible [135]. This would enable greater coordination between ground-based and orbit-based resident space object tracking.

NN Stability. Additionally, as identified in section 1.1, there remains a fourth barrier restricting NN adoption into spacecraft GNC systems; that of NN stability. It follows that a pose estimation architecture should be selected, compressed and deployed onto spacecraft hardware (i.e. an FPGA). The stability of the pose estimation architecture must be analytically and empirically investigated with a varied range of input images.

The author hopes the above incorporated work succeeded in its mission to overcome barriers limiting the incorporation of machine vision advances to spacecraft GNC; as well as to provide novel engineering solutions to a few of the many challenges associated with spacecraft pose estimation.

Bibliography

[1] Saqib Javed, Andrew Price, Yinlin Hu, and Mathieu Salzmann. Module-wise network quantization for 6d object pose estimation. In *arXiv Preprint: AAAI 2024*, February 2024.

[2] P. F. Proença and Y. Gao. Deep learning for spacecraft pose estimation for photorealistic rendering. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6007–6013, 2020.

[3] Yusuke Oki, Kent Yoshikawa, Hiroshi Takeuchi, Shota Kikuchi, Hitosi Ikeda, Daniel J. Scheeres, Jay W. McMahon, Junichiro Kawaguchi, Yuto Takei, Yuya Mimasu, Naoko Ogawa, Go Ono, Fuyuto Terui, Manabu Yamada, Toru Kouyama, Shingo Kameda, Kazuya Yoshida, Kenji Nagaoka, Tesuo Yoshimitsu, Takanao Saiki, and Tuichi Tsuda. Orbit insertion strategy of hayabusa2's rover with large release uncertainty around the asteroid ryugu. *Astrodynamics*, 4:309–329, November 2020.

[4] James M. Gimwood, Barton C. Hacker, and Peter J. Vorzimmer. Project gemini technology and operations a chronology. https://history.nasa.gov/SP-4002/contents.htm.

[5] Douglas Zimpfer, Peter Kachmar, and Seamus Tuohy. Autonomous rendezvous, capture and in-space assembly: Past, present and future. *AIAA 1st Space Exploration Conference: Continuing the Voyage of Discovery*, February 2005.

[6] Matt Pyrak and Joseph Anderson. Performance of northrop grumman's mission extension vehicle (mev) rpo imagers at geo. In *SPIE 12115 Autonomous Systems: Sensors, Processing and Security for Ground, Air, Sea and Space Vehicles and Infrastructure 2022*, June 2022.

[7] JAXA. About engineering est satellite "kiku-vii" (ets-vii). https://global.jaxa.jp/article/2017/special/debris/nishida.html, 1997.

[8] Jason L. Forshaw, Guglielmo S. Aglietti, Nimal Navarathinam, Haval Kadhem, Thierry Salmon, Aurélien Pisseloup, Eric Joffre, Thomas Chabot, Ingo Retat, Robert Axthelm, Simon Barraclough, Andrew Ratcliffe, Cesar Bernal, François Chaumette, Alexandre Pollini, and Willem H. Steyn. Removedebris: An in-orbit active debris removal demonstration mission. *Acta Astronautica*, 127:448–463, November 2016.

[9] Brook Sullivan, David Barnhart, Lisa Hill, Paul Oppenheimer, Bryan L. Benedict, Gerrit Van Ommering, Laurie Chappell, John Ratti, and Peter Will. Darpa phoenix payload orbital delivery system (pods): "fedex to geo". In *AIAA 2013*, September 2013.

[10] Mark. C. Priyant and Kamath Surekha. Review of active space debris removal systems. *Journal of Space Policy*, 47:194–206, February 2019.

[11] Robin Biesbroek, Sarmad Aziz, Andrew Wolahan, Stefano Cipolla, Muriel Richard-noca, and Luc Piguet. The clearspace-1 mission: Esa and clearspace team up to remove debris. In *8th European Conference on Space Debris*, volume 8(1), 2021.

[12] JAXA. Commercial removal of debris demonstration (crd2). https://www.kenkai.jaxa.jp/eng/research/crd2/crd2.html, 2022.

[13] Jason Rainbow. Starfish space raises $14 million for in-orbit servicers. https://spacenews.com/starfish-space- raises-14-million-for-in-orbit-servicers/, 2023.

[14] Alessio A. Grompone. *Vision-Based 3D Motion Estimation for On-Orbit Proximity Satellite Tracking and Navigation*. PhD thesis, California Naval Postgraduate School, June 2015.

[15] Bo Naasz, John Van Eepoel, Steve Queen, Michael Southward, and Joel Hannah. Flight results of the hst sm4 relative navigation sensor system. In *33rd Annual AAS Guidance and Control Conference*, February 2010.

[16] Son-Goo Kim, John Crassidis, Yang Cheng, Adam M. Fosbury, and John Junkins. Kalman filtering for relative spacecraft attitude and position estimation. *Journal of Guidance, Control, and Dynamics*, 30(1), February 2005.

[17] Simone D'Amico, Mathias Benn, and John L. Jørgensen. Pose estimation of an uncooperative spacecraft from actual space imagery. *International Journal of Space Science and Engineering*, 2(2):171–189, February 2014.

[18] Yongzhi Su, Mahdi Saleh, Torben Fetzer, Jason Rambach, Nassir Navab, Benjamin Busam, Didier Stricker, and Deferico Tombari. Zebrapose: Coarse to fine surface encoding for 6dof object pose estimation. *IEEE/CVF Computer Vision and Pattern Recognition*, 2022.

[19] Yan Di, Fabian Manhardt, Gu Wang, Xiangyang Ji, Nassir Navab, and Federico Tombari. So-pose: Exploiting self-occulsion for direct 6d pose estimation. *IEEE/CVF International Conference on Computer Vision*, 2021.

[20] Gu Wang, Favian Manhardt, Federico Tombari, and Xiangyang Ji. Gdr-net: Geometry-guided direct regression networks for monocular 6d pose estimation. *IEEE/CVF Computer Vision and Pattern Recognition*, 2021.

[21] Yu Xiang, Tanner Schmidt, Venkatraman Narayanan, and Dieter Fox. Posecnn: A convolutional neural network for 6d object pose estimation in cluttered scenes. *Robotics: Science and Systems (RSS)*, 2018.

[22] Stefan Thalhammer, Markus Leitner, Timothy Patten, and Markus Vincze. Pyrapose: Feature pyramids for fast and accurate object pose estimation under domain shift. *IEEE International Conference on Robotics and Automation*, 2021.

[23] Yinlin Hu, Joachim Hugonot, Pascal Fua, and Mathieu Salzmann. Segmentation-driven 6d object pose estimation. *IEEE/CVF Computer Vision and Pattern Recognition*, 2019.

[24] W. Kehl, F. Manhardt, F. Tombari, S. Ilic, and N. Navab. Ssd-6d: Making rgb-based 3d detection and 6d pose estimation great again. *IEEE/CVF International Conference on Computer Vision*, 2017.

[25] Bugra Tekin, Sudipta N. Sinha, and Pascal Fua. Real-time seamless single shot 6d object pose prediction. *IEEE/CVF Computer Vision and Pattern Recognition*, 2018.

[26] Sumant Sharma. *Pose Estimation of Uncooperative Spacecraft Using Monocular Vision and Deep Learning*. PhD thesis, Stanford University, 2019.

[27] Bo Chen, Jiewei Cao, Albaro Parra, and Tat-Jun Chin. Satellite pose estimation with deep landmark regression and nonlinear pose refinement. *ICCVW 2019*, August 2019.

[28] Yinlin Hu, Sebastien Speierer, Wenzel Jakob, Pascal Fua, and Mathieu Salzmann. Wide-depth-range 6d object pose estimation in space. In *Computer Vision and Pattern Recognition Conference 2021*, 2021.

[29] Shunli Wang, Shuaibing Wang, Bo Jiao, Dingkang Yang, Liuzhen Su, Peng Zhai, Chixiao Chen, and Lihua Zhang. Ca-spacenet: Counterfactual analysis for 6d pose estimation in space. *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2022.

[30] Julien Posso, Guy Bois, and Yvon Savaria. Mobile-ursonet: an embeddable neural network for onboard spacecraft pose estimation. *IEEE International Symposium on Circuits and Systems*, 2022.

[31] Jianing Song, Duarte Rondao, and Nabil Aouf. Deep learning-based spacecraft relative navigation methods: A survey. *Acta Astronautica*, 2022.

[32] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009.

[33] Stefan Hinterstoisser, Vincent Lepetit, Slobodan Ilic, Stefan Holzer, Gary Bradski, Kurt Konolige, and Nassir Navab. Model based training, detection and pose estimation of texture-less 3d objects in heavily cluttered scenes. In *ACCV 2012*, 2012.

[34] Eric Brachman, Alexander Krull, Frank Michel, Stefan Gumhold, Jamie Shotton, and Carsten Rother. Learning 6d object pose estimation using 3d object coordinates. In *ECCV 2014*, 2014.

[35] Berk Calli, Arjun Singh, Aaron Walsman, Siddhartha Srinivasa, Pieter Abbeel, and Aaron M. Dollar. The ycb object and model set: Towards common benchmarks for manipulation research. In *IEEE International Conference on Advanced Robotics (ICAR) 2015*, 2015.

[36] Adel Ahmadyan, Liangkai Zhang, Artsiom Ablavatski, Jianing Wei, and Matthias Grundmann. Objectron: A large scale dataset of object-centric videos in the wild with pose annotations. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2021.

[37] Jeremy Reizenstein, Roman Shapovalov, Philipp Henzler, Luca Sbordone, Patrick Labatut, and David Novotny. Common objects in 3d: Large-scale learning and evaluation of real-life 3d category reconstruction, 2021.

[38] Yang Fu and Xiaolong Wang. Category-level 6d object pose estimation in the wild: A semi-supervised learning approach and a new dataset, 2022.

[39] European Space Agency Advanced Concept Team and Stanford University Space Rendezvous Laboratory. Spacecraft pose estimation dataset. https://kelvins.esa.int/satellite-pose-estimation-challenge/, 07 2019.

[40] European Space Agency Advanced Concept Team and Stanford University Space Rendezvous Laboratory. Bridging the domain gap with speed+. https://kelvins.esa.int/pose-estimation-2021/, March 2021.

[41] Mohamed Adel Musallam, Vincent Gaudilliere, Enjie Ghorbel, Kassem Al Ismaeil, Marcos Damian Perez, Michel Poucet, and Djamila Aouada. Spacecraft recognition leveraging knowledge of space environment: Simulator, dataset, competition design and analysis. In *2021 IEEE International Conference on Image Processing Challenges (ICIPC)*, September 2021.

[42] William S. Armstrong, Spencer Drakontaidis, and Nicholas Lui. Synthetic data for semantic image segmentation of imagery of unmanned spacecraft. In *2023 IEEE Aerospace Conference*, March 2023.

[43] Andrew Price and Kazuya Yoshida. A monocular pose estimation case study: The hayabusa2 minerva-ii2 deployment. In *Computer Vision and Pattern Recognition Conference Space Robotics Workshop 2021*, 2021.

[44] Hao-Shu Fang, Chenxi Wang, Minghao Gou, and Cewu Lu. Graspnet-1billion: A large-scale benchmark for general object grasping. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition(CVPR)*, pages 11444–11453, 2020.

[45] Angel X. Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, Jianxiong Xiao, Li Yi, and Fisher Yu. Shapenet: An information-rich 3d model repository, 2015.

[46] Matt Deitke, Dustin Schwenk, Jordi Salvador, Luca Weihs, Oscar Michel, Eli VanderBilt, Ludwig Schmidt, Kiana Ehsani, Aniruddha Kembhavi, and Ali Farhadi. Objaverse: A universe of annotated 3d objects. In *arXiv Preprint*, 2022.

[47] Laura Downs, Anthony Francis, Nate Koenig, Brandon Kinman, Ryan Hickman, Krista Reymann, Thomas B. McHugh, and Vincent Vanhoucke. Google scanned objects: A high-quality dataset of 3d scanned household items, 2022.

[48] Minghao Gou, Haolin Pan, Hao-Shu Fang, Ziyuan Liu, Cewu Lu, and Ping Tan. Unseen object 6d pose estimation: A benchmark and baselines, 2022.

[49] Lorenzo Pasqualetoo Cassinis, Robert Fonod, Eberhard Gill, Ingo Ahrns, and Jesus Gil Fernandez. Cnn-based pose estimation system for close-proximity operations around uncooperative spacecraft. In *AIAA Scitech 2020*, January 2020.

[50] Dung Anh Hoang, Bo Chen, and Tat-Jun Chin. A spacecraft dataset for detection, segmentation and parts recognition. In *CVPR2021 Workshop, AI4Space*, June 2021.

[51] European Space Astronomy Centre (ESAC) and Ideorum. Science satellite fleet. https://scifleet.esa.int/, July 2023.

[52] Chen Song, Jiaru Song, and Qixing Huang. Hybridpose: 6d object pose estimation under hybrid representations. *IEEE/CVF Computer Vision and Pattern Recognition*, 2020.

[53] James O' Neill. An overview of neural network compression. *Department of Computer Science University of Liverpool Whitepaper*, August 2020.

[54] Ritchie Zhao, Yuwei Hu, Jordan Dotzel, Christopher De Sa, and Zhiru Zhang. Improving neural network quantization without retraining using outlier channel splitting. In *ICML 2019*, January 2019.

[55] Zhaowei Cai and Nuno Vasconcelos. Rethinking differentiable search for mixed-precision neural networks. In *Computer Vision and Pattern Recognition 2020*, April 2020.

[56] Tannen S. VanZwieten, Michael R. Hannan, and John H. Wall. Evaluating the stability of nasa's space launch system with adaptive augmenting control. In *10th International ESA Conferrence on Guidance, Navigation and Control Systems*, June 2017.

[57] Fred F. Afagh. Mech 5505: Theory of stability course notes, Winter 2013.

[58] Alessandro Abate, Daniele Ahmed, Mirco Giacobe, and Andrea Peruffo. Formal synthesis of lyaponuv neural networks. In *IEEE Control Systems Letters*, volume 5(3), pages 773–778, June 2020.

[59] Honkai Dai, Benoit Landry, Lujie Yang, Marco Pavone, and Russ Redrake. Lyaponuv-stable neural-network control. In *Robotics: Science and Systems 2021*, July 2021.

[60] Mazaher Kianpour and Shao-Fang Wen. Timing attacks on machine learning: State of the art. In *SAI Intelligent Systems Conference*, pages 111–125, 2019.

[61] Andrew Price, Chen Zhao, Saqib Javed, Marc Pitteloud, Nicolas Szwajcok, Tim Vaughan-Whitehead, and Mathieu Salzmann. Epfl cvlab spacecraft dataset. https://drive.google.com/drive/u/1/folders/1iQC9RAGFW1HdZfYfzo2MnBwQWyMcXnZz, June 2023.

[62] Connor Beierle and Simone D'Amico. Variable-magnification optical stimulator for training and validation of spaceborne vision-based navigation. *Journal of Spacecraft and Rockets*, 56(11):1–13, February 2019.

[63] S. Kameda, H. Suzuki, T. Takamatsu, Y. Cho, T. Yasuda, M. Yamada, H. Sawada, R. Honda, T. Morota, C. Honda, M. Sato, Y. Okumura, K. Shibasaki, S. Ikezawa, and S. Sugita. Preflight calibration test results for optical navigation camera telescope (onc-t) onboard the hayabusa2 spacecraft. *Space Science Reviews*, 208:17–31, May 2016.

[64] H. Suzuki, M. Yamada, T. Kouyama, E. Tatsumi, S. Kameda, R. Honda, H. Sawada, N. Ogawa, T. Morata, C. Honda, N. Sakatani, M. Hayakawa, Y. Yokota, Y. Yamamoto,

and S. Sugita. Initial inflight calibration for hayabusa2 optical navigation camera (onc) for science observations of asteroid ryugu. *Icarus*, 300:341–359, January 2018.

[65] Srishty Dwivedi, Ram Narayan, and Lalita Gupta. *Digital Image Enhancement and Reconstruction*. Academic Press, 2023.

[66] Trupti Mahendrakar, Ryan White, Markus Wilde, and Madhur Tiwari. Spaceyolo: A human-inspired model for real-time, on-board spacecraft feature detection. In *2023 IEEE Aerospace Conference*, 03 2023.

[67] Andrew Price, Kentaro Uno, Swapnil Parekh, Til Reichelt, and Kazuya Yoshida. Render-to-real image dataset and cnn pose estimation for down-link restricted spacecraft missions. In *2023 IEEE Aerospace Conference*, March 2023.

[68] Jonathan Dupuy and Wenzel Jakob. An adaptive parameterization for efficient material acquisition and rendering. *Transactions on Graphics (Proceedings of SIGGRAPH Asia)*, 37(6):274:1–274:18, 11 2018.

[69] GoEngineer. Solidworks visualize. https://www.goengineer.com/solidworks/technical-communication/solidworks-visualize, 2023.

[70] OpenGL Team. Rendering pipeline overview. https://www.khronos.org/opengl, November 2022.

[71] Benjamin S. Heasly, Nicolas P. Cottaris, Daniel P. Lichtman, Bei Xiao, and David H. Brainard. Journal of vision. *Journal of Applied Optics*, 14(2):1–22, February 2014.

[72] Blender Foundation and the Cycles Team. Cycles: Open source production rendering. https://www.cycles-renderer.org/development/, 2022.

[73] TurboSquid Team. Professional spacecraft 3d models. https://www.turbosquid.com/3d-model/spacecraft, 2023.

[74] Jesse Richter-Klug and Udo Frese. Handling object symmetries in cnn-based pose estimation. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, May 2021.

[75] F. O. Bartell, E. L. Dereniak, and W. L. Wolfe. The theory and measurement of bidirectional reflectance distribution function (brdf) and bidirectional transmittance distribution function (btdf). In *PIE 0257, Radiation Scattering in Optical Systems*, March 1981.

[76] Fred E. Nicodemus. Directional reflectance and emissivity of an opaque surface. *Journal of Applied Optics*, 4(7):767–775, 1965.

[77] Stephen R. Marschner, Stephen H. Westin, Eric P. F. Lafortune, Kenneth E. Torrance, and Donald P. Greenberg. Image-based brdf measurement including human skin. In *Eurographics Worksop on Rendering Techniques*, pages 131–144, 1999.

[78] Felix R. Hoots, Ronald L. Roehrich, and TS Kelso. Spacetrack report no. 3: Nodels for propagation of norad element sets. *NORAD Report*, December 1988.

[79] Andrew Price. Orbit and tumble integrator. https://github.com/spaceguy-price/orbit_tumble_integrator, July 2023.

[80] Phil Davis. How big is the solar system? https://solarsystem.nasa.gov/news/1164/how-big-is-the-solar-system/, February 2020.

[81] Greg Kopp and Judith L. Lean. Spacetrack report no. 3: Nodels for propagation of norad element sets. *Geophysical Research Letters*, 38(1), Jan 2011.

[82] John Maurer. Retrieval of surface albedo from space. *University of Hawaii Remote Sensing Field Methods Course Paper*, December 2002.

[83] P. R. Goode, J. Qiu, V. Yurchyshyn, J. Hickey, M. C. Chu, E. Kolbe, C. T. Brown, and S. E. Koonin. Earthshine observations of the earth's reflectance. *Geophysical Research Letters*, 28(9), May 2001.

[84] Shen-En Qian. Hyperspectral satellites, evolution and development history. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 14:7032–7056, June 2021.

[85] Lachezar Hristov Filchev. Satellite hyperspectral earth observation missions – a review. https://www.researchgate.net/publication/290899564_Satellite_Hyperspectral_Earth_Observation_Missions_-_A_Review, January 2014.

[86] Julie Transon, Rapha el D'Andrimont, Alexandre Maugnard, and Pierre Defourny. Survey of hyperspectral earth observation applications from space in the sentinel-2 context. *MDPI Special Issue Spatial Enhancement of Hyperspectral Data and Applications*, 10(2):157, January 2018.

[87] Steve Hobbs, Leonard Felicetti, Cameron Leslie, Samuel Rowling, George Brydon, Mekhi Dhesi, Toby Harris, Lounis Chermak, and Umair Soori. Towards in-orbit hyperspectral imaging of space debris. In *2nd Neo and Debris Detection Conference*, January 2023.

[88] Howard Curtis. Orbital mechanics for engineering students, 2005.

[89] Yazhong Luo, Jin Zhang, and Guojin Tang. Survey of orbital dynamics and control of space rendezvous. *Chinese Journal of Aeronautics*, 27(1):1–11, February 2014.

[90] Ansys Inc. Systems tool kit. https://www.ansys.com/products/ missions/ansys-stk, 2023.

[91] MathWorks Inc. Aerospace toolkit. https://ch.mathworks.com/ help/aeroblks/spacecraft.html, 2023.

[92] Nasa operational simulator for small satellites. https://github.com/nasa/nos3, 2023.

[93] J. Peraire and S. Widnall. Lecture 26 - 3d rigid body dynamics: The inertia tensor. https://ocw.mit.edu/courses/ 16-07-dynamics-fall-2009/ dd277ec654440f4c2b5b07d6c286c3fd _MIT16_07F09_Lec26.pdf, Fall 2008.

[94] Paolo Cignoni andMarco Callieri, Massimiliano Corsini, Matteo Dellepiane, Fabio Ganovelli, and Guido Ranzuglia. Meshlab: an open-source mesh processing tool. In *Sixth Eurographics Italian Chapter Conference*, pages 129–136, 2008.

[95] Massimiliano Corsini, Paolo Cignoni, and Roberto Scopigno. Efficient and flexible sampling with blue noise properties of triangular meshes. *IEEE Transaction on Visualization and Computer Graphics*, 18(6)(6):914–924, 2012.

[96] Joseph John Thomson. On the structure of the atom: an investigation of the stability and periods of oscillation of a number of corpuscles arranged at equal intervals around the circumference of a circle; with application of the results to the theory of atomic structure. *Philosophical Magazine*, 6(7):237–265, March 1904.

[97] MathWorks Inc. randrot uniformly distributed random rotations. https://ch.mathworks.com/help/nav/ref/randrot.html, 2023.

[98] Autar Kaw. Gauss-seidel method. https://math.libretexts.org/Bookshelves/Linear_Algebra/ Introduction_to_Matrix_Algebra_(Kaw)/ 01

[99] Anton Semechko. S2-sampling toolbox. https://github.com/AntonSemechko/S2-Sampling-Toolbox.

[100] Open Source Blender Project. Official mirror of blender. https://github.com/blender/blender/.

[101] Richard Hartley and Andrew Zisserman. *Multiple view geometry in computer vision*. Cambridge university press, January 2011.

[102] NVIDIA Inc. Vision programming interface: Lens distortion correction. https://docs.nvidia.com/vpi/algo_ldc.html.

[103] Connor Shorten and Taghi M. Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(60), July 2019.

[104] Luis Perez and Jason Wang. The effectiveness of data augmentation in image classification using deep learning. *arXiv*, December 2017.

[105] Robert Levy and Paul Przyborksi. Nasa visible earth. https://visibleearth.nasa.gov/, 2023.

[106] Takashi Watanabe, Tomoaki Takeuchi, Osamu Ozawa, Hirohisa Komanome, Tomoyuki Akahori, and Kunihiko Tsuchiya. A radiation hardened cmos image sensor with almost zero dark current increase during radiation. https://nasa3d.arc.nasa.gov/, 2019.

[107] Gilbert F. Amelio. Charge-coupled devices. *Scientific American*, 230(2), February 1974.

[108] MathWorks Inc. imnoise add noise to an image. https://ch.mathworks.com/help/images/ref/imnoise.html, 2023.

[109] Intel Realsense. Intel realsense depth camera d435i. https://www.intelrealsense.com/depth-camera-d435i/, 2023.

[110] Sifeng Bi, Matteo Broggi, and Michael Beer. The role of the bhattacharyya distance in stochastic model updating. *Journal of Mechanical Systems and Signal Processing*, 117:437–452, February 2019.

[111] Ryan Park and Alan B. Chamberlin. Jet propulsion laboratory horizons system. https://ssd.jpl.nasa.gov/horizons/, 2023.

[112] John F. Canny. A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 8(6):679–698, 1986.

[113] Dana H. Ballard. Generalizing the hough transform to detect arbitrary shapes. *Pattern Recognition*, 13(2):111–122, 1981.

[114] David G. Lowe. Three dimensional object recognition from single two dimensional images. *Artificial Intelligence*, 31(3):355–395, 1985.

[115] Tomáš Hodaň, Jiří Matas, and Štěpán Obdržálek. On evaluation of 6d object pose estimation. *European Conference on Computer Vision*, 2016.

[116] Yongzhi Su, Mahdi Saleh, Torben Fetzer, Jason Rambach, Nassir Navab, Benjamin Busam, Didier Stricker, and Federico Tombari. Zebrapose: Coarse to fine surface encoding for 6dof object pose estimation. *arXiv preprint arXiv:2203.09418*, 2022.

[117] Vincent Lepetit, Francesc Moreno-Noguer, and Pascal Fua. Epnp: An accurate o(n) solution to the pnp problem. *International Journal of Computer Vision*, 81:155–166, 2009.

[118] Mate Kisantal, Sumant Sharma, Tae Ha Park, Dario Izzo, Marcus M artens, and Simone D'Amico. Satellite pose estimation challenge: Dataset, competition design and results. *IEEE Transactions on Aerospace and Electronics Systems*, April 2020.

[119] Kaiming He, Xiangy Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *arXiv Tech Report*, December 2015.

[120] Yuichi Tsuda, Makoto Yoshikawa, Masanao Abe, Hiroyuki Minamino, and Satoru Nakazawa. System design of the hayabusa 2 - asteroid sample return mission to 1999 ju3. *Acta Astronautica*, 91:356–362, November 2013.

[121] MathWorks Inc. estimateworldcamerapose estimate camera pose from 3-d to 2-d point correspondences. https://ch.mathworks.com/help/vision/ref/estimateworldcamerapose.html, 2020.

[122] Ke Sun, Bin Xiao, Dong Liu, and Jingdong Wang. Deep high-resolution representation learning for human pose estimation. *CVPR 2019*, February 2019.

[123] Tsung-Yi Lin, Michael Maire, Serge Belongie, Lubomir Bourdev, Ross Girshick, James Hays, Pietro Perona, Deva Ramanan, C. Lawrence Zitnick, and Piotr Dollár. Microsoft coco: Common objects in context. *ECCV 2014*, February 2014.

[124] Steven W. Smith. *The Scientist and Engineer's Guide to Digital Signal Processing. Second Edition.* California Technical Publishing, 1999.

[125] Joseph Redmon and Ali Farhadi. Yolov3: An incremental improvement. *arXiv Preprint*.

[126] Nhat-Duy Nguyen, Tien Do, Thanh Duc Ngo, , and Duy-Dinh Le. An evaluation of deep learning methods for small object detection. *Journal of Electrical and Computer Engineering*, April 2020.

[127] Tomáš Hodaň, Dániel Baráth, and Jiří Matas. EPOS: Estimating 6D pose of objects with symmetries. *IEEE/CVF Computer Vision and Pattern Recognition*, 2020.

[128] Raghuraman Krishnamoorthi. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv Preprint: Google WhitePaper*, June 2018.

[129] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. A survey of quantization methods for efficient neural network inference. *Book Chapter: Low-Power Computer Vision Improve the Efficiency of Artificial Intelligence*, Section III.III, June 2021.

[130] Sergey Ioffe. Batch renormalization: Towards reducing minibatch dependence in batch-normalized models. *arXiv Preprint*, February 2017.

[131] Saqib Javed. Hardware-friendly mixed-precision neural networks, March 2021.

[132] Aojun Zhou, Anbang Yao, Yiwen Guo, Lin Xu, and Yurong Chen. Incremental network quantization: Towards lossless cnns with low-precision weights. In *International Conference on Learning Representations 2017*, February 2017.

[133] Zhewei Yao, Zhen Dong, Zhangcheng Zheng, Amir Gholami, Jiali Yu, Eric Tan, Leyuan Wang, Qijing Huang, Yida Wang, Michael W. Mahoney, and Kurt Keutzer. Hawqv3: Dyadic neural network quantization. In *International Conference on Learning Representations 2021*, June 2021.

[134] A Penttil a, V.G. Shevchenko, O. Wilkman, and K. Muinonen. H, g_1, g_2 photometric phase function extended to low-accuracy data. *Planetary and Space Science*, 123:117–125, April 2016.

[135] Stephan Hellmich, Elisabeth Andréa Cécile Rachith, Irureta-Goyena Chang, Bel'en Yu, and Jean-Paul Kneib. Harvesting large astronomical data archives for space debris observations. *2nd NEO and Debris Detection Conference, ESOC, Darmstadt, Germany*, 2(1), February 2023.

Acknowledgements

To Professor, Doctor and Mentor Kazuya Yoshida. Your career has included groundbreaking spacecraft kinematics and contact dynamics work, still referenced today; contributions to in-orbit satellites; and inspiring the next generation through mentoring and guidance. You are so giving. This all began when you charitably spent several hours touring me through your laboratory on a chance encounter in Sendai, Japan. You opened doors for me to work on JAXA deep space imagery and collaborate with a plethora of international partners. Your humble, professional and passionate demeanor made you a wonderful supervisor. I will forever cherish every stimulating one-on-one discussion opportunity I had with you. Thank you for providing me with the opportunity to join the Tohoku Space Robotics laboratory.

To Doctor Mathieu Salzmann. I still cannot believe my luck becoming acquainted with you. Your contributions with Clearspace are future defining and a great motivator for all the wonderful research you support at the EPFL Computer Vision laboratory. The amount of time, guidance and thought you manage to provide individual students with is monumental. Your guidance has greatly shaped my research and career trajectory. I learned so much in so short a time.

For supporting me in Japan and with my research, my heartfelt thanks to

Alex Wright	Dr. David Rodriguez	Dr. Kentaro Uno	Dr. Warley Ribeiro
Saqib Javed	Sara Rodriguez	Til Reichelt	Swapnil Parekh
Sayo Sugimori	Chihiro Watanabe	Dr. Krzysztof Lis	Federico Stella
Chen Zhao	Dr. Stephan Hellmich	Dr. Shreya Santra	Keita Kobashi
Daven Doobay	Dr. Yasuhisa Hirata	Dr. Shingo Kagami	Dr. Toshinori Kuwahara

To my ammi (Rohini), thaththi (Yapa), and malli (Pramodh). Pursuing a PhD and raising a family are both full-time jobs. Thank you for covering the latter when I needed time for the former.

To my mother (Linda), father (Dwayne), and brother (Devon). In a lot of ways, this is your fault. Thank you for your support, love and saying I could do anything. I am not sure I am still astronaut material, but we all know where my love of space began.

To my sons, Bodhi and Bhumi. To be honest, you did not expedite the PhD process, but I love you dearly. I needed the love after long days in the lab.

To my beautiful wife, Upekha. We sacrificed our jobs and support networks to pursue this PhD in Japan. You are the strongest most loving partner. You picked up all the slack. You enable me. Thank you.

November 2023 Tohoku University Sendai JAPAN

Andrew Lawrence Price